conzyx

COMPLETE DOCUMENTATION

Conzyx

Every screen, every feature, every setting. Documented from the source code.

VERSION 3.1.0 · OCTOBER 2026 · CODEAIR SOFTWARE SOLUTIONS

CONTENTS

What is inside

A 1. What Conzyx Is

A 2. System Requirements

A 3. Installation (Overview)

A 4. First Launch: The Startup Journey

A 5. Sign-in and Account Options

A 6. Navigation: The Five Root Screens

A 7. The Projects Screen

A 8. The Workspace Concept

A 9. Home Tab

A 10. Chat Tab

A 11. Changes Tab

B B1. The Agent System: An Overview

B B2. The Agent Tab Bar

B B3. Your API: The Bring-Your-Own-API Tab

B B4. Claude Code Tab

B B5. DeepSeek Harness Tab

B B6. Antigravity CLI Tab

B B7. Codex CLI Tab

B B8. Gemini API Mode (Inside the Antigravity Tab)

B B9. Multi-Agent Engineering Teams

B B10. Session Lifecycle, Foreground Work, and Undo

B B11. Costs and Billing: What the App Actually Says

C Terminal

C Files

C Code Editor

C Preview Browser

C Command Palette

C Global Search

C Pen Testing

D Chapter D1: Skills

D Chapter D1 continued: Skills import pipeline, updates, and the MCP Platform tab

D Chapter D2: Marketplace

D Chapter D2 continued: Marketplace install, updates, upstream tracking, publishing

D Chapter D3: Settings

D Chapter D3 continued: Settings, Environment updates, App updates, Permissions & Safety, Contributors

D Chapter D4: In-app updates

D Chapter D5: Privacy policy summary

E Chapter 1: How Conzyx Works Under the Hood

E Chapter 2: The Linux Runtime: What Gets Installed

E Chapter 3: Build Variants and Editions

E Chapter 4: In-App Updates

E Chapter 5: Android Permissions, Explained

E Chapter 6: System Requirements

E Chapter 7: Backup and Restore

E Chapter 8: Troubleshooting

E Chapter 9: Frequently Asked Questions

1. What Conzyx Is

Conzyx is an AI-powered software engineering workstation that runs natively on Android. It combines a rootless Linux userspace (Ubuntu 20.04 on ARM64, run through PRoot, no root required) with a full mobile IDE, multi-agent AI engineering teams, autonomous GitHub workflows, an Android app factory, and hybrid cloud execution. In one sentence, from the product README: it is a desktop-class AI software engineering workstation for Android.

The app is developed and published by Codeair Software Solutions. Its Android application ID is `com.codeair.mhe`. The build documented in this manual is version 3.1.0 (versionCode 37), built from the `v3` branch of the `sunmughan/Conzyx` repository.

1.1 Two editions: Online and Offline

Conzyx ships as two APK editions that share the same application ID, the same feature set, and the same codebase. Only the runtime delivery differs:

Edition	Approximate size	How it works
Online	~150 MB	Lightweight download. Linux runtime stacks are downloaded on demand from verified releases after install. Best for devices with limited storage. Requires network during first-launch setup.
Offline	~1.2 GB	Fully bundled. All toolchains (core, Python, Android SDK, Claude Code, DeepSeek, Antigravity) ship inside the APK. Works without a network connection after install.

Both editions update the same way: installing a newer APK over the existing one preserves projects and settings, because release APKs are signed with the same permanent release key. The app also checks for new releases on its own and notifies you when an update is available.

1.2 What the app contains

The layers of the app, as shipped in this version:

- **Mobile IDE.** Multi-tab code workspace with pinned tabs and state restoration across Android backgrounding, a built-in file manager, a touch-optimized coding toolbar, a universal command palette, global search and replace, and crash-safe file operations.
- **Rootless Linux terminal.** Real Ubuntu 20.04 ARM64 userspace via PRoot. Node.js, Python, Git, and Clang/LLVM toolchains, with persistent terminal sessions and workspace checkpoints.

- **AI coding assistance.** In-app chat with a multi-provider gateway (Anthropic Claude, DeepSeek, OpenRouter, and OpenAI-compatible endpoints, with automatic failover), intent classification, a token-budget-aware context engine, persistent project memory, MCP (Model Context Protocol) support, and installable skill packages.
- **Multi-agent engineering team.** Ten specialized roles (Manager, System Architect, Frontend, Backend, QA, DevOps, Security Auditor, DBA, Docs Writer, UX Designer) that run real agent sessions through the installed runtimes, coordinated by a DAG task scheduler.
- **Autonomous GitHub workflows.** Issue triage to pull request, PR automation, issue management, release automation, and workflow dispatch from the app.
- **Android App Factory.** AI blueprints, project scaffolding for Jetpack Compose, Flutter, React Native, and HTML5, on-device Gradle APK builds, APK signing verification, and Play Store listing preparation.
- **Cloud hybrid execution.** Authenticated cloud workers for offloading heavy jobs, with per-job cost estimates and rolling 30-day budget caps.
- **AI Developer Marketplace.** A catalog of agents, MCP servers, skills, and starter templates with one-click install, plus a seller center.
- **Safety engine.** Dangerous command interception, sensitive file shielding, granular permission scopes (Once, Session, Project, Always, Deny), an emergency kill switch, and audit logging.
- **Push notifications.** Firebase Cloud Messaging receiver with token registration at startup, in both editions.

1.3 Honesty contract

A deliberate design rule across the codebase is that the app reports verified outcomes only. Unconfigured backends report `NOT_CONFIGURED`, and simulations are labeled `SIMULATED` and never count as success. Where this manual describes a control, it describes only what the code actually does.

2. System Requirements

Requirement	Minimum	Recommended
Android version	9.0 (API 28)	12 or newer
CPU architecture	ARM64 (aarch64)	ARM64
RAM	3 GB	4 GB or more (large builds, multi-agent runs)
Free storage	2 GB	4 GB (projects, build caches)
Root	Not required. The app runs 100 percent in rootless userspace.	

> The device check during first launch enforces the ARM64 requirement. Devices whose supported ABIs do not include ARM64 cannot continue past the compatibility check. The check also reads total device RAM and labels the device "Full mode" at 7.5 GB or more of usable RAM, otherwise "Lite mode". Android version and memory are reported for information; the processor check is the hard gate.

No root, no custom ROM, and no special permissions beyond normal app sandboxing are required. Permissions the app may request later (notifications, microphone for voice input, "install unknown apps" for APK installs and updates, battery optimization exemption for background tasks) are requested in context, at the moment they are needed.

3. Installation (Overview)

Detailed installation steps, variant selection, building from source, and troubleshooting belong to Part E of this manual. The high-level path for the released APKs is:

1. Open the release page for `sunmughan/Conzyx` on GitHub.
2. Download the edition you want: `app-online-release.apk` (lightweight, downloads runtimes on demand) or `app-offline-release.apk` (everything bundled, works offline after install).
3. On the phone, allow **Install unknown apps** for the browser or file manager when prompted.
4. Open the APK and tap **Install**.
5. **Online edition only:** on first launch, keep the app open and on Wi-Fi while it downloads the verified Linux runtime stacks.

To update, install the newer APK over the existing one. Projects and settings are preserved. The app also checks for updates itself and shows an in-app update dialog with download progress when a newer release is available (see the Projects screen section for that dialog).

4. First Launch: The Startup Journey

When Conzyx starts, `MainActivity` creates the `MainViewModel` and renders `PocketDevApp`. Before you reach the normal app screens, the app walks through a staged startup sequence. Each stage has its own full-screen UI. The stages, in order, are:

1. **Checking** - a loading screen while the app inspects its state.
2. **Sign-in gate** - the auth screen appears here on a fresh install (unless you continue as a guest).
3. **Background task setup** - notification and battery permissions in three steps.
4. **Runtime setup prompt** - pick a coding agent and optional toolchains, then install.
5. **Installing** - the runtime stacks download and install, with a detailed progress option.
6. **Initializing** - a loading screen while services come up.
7. **Model setup** - connect the AI provider (or sign in to Antigravity).
8. **Product tour** - a three-slide onboarding carousel.

If anything fails during install or initialization, an error screen offers **Retry** and a way back to the setup prompt. A theme toggle is available on every startup screen, so you can switch light and dark mode before you ever reach the main app.

4.1 Checking and the loading screen

The very first stage is `CHECKING`, shown as a startup loading screen with a pulsing launch experience animation. This is also the only stage during which the sign-in gate is suppressed, so the app can determine its state before deciding whether to show the auth screen.

4.2 The sign-in gate

After checking, if you are not signed in and have not dismissed the auth screen, Conzyx shows the full sign-in screen before any setup continues. (The same screen can be reopened later from Settings, and it appears as a modal if you tap a sign-in entry point while signed out.) Full details of the sign-in options are in Chapter 5. If you choose **Continue as Guest (Offline Mode)**, setup proceeds without an account.

4.3 Background task setup (3 steps)

Next comes the background task setup screen, titled "Prepare for reliable setup". It explains that setup time depends on the toolchains you will choose next, and that you may leave Conzyx in

the background while it works. A three-dot stepper tracks your progress through the steps, and a summary card shows the status of all three permissions at a glance.

Step 1: Task notifications

"See live progress and receive an alert when Claude finishes or needs your attention." Tapping the primary button requests the `POST_NOTIFICATIONS` runtime permission on Android 13 and newer (the app creates its notification channels only after you explicitly tap). If you previously denied the permission, the button changes to **Open notification settings** and takes you to the app notification settings screen. You can also tap **Continue without notifications** to skip. Privacy note shown on screen: only task progress, completion, and error notifications are sent.

Step 2: Background reliability

"Allow Conzyx to continue a task when you lock the phone or switch to another app." The button opens the battery optimization settings so you can set Conzyx to Unrestricted. On device brands known for aggressive background app killing, the text adapts: it names the manufacturer, asks you to also enable Autostart, and adds a separate **Open [brand] Autostart Settings** button. You can tap **Continue without battery exemption** to skip. Privacy note: you remain in control and can stop every task from its notification.

Step 3: Task protection

"Keep the CPU awake only while a visible coding task is running, then release it automatically." On aggressive-killer devices the wording is stronger ("Keep the CPU and network active while tasks run. Make sure battery optimization is set to Unrestricted."). Tapping **Enable and finish** confirms and calls `viewModel.finishBackgroundSetup()`. Privacy note: the screen stays off, protection is capped at 90 minutes, and it stops with the task.

A footer note reminds you that these can be changed later, and that Android may still stop exceptionally heavy work when the device is low on memory.

4.4 Runtime setup prompt: choose your tools

The runtime setup prompt screen has three sections under the headline "Choose your tools" and the line "Ubuntu · Node.js · npm · Git" (the base environment, always included):

Coding agent

You pick one of five coding agents. Each row shows the agent title, subtitle, and download size:

Agent	Subtitle (as shown)	Download size
Claude Code	Anthropic's coding agent · broad provider support	71.8 MB

DeepSeek Harness	Official DeepSeek coding agent · API-key providers	26.5 MB
Antigravity CLI	Google's official coding agent · Google account	39.9 MB
Codex CLI	OpenAI's coding agent · OpenAI API key	150 MB
Your API	Your own API key · OpenAI / Claude-compatible	(none listed)

Your choice is persisted via `viewModel.selectAgent()` and is used both for setup and later in Settings. The default selection is Claude Code.

Optional toolchains

Six optional development stacks can be toggled on or off (the Web stack cannot be toggled off, it is always included):

Stack	What it is for	Installs
Web (JavaScript / TypeScript)	Websites and web apps with HTML, CSS, and JS frameworks	Node.js and npm (already included)
Python	Scripts, automation, data work, and Python backends	python3, pip, venv, and build tools
Android (Java / Kotlin)	Build Android app projects and install them directly on this phone	JDK 17, ARM64 Android SDK 36, Build Tools 35, Gradle 8.14.3, and an offline Maven cache
C / C++	Fast compiled programs, algorithms, and systems code	gcc, g++, make, cmake, gdb
PHP	Websites and apps with PHP, classic sites and Laravel projects	php-cli, common extensions, and Composer
Browser Automation (Chromium and Puppeteer)	Automated browser control, live preview inspection, and web interaction	chromium-browser, puppeteer-core, and pocket-browser CLI

Each stack row shows a download label and the screen shows a setup time estimate based on your selection. Toggling calls `viewModel.toggleDevStack()`.

Install

The bottom button reads **Install Conzyx** on compatible devices (or "Device not supported" otherwise) and calls `viewModel.startRuntimeSetup()`, which moves to the Installing stage. On the online edition this is the step where the app downloads the verified runtime stacks, so stay on Wi-Fi and keep the app open.

4.5 Installing, initializing, and errors

During installation the app can show a detailed runtime installation screen with progress. You can cancel the setup from that screen (`viewModel.cancelRuntimeSetup()`). The initializing stage returns to the loading screen while services start.

If startup fails, the error screen shows the error message, whether the failure looks like an offline problem, and the startup logs. Two actions are available: **Retry** (`viewModel.retryStartup()`) and a back action that returns to the setup prompt (`viewModel.backToSetup()`).

4.6 Model setup

Which model setup screen you see depends on the coding agent you chose:

Provider setup (all agents except Antigravity)

A stepped wizard titled "Set up Conzyx":

1. **Device check.** Brand mark, the headline "Your phone is the workspace", and three check rows: Memory (total GB usable, and whether the device qualifies as "Full mode" at 7.5 GB or "Lite mode" below), Processor (the device ABI, must be ARM64), and Android (the Android release). A notice warns that you should only open projects you trust, because the local Linux environment is a compatibility layer, not a hardened security sandbox. The Continue button is enabled only on ARM64 devices.
2. **Provider choice.** Pick the AI provider for the selected agent. You can also switch the coding agent from here via an agent picker sheet.
3. **Credentials.** Enter the base URL (prefilled with the provider default), model name, API key, and, for DeepSeek Harness with a custom provider, the API protocol (`anthropic-messages`, `openai-completions`, or `openai-responses`, auto-inferred from the URL when possible). Two verification actions are available: discover models against the endpoint and validate the connection. Saving calls `viewModel.finishOnboarding()`, which persists the provider profile and the key in the encrypted credential store.

Antigravity onboarding

If you chose Antigravity CLI, you see "Set up Antigravity" instead. The screen explains that Conzyx runs Google's official `agy` CLI inside its private Linux environment, that Google handles authentication, and that `agy` owns the saved session. The flow:

1. Tap **Sign in with Google** (`viewModel.startAntigravityLogin()`). Status text tracks the login: starting, completing, or error.
2. Google sign-in opens in your browser. Copy the one-time code shown after approval. A **Copy sign-in URL** button copies the authorization URL if the browser did not open.
3. Paste the code into the **Authorization code** field and tap **Complete sign-in** (`viewModel.submitAntigravityCode()`).
4. Once connected, a green confirmation shows the account email ("Connected as …") and a **Continue** button (`viewModel.finishAntigravityOnboarding()`) finishes setup.

A "Use another coding agent" link opens the agent picker if you want to switch. A warning box states plainly: automatic tool approval is enabled for Antigravity, so it can edit project files and run commands without confirmation, though changes remain reviewable in Conzyx.

4.7 Product tour

After model setup completes, first-time users see the product tour (`ProductOnboardingScreen`), a three-slide carousel titled "Conzyx Tour" with a **Skip** button at the top. Each slide shows an icon, a headline, a sub-headline, a highlights card with checkmarks, small badge chips, and a closing quote. The slides are:

1. **"Chat with AI that writes code."** "Describe what you want. The AI writes, runs, and fixes code on your phone." Highlights: ask in plain words and get working code back in seconds; the agent runs builds and tests on the device; it fixes its own errors until the code works. Badges: No setup, On-device, Self-fixing. Quote: "Your pocket pair programmer, available 24/7."
2. **"Preview real apps instantly."** "See your work running in the built-in browser as you build." Highlights: live preview with mobile, tablet, and desktop viewports; inspect elements and read console logs without leaving the app; localhost ports auto-detected from your projects. Badges: Live preview, DOM inspector, Localhost. Quote: "What you build is what you see, the moment you build it."
3. **"Ship to GitHub in one tap."** "Commit, push, and open pull requests without touching a terminal." Highlights: sign in with GitHub once, push from any project; AI writes commit messages and PR descriptions; review diffs and merge from your phone. Badges: One-tap push, AI commits, PR ready. Quote: "From idea to pull request, entirely from your pocket."

Navigation: dot indicators show your position. On the first slide, a prominent full-width **Try it now** button ends the tour immediately. Otherwise, **Next (1/3)** / **Next (2/3)** advance, **Previous** goes back, and the last slide's button reads **Get Started**. Skipping or finishing calls `viewModel.finishAppTour()`, which persists the tour as completed so it never shows again. You can replay the tour any time from Settings (it calls `viewModel.replayAppTour()`).

> Order of operations on a truly fresh install: loading screen, sign-in screen, background task setup, tool picker and install, model setup, then the product tour, and finally the Projects screen. Signing in happens before any setup, and the tour happens after everything is installed and connected.

5. Sign-in and Account Options

Conzyx uses Firebase Authentication for identity. Signing in is optional: you can use the app as a guest, but signing in saves your profile and unlocks account-backed features. The sign-in screen (`AuthScreen`, driven by `AuthViewModel`) is shown automatically after the checking stage when you are signed out, and can be reopened from Settings later.

5.1 What the sign-in screen shows

At the top: the Conzyx logo, the word "Conzyx", and a subtitle that changes with the mode: "Sign in to access your workspace" or "Create an account to save your profile". Below is a card containing:

- **Mode tabs.** A two-segment toggle: **Sign In** and **Create Account**. Switching clears any previous error or success message.
- **Form fields.** In sign-in mode: Email address and Password. In create-account mode: Display Name, Email address, Password, and Confirm Password. The password field has an eye icon to show or hide the password. Pressing Done on the keyboard in sign-in mode submits immediately.
- **Forgot password?** A link under the password field in sign-in mode. It opens a **Reset Password** dialog: enter your email and tap **Send Reset Link** (`viewModel.sendPasswordReset()`). The address is validated before sending, and a success message confirms the email was sent.
- **Submit button.** Reads **Sign In** or **Create Account**, and shows a spinner while the request is in flight. Validation happens first (see 5.3).
- **Social sign-in.** Below an "OR" divider, two circular icon buttons: **Sign in with Google** (one-tap Google sign-in through the AndroidX Credentials API) and **Sign in with GitHub** (OAuth via Firebase, using `github.com` as the provider). Each shows a spinner while its own flow runs. These buttons appear only when the Google sign-in handler is available on the device.
- **Continue as Guest (Offline Mode).** A text button below the card that dismisses the auth screen without signing in.

Errors and confirmations appear as snackbars at the bottom of the screen and are dismissed from state after showing.

5.2 Behind the scenes: the sign-in calls

Every action on the screen maps to a function on `AuthViewModel`, which delegates to the app's `AuthRepository` (Firebase):

Action	Function	What happens
Sign In button	`signInWithEmail(email, password)`	Validates the email and password, then signs in with Firebase email/password auth.
Create Account button	`signUpWithEmail(name, email, password, confirmPassword)`	Validates all fields, creates the Firebase account, and sends a verification email. Shows "Account created! A verification email has been sent."
Google button	`signInWithGoogleToken(idToken)`	Requests a Google ID token from the system credential manager, then signs in to Firebase with it. Cancelling the Google picker is treated as a silent cancel, not an error.
GitHub button	`signInWithGitHub(activity)`	Starts the Firebase GitHub OAuth flow.
Forgot password submit	`sendPasswordReset(email)`	Sends the Firebase password-reset email. Confirms with "Password reset email sent to …".
Continue as Guest	(UI state only)	Dismisses the auth screen for this session without signing in. No account is created.

5.3 Validation rules

- Email must be non-blank and match the Android email address pattern, otherwise "Please enter a valid email address."
- Sign-in requires a non-blank password, otherwise "Please enter your password."
- Sign-up requires a password of at least 6 characters, otherwise "Password must be at least 6 characters."
- Sign-up requires the two password fields to match, otherwise "Passwords do not match."

5.4 Email verification

After creating an account with email and password, Firebase sends a verification email. If your account requires verification, the app shows a non-dismissible **Email Verification Required** dialog over the main screens:

- It tells you a verification email was sent to your address and asks you to check your inbox to unlock all features.
- A status line reads "Auto-verifying in background..." (or "Checking verification status..." while a check is running). The dialog polls your verification status every 4 seconds and also re-checks automatically whenever you return to the app from your email client.
- **I've Verified** manually refreshes the status (`viewModel.refreshVerificationStatus()`). On success it reports "Email verified successfully!", otherwise "Email not verified yet. Please check your inbox."
- **Resend Email** sends another verification email (`viewModel.resendEmailVerification()`).
- **Sign Out** signs out of the unverified account.

You cannot dismiss this dialog by tapping outside it. You must verify or sign out.

5.5 Account management (available to signed-in users)

The `AuthViewModel` also exposes these operations, surfaced in Settings (documented in the Settings part of this manual):

- **Update display name** (`updateDisplayName`): rejects blank names.
- **Change password** (`changePassword`): requires the current password; the new password must be at least 6 characters and match its confirmation.
- **Delete account** (`deleteAccount`): permanently deletes the Firebase account.
- **Sign out** (`signOut`).

6. Navigation: The Five Root Screens

Once startup is complete, the app lands on the **RootScreenHost**: five root destinations switched from a bottom navigation bar. The bar shows an icon and a label for each screen, highlights the active one in the theme primary color with a soft indicator pill, and hides itself while the keyboard is visible so the keyboard gets full width.



Tab	Label	Icon	What it opens
Projects	Projects	Folder	The project list, quick creation, import tools, studios, and update banner. This is the home screen of the app. Covered in depth in Chapter 7.
Agent	Agent	SmartToy	The agent and provider control center: AI provider profiles and API keys, installed coding agents, Antigravity accounts, and the multi-agent team orchestrator. Covered in the Agent part of this manual.
Skills	Skills	Extension	Installed skill packages and the MCP server manager: search and import skills, check for updates, install and run MCP servers, and review the audit log. Covered in the Skills part of this manual.
Market	Market	Storefront	The AI Developer Marketplace: browse and install agents, MCP servers, skills, and templates. Covered in the Marketplace part of this manual.
Settings	Settings	Settings (gear)	App settings: theme, provider, dev stacks, updates, backups, security and permission policies, privacy policy, account management, and replaying the product tour. Covered in the Settings part of this manual.

6.1 Overlay screens: studios and marketplace

Some destinations open as full-screen **overlays** on top of everything else, managed by the `PocketDevNavigator`. Only one overlay is visible at a time; opening one closes the others, and the system back button closes the current overlay. The overlays are:

- **App Factory** (`navigator.openAppFactory()`) - the Android App Factory studio: blueprints, scaffolding, builds, keystores, and Play Store listing preparation.
- **Team Studio** (`navigator.openTeamStudio()`) - the multi-agent engineering team studio, driven by `viewModel.teamOrchestrator`.
- **Cloud Hybrid** (`navigator.openCloudHybrid()`) - the cloud hybrid execution screen, driven by `viewModel.cloudHybridCoordinator`.
- **Marketplace** (`navigator.openMarketplace()`) - the same marketplace screen used by the Market tab.

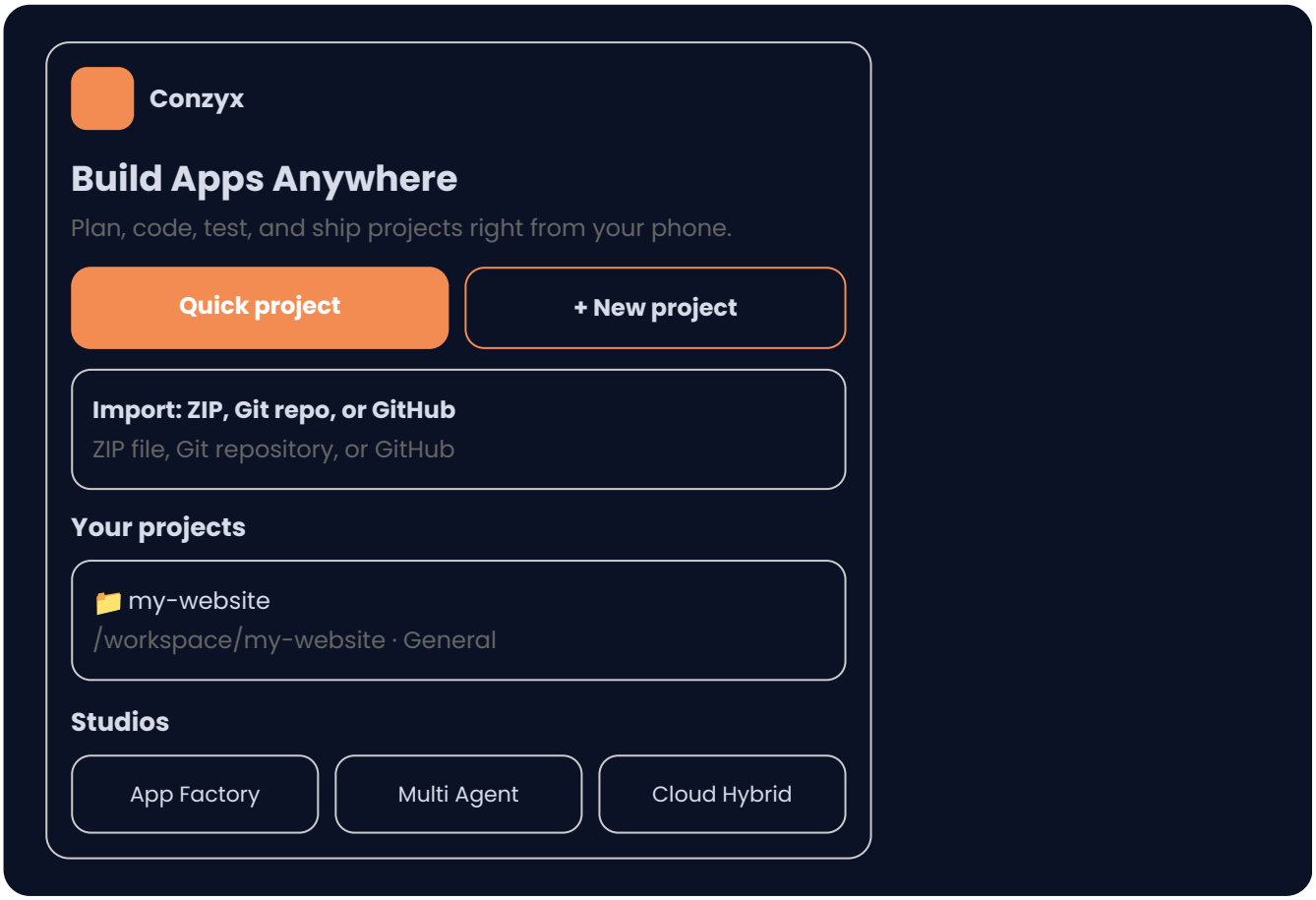
The three studios (App Factory, Multi Agent, Cloud Hybrid) each show a one-time onboarding explainer dialog the first time you open them. Tapping through or exiting marks that studio's onboarding as seen in app preferences, so it never shows again. These overlays are launched from the Studios cards on the Projects screen and from the workspace command palette.

6.2 Global behaviors

- **Toasts.** The app surfaces short status messages (for example, "Task continues in the background") as Android toasts. Each toast is consumed from state after showing, so it appears exactly once.
- **Back handling.** System back is context-sensitive: it closes overlays first, then dialogs and sheets, then returns a workspace tab to Chat, then leaves the project. The exact chain is described in the workspace chapter.
- **Theme.** The app supports light and dark themes, toggleable from startup screens, the terminal, and Settings.

7. The Projects Screen

The Projects screen is the app's home. It is the default root tab, and you return here whenever you leave a workspace. From top to bottom it contains: the app header, the headline, the Quick project and New project buttons, the Import section, the app-update banner (only when an update is available), the "Your projects" list, and the Studios section.



7.1 Header and headline

The top bar shows the Conzyx brand mark and the word "Conzyx". Below it, the headline reads **"Build Apps Anywhere"** with the sub-headline **"Plan, code, test, and ship projects right from your phone."**

7.2 Quick project

The filled **Quick project** button (chat icon) creates a project instantly with no questions asked and opens it immediately. Behind the scenes it calls `viewModel.createQuickProject()`, which:

- Generates a random two-word identity (an adjective plus a noun, for example "bright falcon") with a unique slug, so quick projects are easy to tell apart.

- Creates the project with the description "Quick project workspace", language "General", and kind `QUICK_PROJECT`.
- Creates its workspace directory and saves a first chat titled "New chat".
- Puts it at the top of the project list and opens it, unless another task or terminal command is already running, in which case it shows a toast ("… created. The current task continues in the background; open it when ready.") and leaves it in the list.

7.3 New project dialog

The outlined **New project** button (plus icon) opens the **Create project** dialog with these controls:

- **Project name** text field. If you picked a device folder first (below), its name pre-fills the field.
- **Starter Template** chips. A horizontally scrolling row of every available `ProjectTemplate` (starting with None). Tapping a chip selects it; the selected chip is highlighted. If you pick a template, it is scaffolded into the project right after creation (`viewModel.scaffoldProjectTemplate()`).
- **Select folder from device (Optional).** Opens the Android folder picker. The chosen folder is copied into the new project (a persistable read permission is taken). The dialog then shows a "Binding device folder" card with the folder name and a remove (X) button. During import, `.git`, `.gradle`, `build`, `node_modules`, `.idea`, `.DS_Store`, `__pycache__`, and `.next` directories are skipped, and copying is capped at 3000 files.
- **Terminal folder preview.** Once a name or folder is chosen, the dialog shows the terminal path the project will use, in monospace: `/workspace/<slug>`. The slug is derived from the name by lowercasing, stripping accents, replacing non-alphanumeric runs with hyphens, trimming to 48 characters (falling back to `project` if empty).

Create is enabled only when the name is non-blank or a folder was selected. Tapping it calls `viewModel.createProject(name, folderUri)`, scaffolds the chosen template if any, then closes and resets the dialog.

7.4 Import section

The **Import: ZIP, Git repo, or GitHub** card expands and collapses on tap. It auto-expands while an import or clone is in progress. Inside:

- **Folder** - opens the Android folder picker and imports the selected device folder as a project (`viewModel.importFolderProject()`). While importing, the button reads "Importing…" with a spinner.

- **ZIP** - opens the file picker (`GetContent` with `*/*`) and imports the chosen archive as a project (`viewModel.importZipProject()`). Imports are blocked while another import, task, or terminal command is running.
- **Git URL** - opens the **Clone Git repository** dialog (see 7.5). While cloning, the button reads "Cloning…" with a spinner.
- **GitHub row** - shows **Connect GitHub** ("Sign in to access your repositories") or **GitHub · @username** ("Browse public and private repositories") once connected. Tapping opens the GitHub dialog (see 7.6). If you are connected but the repository list is empty, tapping refreshes it.
- A status line under the buttons shows the current import or clone message (for example, "Reading project archive…").

7.5 Clone Git repository dialog

Paste a public HTTPS repository URL (placeholder: `https://github.com/owner/repository.git`). The dialog explains that the complete Git history and the current branch are kept. **Clone project** calls `viewModel.clonePublicGitRepository(url)` and closes the dialog. Dismissing the dialog never stops the clone: the clone runs detached in the ViewModel scope. While a clone runs, the dismiss button becomes **Cancel clone** (`viewModel.cancelGitClone()`), which is the real stop.

7.6 GitHub dialog

The GitHub dialog walks through the official GitHub CLI device-code flow:

1. **Disconnected.** The message "Sign in with GitHub's official CLI to browse public and private repositories." and a **Sign in with GitHub** button (`viewModel.startGitHubLogin()`).
2. **Starting.** A spinner with "Starting GitHub sign-in…".
3. **Awaiting approval.** The instructions "Enter this one-time code in the GitHub page opened in your browser." A large monospace code card: tapping it copies the code to the clipboard. "Tap the code to copy it. Conzyx will connect automatically after approval." A **Generate new code** button (`viewModel.generateNewGitHubCode()`) issues a fresh code.
4. **Connected.** A search field filters your repositories by name. Each row shows a key icon (private) or code icon (public), the full `owner/repo` name, and "Private/Public · default-branch". A refresh button reloads the list (`viewModel.refreshGitHubRepositories()`). Tapping a repository closes the dialog and clones it (`viewModel.cloneGitHubRepository(repository)`).

While a clone is running, the only action is **Cancel clone**; when connected and idle, the dialog offers **Close** and a **Disconnect** button (`viewModel.disconnectGitHub()`).

7.7 App update banner and dialog

When the app detects a newer release, an orange-tinted banner appears: **"Conzyx {version}"** with "A new update is ready" and an **Update** link. Tapping it opens the **Update to {version}** dialog:

- Release notes (or a default line: "Get the latest improvements and fixes for Conzyx.").
- Download size in megabytes, when known.
- If the app lacks the "install unknown apps" permission, a warning box explains: "Allow 'Install unknown apps' for Conzyx. Without this permission, Android will not install the update." The primary button then reads **Grant permission** and opens the system settings screen for it.
- While downloading: a progress bar with "Downloading X / Y · Z%". While installing: "Download verified. Opening Android installer…". Errors are shown in red.
- Buttons: **Download and install** (`viewModel.installAppUpdate()`), **Cancel** during download (`viewModel.cancelAppUpdateDownload()`), and **Later** to dismiss. Dismissing is always safe: the download and install continue detached, and the Android installer takes over at the end. Reopen the dialog from the banner.

7.8 Your projects list

Under the **"Your projects"** heading, each project is a card. Tapping a card opens the project (`viewModel.openProject()`). A card shows:

- A folder icon tile, the project name (bold, single line), and, for quick projects, the line "Quick project" instead of the description.
- The terminal path in monospace: `/workspace/<slug>` .
- A metadata line: language, then the last-updated date and time (for example, "General · 14 Oct 2026, 11:32").
- If an AI task is running in that project: a small spinner and the blue label **"Task running"**. If a terminal command is running: **"Terminal running"**.
- An overflow menu (three dots) with **Rename project** and **Delete project**.

Rename project opens a dialog with the current name pre-filled; **Save** is enabled only when the name is non-blank and calls `viewModel.renameProject(id, newName)` .

Delete project opens a confirmation dialog: "Delete this project?" with the explicit warning "Its chats, files, attachments, changes, and terminal history will be permanently removed." Confirming calls `viewModel.deleteProject(id)` . Deletion is permanent; there is no trash or undo.

> If the list is empty, a centered card explains: "No projects yet" / "Create a named project or start instantly with a Quick Project."

7.9 Studios section

At the bottom of the Projects screen, the **Studios** section shows three tappable cards that open the full-screen overlays described in Chapter 6:

- **App Factory** (phone icon) - opens the Android App Factory overlay.
- **Multi Agent** (group icon) - opens the multi-agent team studio overlay.
- **Cloud Hybrid** (cloud icon) - opens the cloud hybrid execution overlay.

The first time you open each studio, a one-time onboarding explainer dialog appears; proceeding or exiting marks it seen so it never shows again.

8. The Workspace Concept

A **project** in Conzyx is a named unit of work with its own directory, chat history, files, terminal sessions, and AI-generated changes. When you tap a project card, the app opens the project's **workspace**: a full-screen IDE surface with a top bar, a bottom tab bar, and the active tab's content. The workspace is where all real work happens.

8.1 What happens when you open a project

Tapping a project calls `viewModel.openProject(project)`. Behind the scenes the app:

- Remembers the project as the last active one and configures the agent's working roots for it.
- Restores the persisted editor session (dirty buffers, cursor and scroll positions, open tabs) and the saved terminal sessions.
- Loads the project's chats (creating a "Main chat" if none exist) and reopens the last active chat with its saved messages. A brand-new chat greets you with: "Hi! Tell me what you want to build or change."
- Restores any interactive roadmap from the last message and rebuilds the task scratchpad from it.
- Refreshes the file listing and reloads any AI-made changes that are still pending review.
- Resets per-open state: no live process, no running flags, a fresh terminal command state, and the preview cleared.

The workspace always opens on the Chat tab. If a file was left open in the editor, the code editor surface opens instead (the editor is covered in the IDE part of this manual).

8.2 Read-only mode

If you try to open a different project while an AI task or terminal command is still running in the current one, Conzyx does not interrupt that work. Instead it opens the requested project in **read-only** mode: you can read its chats and history, but the composer is replaced with a "Read-only history" bar. The bar explains the situation ("Another project has a running task. You can read this chat, but cannot send a message.") and, once the other task finishes, offers an **Open** button to activate the project for real (`viewModel.activateReadOnlyProject()`).

8.3 Leaving a project

Tapping the back arrow in the workspace top bar, or the Home tab in the bottom bar, calls `viewModel.closeProject()`:

- If a task or terminal command is running, the workspace closes but the work continues in the background, with a toast: "Task continues in the background" or "Terminal command continues in the background".
- Chat messages are persisted first.
- If the project has zero user messages, zero user files, and nothing running, it is treated as an unused empty project and deleted immediately so it does not clutter the project list.
- Otherwise the project returns to the list with everything preserved: chats, messages, files, attachments, changes, and terminal history.

8.4 The workspace top bar

The top bar is shown on every tab except Browser and Terminal. From left to right:

- **IDE title block.** A phone icon and the word "IDE". Tapping or long-pressing it shows a toast with the project's full name (useful when the name is truncated).
- **Build and run button** (play icon). Appears only when the project is detected as an Android project. It is disabled while a build, AI task, or terminal command is running. If Android 8+ requires the "install unknown apps" permission and it is not granted, tapping opens the system settings for it first. Otherwise it calls `viewModel.buildAndRunAndroidApp()`.
- **Project chats button** (history icon). Opens the **Project chats** dialog: a **New chat** button at the top, then the list of saved conversations. The active chat is highlighted and marked "Current chat" with a green check; others are labeled "Saved conversation". Each row has rename (pencil) and delete (trash) buttons; delete is only offered when more than one chat exists, and deleting asks for confirmation ("All messages and roadmap progress in this conversation will be permanently deleted."). While a task is running you can create new chats but cannot switch to them until the task finishes; the dialog says so explicitly. Creating, switching, renaming, and deleting map to `viewModel.createChat()`, `switchChat()`, `renameChat()`, and `deleteChat()`.
- **Command palette button** (three vertical dots). Opens the workspace command palette (see 8.7).
- **Back button** (back arrow, labeled "Projects"). Leaves the workspace.
- **Running spinner.** A small progress indicator appears at the far right while an AI task is running.

On the Chat tab only, a second line under the title shows the active chat's title (or "Chat") in small grey text.

8.5 The bottom tab bar

The workspace bottom navigation has five items. The bar hides while the keyboard is visible.

Tab	Label	Icon	Behavior
Home	Home	Home	Leaves the workspace and returns to the Projects screen (same as the back button). It is never shown as the selected tab. See Chapter 9.
Chat	Chat	AutoAwesome (sparkles)	The AI conversation. Covered in Chapter 10.
Files	Files	Folder	The project file manager. Refreshes the file list on every entry. Covered in the IDE part of this manual.
Terminal	Terminal	Terminal	The project Linux terminal. Opening it prepares the terminal session. Covered in the Terminal part of this manual.
Browser	Browser	Language (globe)	The embedded browser and visual dev tools. Covered in the Visual Development part of this manual.

Notably, the **Changes** tab is not in the bottom bar. It is reached through the "Files changed" banner on the Chat tab (see 8.6) or from the command palette. The system back button, from any tab other than Chat, returns to Chat first; only on Chat does back leave the project.

8.6 Banners above the tab content

Three informational banners can appear between the top bar and the tab content:

- **Runtime recovery banner** (red-tinted). Appears when the runtime reports a recovery status, for example after the app process was killed mid-task. Tapping the banner expands the full status text. If the task state was preserved, a **Resume task** button appears (`viewModel.resumeInterruptedTask()`); otherwise a spinner shows while Conzyx reconnects without discarding the task.
- **Interrupted session banner** (primary-tinted). "Interrupted Session Found", showing the interrupted task's first 60 characters and the project title. Tapping expands the full prompt.

Resume session resumes it (`viewModel.resumeInterruptedSession()`); the X button dismisses it (`viewModel.dismissInterruptedSession()`).

- **Files changed banner.** Shown on the Chat tab whenever the AI has unreviewed changes. It shows "Files changed" with the total added lines in green and removed lines in red, plus the first few changed file paths. Tapping it jumps to the Changes tab.

8.7 The workspace command palette

The palette (opened from the top bar's three-dot button) is a searchable list of workspace commands. Typing filters by title and keywords. The commands are:

Command	Does
Go to Chat	Switches to the Chat tab.
Go to Files	Switches to the Files tab and refreshes the file list.
Open Terminal	Switches to the Terminal tab and prepares the terminal session.
Review Changes	Switches to the Changes tab.
Open Preview	Switches to the Browser tab.
Refresh Project Files	Reloads the file listing without switching tabs.
Bind Folder from Device	Opens the Android folder picker to bind a device folder into the project.
Android App Factory	Opens the App Factory overlay.
Multi-Agent Engineering Team	Opens the Team Studio overlay.
Cloud + Phone Hybrid	Opens the Cloud Hybrid overlay.
AI Developer Marketplace	Opens the Marketplace overlay.

8.8 Destructive command confirmation

When the terminal flow flags a command as potentially destructive (it can delete files, rewrite Git history, or change the project significantly), the workspace shows a modal dialog: **"Run potentially destructive command?"** with the command rendered in a monospace preview

box. **Run anyway** confirms (`viewModel.confirmProjectTerminalCommand()`); **Cancel** aborts it (`viewModel.cancelProjectTerminalCommand()`).

9. Home Tab

The Home item in the workspace bottom bar is a shortcut, not a content tab. Tapping it calls the same `viewModel.closeProject()` used by the back button: it leaves the workspace and returns to the Projects screen (see 8.3 for the exact behavior, including background continuation and the auto-deletion of unused empty projects).

Two implementation details worth knowing:

- Home is never rendered as the selected tab. The bottom bar's selection logic explicitly excludes it, so the highlight stays on the tab you came from (in practice, you leave the workspace immediately, so you never see it anyway).
- If the workspace's internal tab state were ever set to Home programmatically, it renders the same Chat content as the Chat tab, so there is no separate "Home" screen to get stuck on.

> In short: in the workspace, Home means "go back to Projects". There is no dashboard or summary screen behind it.

10. Chat Tab

The Chat tab is the heart of the workspace: a conversation with the coding agent that can read files, run commands, edit code, and report back. The screen is a column with the message list on top and the composer at the bottom. While an agent is working in this chat, the screen is kept on automatically; the flag is released when the task finishes or you leave the tab.

10.1 The message list

Messages render newest-last in a scrolling list. When a new token or update arrives, the list auto-scrolls to the bottom only if you are already at the bottom and have not scrolled up to read; if you scrolled up, a floating **Latest** pill appears at the bottom-center, and tapping it animates back to the newest item. Switching chats resets this state and jumps to the end.

Empty state

With no messages, no running task, and no pending approval, the list shows a centered empty state: a chat icon, **"Start a conversation"**, and "Ask Conzyx anything: describe what to build, or mention a file to discuss."

Message bubbles

- **Your messages** align right, use the primary-container bubble, take up to 82 percent of the width, and render as Markdown.
- **Assistant messages** align left, use the surface bubble, take up to 92 percent of the width. While a response is still streaming, it renders as plain text (the app deliberately avoids re-parsing growing Markdown on every token to stay responsive); rich formatting is applied once the response completes.
- **Code blocks** in assistant messages offer a run action: tapping it switches to the Terminal tab and prepares the command for execution.
- **Attachments** on a message render as chips with an icon (image or document), the file name, and the formatted size.
- **Copy button.** Every bubble has a copy icon in its action bar. Tapping copies the full message text and toasts "Prompt copied to clipboard" or "Response copied to clipboard"; the icon briefly becomes a green check.
- **"Worked for …".** Assistant messages that did work show the elapsed work time (for example, "Worked for 2m 5s") in the action bar.
- All message text is selectable.

Completed and stopped work blocks

When a task finishes, its activity history collapses into a **work block card**: an expandable list of the steps taken, followed by a summary line reading either **"Task completed · Worked for {duration}"** or **"Task stopped · Worked for {duration}"**.

10.2 Live activity: what the agent is doing right now

While a task runs, a live process panel appears above the newest messages. It shows a headline with the current activity and a live elapsed timer (for example, "Edit · updating MainActivity.kt · 45s"), plus animated thinking dots. Below the headline, each activity step is a tappable row:

- An icon chosen by the type of work: terminal icon for commands and Bash, pencil for Write/Edit/NotebookEdit, document for Read, magnifier for Glob/Grep, sparkles otherwise.
- A one-line summary: the step name plus the first ~105 characters of its detail, with whitespace collapsed.
- An expand chevron. Expanding shows the full detail: commands render in monospace, everything else as Markdown. The "Think" step's detail reads "Reviewed the request and planned the next action" when it is token-based reasoning.

10.3 Approval cards: the agent asks permission

When the agent wants to do something that needs your approval, an approval card appears in the message list: **"Review this action"** with a warning icon, the agent's explanation of what it wants to do, and the affected file paths as bullet points. Three buttons decide:

Button	Effect
Reject	Denies the action once (permission level Deny). Calls `viewModel.answerApproval(false)`.
Session	Approves this kind of action for the rest of the session (permission level Session). Calls `viewModel.answerApprovalWithLevel(Session, true)`.
Once	Approves this single action only (permission level Once). Calls `viewModel.answerApprovalWithLevel(Once, true)`.

10.4 Scratchpad: the task plan at a glance

When the agent is working from a plan, a compact **scratchpad pill** appears above the message list (visible when the workspace controls are expanded): "📋 Scratchpad" with live counts, each with a colored dot: **"N running"** in orange, **"N done"** in green, **"N pending"** in grey. Tapping the pill expands the **Task Scratchpad** card, a scrollable list (up to about 240 dp tall) where each task row shows:

- A status badge: RUNNING (orange), DONE (green), FAILED (red), or PENDING (grey).
- The task title and its detail line.
- The duration in seconds, when recorded.

A collapse button at the top closes the card. If the scratchpad has items while the workspace controls are hidden, the tune button in the composer shows a small orange dot as a reminder.

10.5 Engineering state banner

During autonomous runs, a banner above the messages shows the live engineering state machine status with a one-tap **Rollback to Baseline** button (when a checkpoint exists), which calls `viewModel.rollbackToBaseline()`. The states you may see:

- **Formulating Plan** - the agent is planning.
- **Implementing Changes** - the agent is editing code.
- **Building Project** - a build is running.
- **Running Tests** - tests are running.
- **Analyzing Failure** - a failure is being diagnosed.
- **Self-Healing (n/m)** - automatic fix attempt n of m.
- **Retesting Fix** - verifying the fix.
- **Reviewing Changes** - reviewing the result.
- **Waiting for Approval** - paused for your approval (see 10.3).
- **Verified and Completed** - done and verified.
- **Task Failed** - the run failed.
- **Rolled Back to Baseline** - the workspace was restored to the pre-task checkpoint.

The analyzing state is intentionally hidden by user request: no banner appears while the agent is analyzing the request.

10.6 Emergency kill switch banner

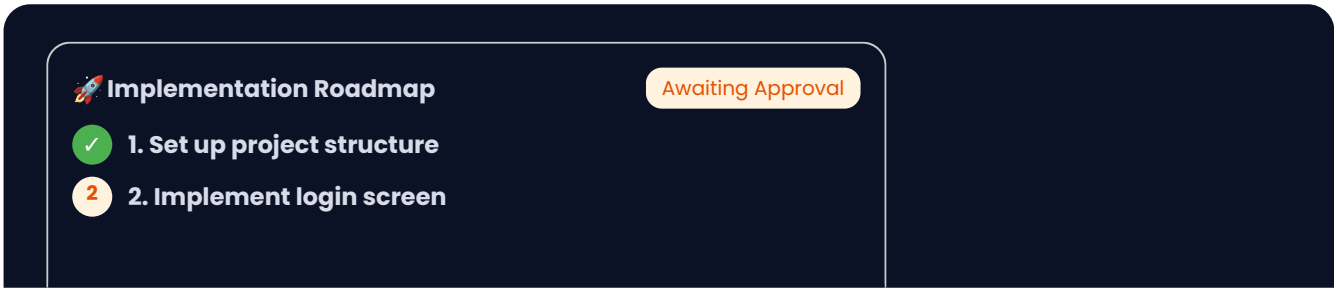
If the emergency kill switch is engaged, a red banner appears: **"Emergency Kill Switch Active"** / "All agent tools and processes are blocked." with a **Reset** button (`viewModel.resetEmergencyKillSwitch()`). While engaged, sending a prompt is blocked with the toast: "Emergency kill switch is ACTIVE: agent sessions are blocked until it is turned off." The kill switch itself is managed in Settings (see the Settings part of this manual).

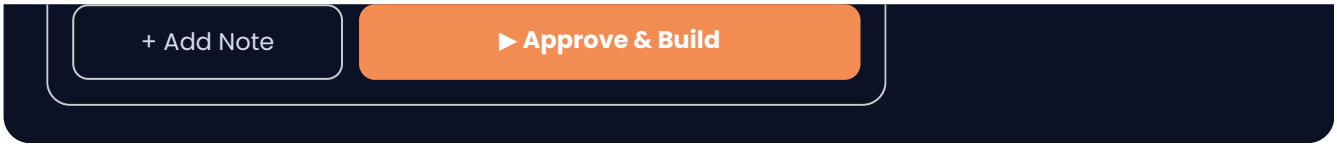
10.7 Roadmap cards: approve the plan before the build

In Unified mode (the default), the agent first proposes an **implementation roadmap** as an interactive card inside its message. The card has:

- A header with a rocket icon (or a green check once approved/completed), the roadmap title (default "Implementation Roadmap"), and a status badge: **"Awaiting Approval"** (orange), **"Approved"** (green), or **"Completed"** (green).
- An expand/collapse chevron. The card starts expanded.
- The roadmap summary text.
- A **steps checklist**: each step shows its number in a circle (green check and strikethrough title once completed, orange number while in progress), the step title, its description, and any affected files as small monospace chips showing just the file names. Every incomplete step has a comment icon.
- **"User Notes and Guidance (n)"** - your annotations on the roadmap, listed as small cards with the author name.
- An **Add Note** button and, until approved, a prominent **Approve and Build** button. After approval the button area reads "Approved and Building" in green; when all steps complete, "All steps completed".

Tapping **Add Note** (or a step's comment icon) opens the comment dialog: **"Comment on: {step title}"** or **"Add Guidance to Roadmap"**. It explains: "Provide custom instructions, edge cases, or adjustments. The AI will strictly follow your guidance when building." Type your note and tap **Add Note** (enabled only when the text is non-blank). Submitting calls `viewModel.addRoadmapComment()`, and **Approve and Build** calls `viewModel.approveAndBuildRoadmap()`.





10.8 The composer: every control

The composer is a rounded input bar pinned above the keyboard. Its placeholder text names the current mode and agent, for example "Unified with Claude Code…". The bar is pill-shaped for short prompts and becomes a taller rounded rectangle (up to 150 dp) once the text wraps or exceeds 50 characters.

Control	What it does
Text field	Multi-line input for your prompt. Typing `@` starts file-mention autocomplete (see 10.9).
Attach (paperclip)	Opens the system file picker (any file type) to attach files to the message. Maximum 5 attachments. Pending attachments appear as chips above the composer, each with a remove (X) button. Sending with only attachments and no text sends the default prompt "Please review the attached files." Calls `viewModel.addChatAttachments()` and `removePendingAttachment()`.
Tune (sliders)	Toggles the workspace controls: reveals the execution mode selector above the composer and the scratchpad pill above the messages. The icon rotates 180 degrees when active and turns orange; an orange dot appears on it when the scratchpad has items but the controls are hidden.
Mic	Voice input. On Android 6 and newer it requests the microphone permission first; if denied, it toasts "Microphone permission is required for voice input". It then launches the system's speech recognizer ("Speak to Conzyx…") and appends the recognized text to the composer. If the device has no recognizer, it toasts "Voice recognition not supported on this device".
YouTube (red play icon)	Opens the YouTube video dialog. If the composer already contains a YouTube URL, it is pre-filled. Analyzing builds a Gemini video-analysis prompt from the video info, your question, and the focus mode, then sends it as a chat message.
Send (up arrow in a circle)	Sends the prompt (`viewModel.sendPrompt()`). Enabled only when the text is non-blank or attachments are pending; grey when disabled, theme-primary when enabled. Clears the composer after sending.
Stop (white square in a red circle)	Replaces the send button while a task is running. Stops the current AI task (`viewModel.stopTask()`).

10.9 Mentioning files with @

Typing `@` followed by text in the composer opens a **"Mention a file"** suggestion panel above the composer. It searches the project's non-directory files for the query (up to 8 results), each showing the file name and its full path. Tapping a result replaces the `@token` with `@full/path` plus a trailing space. When the prompt is sent, the app resolves mentioned paths against the workspace so the agent reads exactly those files (`resolveMentionedWorkspacePaths` in `MainViewModel.sendPrompt`).

10.10 Execution modes

The mode selector (revealed by the tune button) offers three modes, applied to the next prompt you send:

Mode	Icon	Meaning
Plan	Lightbulb	Formulate a detailed architectural plan without making changes.
Build	Build (wrench)	Directly understand the task and implement changes end-to-end.
Unified	Rocket	Plan first with an interactive roadmap for approval before building.

The selected mode is highlighted and persisted per project (`viewModel.setExecutionMode()`); the composer's placeholder always reflects the current mode and agent.

10.11 Behind the scenes: what sendPrompt does

Tapping send calls `MainViewModel.sendPrompt(prompt)`, which runs these checks and steps in order:

1. If the emergency kill switch is engaged, it shows a toast and does nothing.
2. If the selected agent is Antigravity and you are not signed in to Antigravity, it toasts "Sign in to Antigravity from Settings before starting a task." and does nothing.
3. If the agent is DeepSeek Harness and the provider is Claude (subscription login), it toasts that Claude subscription login is not supported by DeepSeek Harness and asks you to pick a key-based provider in Settings.
4. If the prompt is blank with no attachments, or a task is already running, the send is ignored.
5. It trims the prompt (defaulting to "Please review the attached files." when only attachments were added), resolves `@` file mentions, and updates the chat title from the request text.

6. It appends your message to the chat, clears pending attachments, sets the running state, seeds the live process with a "Think" activity item, builds the initial scratchpad, and records the task start time.
7. It starts the real work through `agentOrchestrator.startTask()` (project ID, workspace directory, your goal) and `agentSessionController.startSession()`, which streams tool activity, reasoning, file changes, and the final response back into the chat UI you see.

10.12 Read-only chat bar

When viewing another project's chat in read-only mode (see 8.2), the composer is replaced by a bar with a history icon, the title **"Read-only history"**, an explanation of why you cannot send, and an **Open** button once the blocking task has finished.

11. Changes Tab

The Changes tab is where you review everything the AI modified in the project. It is intentionally not in the bottom tab bar: you reach it by tapping the **"Files changed"** banner on the Chat tab (which shows total added and removed lines) or through the workspace command palette's **"Review Changes"** command. The pending changes are loaded from the runtime when the project opens (`activeRuntime().loadPendingChanges(projectId)`), so unreviewed changes survive app restarts.

11.1 Header and empty state

At the top, a header card reads **"Changes"** with the sub-line **"Review everything the AI changed."** If nothing has changed, the tab shows an empty state: a code icon, **"No changes yet"**, and "Ask Conzyx to update your project."

11.2 Per-file change cards

Each changed file is a card with a summary row:

- A document icon and the file path (single line, truncated in the middle if long).
- Under the path, a hint: "Tap to review diff" (or "Hide line-by-line diff" when expanded).
- An **open-file** button (open-in-new icon) that opens the file in the code editor.
- The addition count in green (`+n`) and the deletion count in red (`-n`).

Tapping the card expands the **line-by-line diff**: a horizontally scrollable monospace view where each line shows the old line number, the new line number, a marker, and the text. Markers follow the familiar convention: `+` for added lines (green-tinted background), `-` for removed lines (red-tinted background), a space for unchanged context lines, and `·` for informational lines (such as hunk headers). The colors adapt to the light and dark themes.

At the bottom of an expanded card, two buttons act on that file only:

Button	Effect
Undo file	Reverts that file's AI-made changes (`viewModel.undoFileChange(path)`) and collapses the card.
Keep file	Accepts that file's changes, removing them from the pending review list (`viewModel.keepFileChange(path)`) and collapses the card.

11.3 Undo task / Keep changes

When at least one change is pending, a final row at the bottom of the tab offers two project-wide actions:

Button	Effect
Undo task	Reverts all of the last task's changes (`viewModel.undoLastChanges()`).
Keep changes	Accepts all pending changes and clears the review list (`viewModel.keepLastChanges()`).

> Review discipline: the Changes tab is the counterpart to the agent's autonomy. Approvals (Chapter 10.3) gate individual actions while they happen; the Changes tab lets you inspect the final diffs and roll back per file or wholesale after the fact.



B1. The Agent System: An Overview

The **Agent** screen is Conzyx's dedicated AI connection hub. It replaced the old dashboard Terminal tab (Terminal now lives as a button on the project and workspace screens). Everything on this screen answers three questions: which coding agent runs your tasks, how it authenticates, and whether it is healthy right now.

Conzyx ships **five agent engines**. Four of them are real coding-agent binaries installed into the phone's private Linux runtime: **Claude Code** (Anthropic's coding agent, 71.8 MB), **DeepSeek Harness** (the official DeepSeek coding agent, 26.5 MB), **Antigravity CLI** (Google's official coding agent, 39.9 MB), and **Codex CLI** (OpenAI's coding agent, 150 MB). The fifth, **Your API**, is a pure bring-your-own-API chat channel: no binary, no install, no updates. It talks directly to any OpenAI-compatible or Claude-compatible endpoint with your own key.

Each coding agent installs independently, on demand, over a shared core runtime. Only one agent is **active** at a time for chat, but you can install several and switch between them by tapping their tab. The active agent's name and model appear under the "AI Agent" title in the top bar, next to a live status pill.

The status pill

The top-right of the top bar always shows a small status pill that reflects live connectivity:

Pill	Meaning
Online (green dot)	The agent answered its last health check. For Antigravity this means you are signed in and the hello probe succeeded.
Testing… (orange dot)	A connectivity test is running right now.
Attention (red dot)	Something needs your attention: not signed in, or the last test failed.
Not tested (grey dot)	No connectivity test has run yet for the current key-based provider.

For Antigravity the pill is driven by the Google sign-in state and the "Test connection" probe; for key-based providers it is driven by the API ping status.

Where your secrets live

The footer of the Agent screen states the rule plainly: "API keys encrypted in Android secure storage · Terminal is accessible from any project". Provider keys are kept in the Android secure keystore vault. The Gemini API key (for Gemini API mode, covered in chapter B8) is kept in a separate encrypted store, `gemini_api_secure_prefs`, protected by an AES256-GCM master key. Keys are passed to agent processes through their environment, never written to files or command lines.

> **Safety first:** the full-access agents (Claude Code, DeepSeek Harness, Antigravity CLI, and Gemini API mode) run shell commands and edit files inside your project workspace. Before the first launch of a task, the app asks for your explicit approval through a launch gate that names exactly what the agent is allowed to do. Per-tool approvals then pass through the app's safety gateway (kill switch, path audit, and policy engine). The Antigravity card carries the footnote "Automatic tool approval · Runs inside the private Linux workspace".

B2. The Agent Tab Bar

The top of the Agent screen is a row of square agent tabs. The rules that govern it:

- **Your API is always first.** It is the hero tab: it keeps its word mark ("Your API", 11sp, orange) instead of an icon, and when it is not selected it carries an orange border so it stands out. The tab order is: Your API, then your primary agent (Claude Code by default), then the remaining agents in fixed order (Claude Code, DeepSeek Harness, Antigravity CLI, Codex CLI). The row scrolls horizontally if needed.
- **Equal spacing, square tiles.** Every tab is a 56 dp square. The selected tab gets a white tile behind its icon; unselected tabs are transparent.
- **Update badge.** When an update is available for an agent, an orange 6 dp dot appears at the tab's top-right corner.
- **Tapping a tab** switches the viewed configuration card. If that agent is installed, tapping also makes it the active agent. Your API needs no install, so it is always available.
- **While an install is running**, all tabs dim to 40% opacity and taps are disabled so you cannot switch mid-install.



Install cards

Tapping the tab of an agent that is not installed shows its install card:

- Title: "<Agent name> is not installed"
- Body: "Install its <size> agent package to use it with your existing projects." The sizes come from the agent registry: Claude Code 71.8 MB, DeepSeek Harness 26.5 MB, Antigravity CLI 39.9 MB, Codex CLI 150 MB.
- A full-width **Install <Agent name>** button starts the download.

While installing, a progress card replaces the button area. It shows a spinner and a message ("Installing <Agent name> binary…"), a linear progress bar, the downloaded and total byte counts ("12.4 MB / 71.8 MB"), the percentage, the current download speed ("3.1 MB/s"), and a **Cancel install** text button.

Inactive installed agents

If an agent is installed but is not the active one, its tab shows a card that reads "<Agent name> is installed" with the body "Make it your active agent to configure it here." and a **Make <Agent name> active** button. The provider configuration card always belongs to the active agent, so this card keeps the screen honest instead of showing a blank configuration.

Runtime updates

Below the configuration card sits the **Runtime updates** block, and it is strictly per-tab: each tab shows only its own agent's update checker, never another agent's.

- The block never claims "up to date" without a check. Before the first check it says "Tap Check updates to see if a newer version is available". For an uninstalled agent it says "<Agent name> is not installed - install it first, then check for updates". If a check fails it shows the reason. While a task, install, or update is running, the button is disabled and explains why.
- The **Check updates** link (with a refresh icon) runs the check. For Your API it is hidden entirely; the text reads "Custom uses your API directly - no runtime to update".
- When an update is found, a highlighted card shows the agent name, the version jump ("v1.2.3 → v1.4.0"), an orange "UPDATE" label, and an **Update <Agent name>** button. While updating it shows the agent's update message, a progress bar, and a **Cancel update** button.

B3. Your API: The Bring-Your-Own-API Tab

Purpose. "Your API" is the hero tab and the heart of Conzyx's BYOK philosophy: plug in any API key and use the app's full chat power with any provider, with no download and no install. The card's subtitle says it plainly: "Plug in any API key and use the full agent power of the app with any provider. No download needed."

What it is not. Your API is direct API chat. It answers through the model you configure, but it does not edit files, run terminal commands, or use tools. The card's own footnote says so: "Custom chats directly via your API. For code execution (file edits, terminal), install Claude Code or DeepSeek." It is also deliberately excluded as a fallback engine for multi-agent teams, for the same reason: letting it run an engineering task would mark the task completed with zero code written.

Setting it up

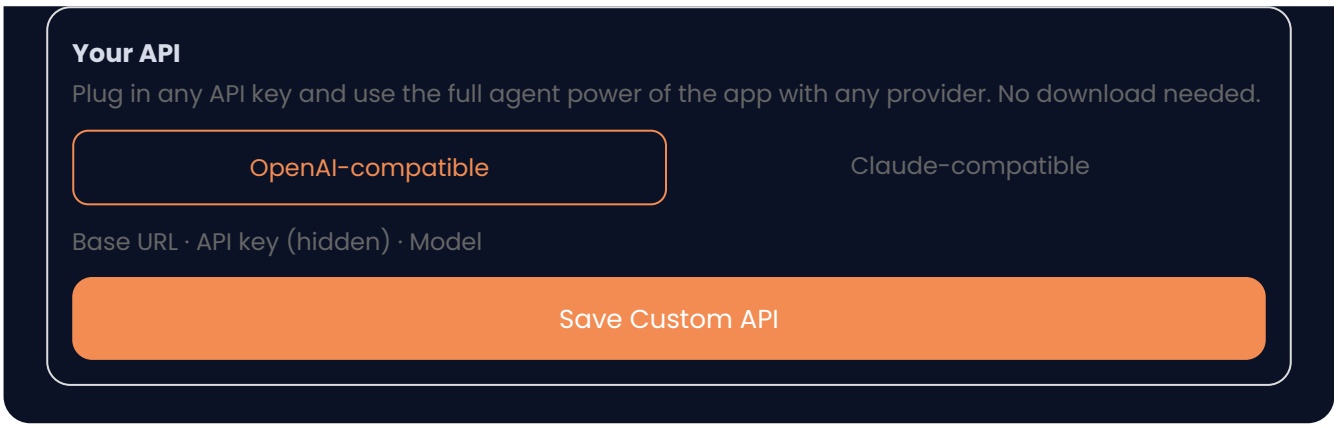
The card contains exactly five controls:

1. **API type selector** (segmented, two options):

Option	Meaning	Request endpoint used
OpenAI-compatible	Default. For OpenAI-style endpoints.	`<base-url>/chat/completions`
Claude-compatible	For Anthropic-style endpoints.	`<base-url>/messages`

   The placeholder text of the Base URL and Model fields changes with the selection (OpenAI: `https://api.openai.com/v1` and `gpt-4o-mini`; Claude: `https://api.anthropic.com` and `claude-sonnet-4-6`).
2. **Base URL**: the provider endpoint, for example a gateway or a self-hosted compatible server.
3. **API key**: password-masked field with a show/hide eye icon. The button stays disabled until the key is non-blank.
4. **Model**: the model ID to call on that endpoint.
5. **Save Custom API** button: enabled only when base URL, model, and key are all non-blank. On success it shows a check and "Saved". If the keystore write fails, the failure reason is shown under the button instead of a false "Saved".

Your API card (mock)



How it runs behind the scenes

The `CustomAgentBridge` class drives this tab. On each message it:

- Verifies that an API key, a base URL, and a model are all saved. If any is missing, the session fails with a message pointing back to the card ("No API key is saved. Add your key in Agent → Custom.", and the equivalents for endpoint and model).
- Sends the conversation (the last 20 messages plus your new prompt) as a streaming request: `POST <base-url>/chat/completions` with `stream: true` and `max_tokens: 4096` for OpenAI-compatible endpoints (Bearer token header), or `POST <base-url>/messages` for Claude-compatible endpoints (`x-api-key` header plus `anthropic-version: 2023-06-01`).
- Streams the reply live. Deltas arrive over server-sent events and are emitted to the chat as they come, so you watch the answer grow. If the endpoint ignores `stream=true` and returns plain JSON, the bridge falls back to reading the whole response and emitting it in chunks.
- Retries one transient failure: a 429 or 5xx on the initial request is retried with backoff before the error path runs.
- Sends up to 20 previous chat messages as history, mapping the app's user/assistant roles to the API's roles.

There is no binary, no workspace, no checkpoint, and no approval channel: `respondToApproval` is a no-op, and undo/accept operations always return false/empty. This is chat, not code execution.

Stopping

Every streaming connection is tracked per session. Tapping stop disconnects the HTTP socket immediately, ending the request mid-stream. A session stopped this way reports **Completed**, not Failed, because the interruption was yours.

Good to know: the Codex CLI tab also stores its OpenAI key, but under its own vault slot ("OPENAI"), so saving a Codex key can never overwrite your Your API key, and vice versa.

B4. Claude Code Tab

Purpose. Claude Code is Anthropic's coding agent, and in Conzyx it is the broadest engine: it supports the widest set of providers (everything except OpenCode Zen and the dedicated OpenAI provider entry). It runs the official `claude` CLI inside the Linux guest with full agentic powers: file reads and edits, shell commands, and streamed reasoning.

Providers you can attach

The **AI provider** card opens with a summary row showing the current provider's title and subtitle. Expanding it lists the providers Claude Code accepts:

Provider	Subtitle (as shown)	Default model
Claude subscription	Pro, Max, Team or Enterprise	default
Anthropic API	Usage billed through Console	claude-sonnet-4-6
OpenRouter	Use your OpenRouter API key	~anthropic/claude-sonnet-latest
DeepSeek	Use your DeepSeek API key	deepseek-v4-flash
Kimi	Anthropic-compatible endpoint	kimi-k2.6
NVIDIA NIM	OpenAI-compatible models hosted by NVIDIA · free tier available	nvidia/llama-3.1-nemotron-70b-instruct
Custom API	Anthropic-compatible endpoint	(you set it)

Switching providers resets the base URL, model, gateway protocol, OpenRouter options, keys-in-progress, and status messages to that provider's defaults.

Endpoint and protocol

The **Endpoint & protocol** row (titled "Custom API settings" for the Custom API provider) expands to:

- **Base URL** field. For fixed-URL providers (OpenCode Zen, NVIDIA NIM) the field is read-only with the supporting text "Fixed by <provider>". For Custom API, the helper text reads "Enter the provider endpoint, then add its API key under Credentials."

- **OpenRouter-only options**, shown for the OpenRouter provider:
  - **Provider order**: optional comma-separated OpenRouter provider slugs (placeholder: "exacto, together"), tried in order. Blank keeps automatic routing.
  - **Allow provider fallbacks**: switch. "Turn off to use only the providers listed above." It is enabled only when a provider order is entered.

Changing the base URL, model, or protocol clears discovered models and per-key connection statuses, so stale results never linger.

Claude subscription token

Selecting the **Claude subscription** provider changes the card: instead of endpoint fields it shows the instruction "Run `claude setup-token` on a computer signed in to your Claude subscription, then save the generated token below." The Credentials row becomes **Subscription token** ("No subscription token saved" when empty), and the action button becomes **Save subscription token**. Saving it stores the token and tells you to send a message to verify your subscription.

Model and access

The **Model & access** section has a link at the top right that reads **Discover models** when nothing is discovered yet, **Discovering…** while working, or **"<n> models"** after a discovery. Tapping it calls the provider's model-list API (you need a key saved first, except for providers that support public discovery such as OpenRouter and OpenCode Zen, which work without one).

The **AI model** row opens the **Available Models** sheet:

- Header shows the provider title and a live count ("3 of 24"), plus a refresh icon to re-discover.
- Search box with placeholder "Search model or type custom ID". Filtering matches the model ID or display name.
- If nothing is discovered yet, the sheet offers **Discover Models from API** and a **Recommended Models** list curated per provider (for example DeepSeek shows deepseek-chat "DeepSeek V3.2 (chat)", deepseek-reasoner "DeepSeek R1 (reasoning)", deepseek-v4-flash "DeepSeek-V4 Flash (fast)"; NVIDIA NIM shows the Nemotron, Llama, Mistral, Qwen, DeepSeek, Gemma and Phi options).
- Type any ID in the search box and a **Use custom model ID** card appears, letting you set a model the provider never advertised.
- Models flagged free carry a green **FREE** badge.

Tapping a model sets it and closes the sheet.

Credentials

The **Credentials** row summarizes the active key ("<name> · Active") and its last check result (Verified, Failed, or Checking). Expanding it shows:

- **Saved keys (n)** with an **+ Add key** / **Cancel** toggle.
- Each key row shows its name and "Active" or "Tap to activate". Tapping a row activates that key. A trash icon removes it. Under a key you may see its individual connection status (a spinner for "Checking connection…", a green check with the verified message, or a warning with the failure message and the provider's own message).
- The add-key form has **Key name** and **API key** fields. The name field is smart: if you paste a value that looks like a key (starts with `sk-` or `ant-`, or is longer than 30 characters with no spaces) while the key field is empty, the app routes it into the key field automatically and names it "<Provider> Key". The key field is password-masked with a show/hide eye.
- **Save API key** is enabled only when both name and key are filled.

The **Test connection** button at the bottom is enabled only when an API key is present, no discovery or validation is running, and (for non-subscription providers) the base URL and model are set. For the subscription provider the button reads **Save subscription token** and shows "Saving token…" while working. On success the profile is saved and a verified status is recorded against the active key; on failure the message and the provider's own message are shown.

How a session runs behind the scenes

The `ClaudeRuntimeBridge` class drives Claude Code. The flow:

1. A session ID is created and **SessionStarted** is emitted; a foreground notification appears ("Starting Claude Code…").
2. The secret is fetched from the vault. If none is saved, the session fails with "No API key is saved for <provider>." (or the subscription-token variant).
3. The stop action is wired, the workspace is prepared, and a checkpoint snapshot of the project is taken before anything runs.
4. For OpenAI-protocol providers a local format gateway is started; for OpenRouter with a custom provider order, a routing gateway is started. Both are closed when the session ends.
5. The CLI is launched as: `claude --bare -p <prompt> --output-format stream-json --include-partial-messages --verbose --model <model> --max-turns 25`. The prompt is wrapped with conversation history and the project workspace path.

6. Output is read as newline-delimited JSON events and mapped onto live events: text deltas stream into the chat, thinking blocks surface as reasoning progress, tool uses surface as ToolStarted/ToolCompleted, and permission requests are routed through the app's safety gateway (kill switch, path audit, policy engine; a "prompt" verdict becomes a deny in headless mode).
7. A provider error detector watches every line: 401, 403, 429, expired, quota, or rate-limit signals fail the session with "The provider rejected the saved API key."
8. When the process exits, changed files are diffed against the pre-run snapshot, the changed paths are saved, and a FilesChanged event carries per-file diffs you can review, undo, or accept file by file.
9. The turn is capped at 25 model turns (`--max-turns 25`).

Claude Code supports session recovery, meaning a conversation can resume in the same project context.

Stopping

Stop destroys the CLI process, waits half a second, and force-destroys it if it is still alive. Pending permission requests are answered "deny" on the way out, and the session reports "Stopped by user".

> **Billing, as the app states it:** Anthropic API usage is "billed through Console"; the Claude subscription entry covers "Pro, Max, Team or Enterprise" plans; OpenRouter, DeepSeek, Kimi, NVIDIA NIM (which notes a "free tier available") and Custom API all say "Use your <provider> API key", meaning you pay that provider directly under its own terms. Conzyx itself adds no charge on top.

B5. DeepSeek Harness Tab

Purpose. DeepSeek Harness is the official DeepSeek coding agent, a 26.5 MB package, and Conzyx drives it in headless one-shot/SDK mode. Its superpower in this app is provider flexibility: it can run against DeepSeek's own API, Anthropic's API, OpenRouter, Kimi, the OpenCode Zen gateway, NVIDIA NIM, or any custom Anthropic/OpenAI-compatible gateway, with the wire protocol selectable per provider.

Providers you can attach

Provider	Subtitle	How the harness reaches it
DeepSeek	Use your DeepSeek API key	Native route `deepseek-official`; key via `DEEPSEEK_API_KEY`
Anthropic API	Usage billed through Console	Custom route `mh-anthropic`, anthropic-messages
OpenRouter	Use your OpenRouter API key	Custom route `mh-openrouter`, anthropic-messages
Kimi	Anthropic-compatible endpoint	Custom route `mh-kimi`, protocol you pick
OpenCode Zen	Models through the OpenCode Zen gateway	Fixed route `opencode-zen`, openai-responses, fixed base URL
NVIDIA NIM	OpenAI-compatible models hosted by NVIDIA · free tier available	Fixed route `nvidia-nim`, openai-completions, fixed base URL
Custom API	Anthropic-compatible endpoint	Custom route `mh-custom`, protocol you pick

Custom (non-native) routes share one key environment variable, `MH_DSH_API_KEY`; the native DeepSeek route uses `DEEPSEEK_API_KEY`. The route, model, and custom gateway are written into the harness's `settings.yaml` (under `/root/.dsh` in the guest) before launch.

Gateway protocol picker

For the protocol-capable providers (Kimi, OpenCode Zen, NVIDIA NIM, Custom API) the Endpoint & protocol section shows a **Gateway protocol** radio list with three wire formats:

- `anthropic-messages`
- `openai-completions`
- `openai-responses`

OpenCode Zen and NVIDIA NIM have a **fixed protocol** (openai-responses and openai-completions respectively) and a fixed base URL, so the picker and the URL field are locked with "Fixed by <provider>". For a Custom API endpoint, typing a base URL auto-suggests the protocol: a URL ending in `/responses` suggests openai-responses, one containing `/anthropic` or `api.anthropic.com` suggests anthropic-messages, one ending in `/v1` suggests openai-completions, and anything else defaults to anthropic-messages. The picker stays editable because a URL alone cannot prove a gateway's wire format.

The OpenRouter provider-order and allow-fallbacks controls described in B4 are also available here.

One hard limit

The Claude subscription login has no DeepSeek Harness equivalent. If you try to run the harness with the Claude subscription provider, the session fails immediately with: "Claude subscription login is not supported by DeepSeek Harness. Pick a key-based provider in Settings."

How a session runs behind the scenes

The `DshRuntimeBridge` class drives the harness:

1. It fails fast if no API key is saved, or if the provider is the Claude subscription login (see above).
2. It asks for your explicit launch approval through the full-access launch gate. The operation is named `dsh --profile sdk (DSH_PERMISSION_MODE=danger-full-access)` and the explanation warns that the harness "can run any shell command and read, write or delete any file in the project workspace without asking again. Approve only if you trust this task."
3. The harness is launched headless as `/usr/local/bin/dsh --profile sdk` with `DSH_PERMISSION_MODE=danger-full-access` in its environment, plus the route's key variable.
4. The bridge speaks JSON-RPC over the process's stdin/stdout: `initialize` (working directory, provider route, model), then `session/prompt` with the task, then `shutdown` when

the run reports it is no longer running. Reasoning deltas stream as reasoning summaries, tool calls as ToolStarted/ToolCompleted (Bash, Read, Glob, Grep, Write, Edit), and assistant text deltas stream into the chat.

5. A checkpoint snapshot is taken before the run; changed files are diffed afterwards with the same review/undo/accept flow as Claude Code.
6. Authentication-style failures (401/403/429, invalid key, expired, quota, rate limit) surface as "The provider rejected the saved API key."

DeepSeek Harness also supports session recovery.

Stopping

Stop destroys the harness process, force-destroys it after half a second if needed, aborts any pending launch approval, and reports "Stopped by user".

B6. Antigravity CLI Tab

Purpose. Antigravity is Google's official coding agent (39.9 MB package). Unlike the key-based tabs, it authenticates with your **Google account** through OAuth, and the app's own Gemini API mode lives inside this tab as an alternative route (chapter B8).

The card is headed **Google account** (20sp bold). The Gemini API section sits above it, separated by a divider.

Signing in with Google

When signed out, the card shows a single **Sign in with Google** button. Tapping it starts agy's official interactive OAuth flow inside a private terminal on the phone. The app drives only the flow; it never reads the resulting credentials. The status moves through:

State	What you see
Starting	Progress bar, "Starting Google sign-in…"
Awaiting code	**Copy Google Sign-in URL** button (copies the authorization URL to the clipboard), a **One-time authorization code** field, and a **Complete Sign-in** button (enabled once you paste the code)
Completing	Progress bar, "Completing Google sign-in…"
Signed in	Account controls (below)
Error	**Reconnect with Google** button with the failure reason

Behind the scenes the app answers the terminal's setup screens for you with the safe defaults: it keeps the official terminal color scheme, selects inline rendering for the captured terminal, **opts out** of the optional interaction-data collection on the privacy screen, and trusts the private authentication workspace folder. Once Google's one-time code is submitted, the CLI finishes and the app detects the signed-in screen, extracts your account email from the terminal output, and records the account.

> **Credential hygiene:** the token file is written by agy itself into the app's private credential area. Conzyx never reads, copies, or logs its contents. "Sign out all" deletes the official token file plus every per-account token folder; if any deletion fails, logout aborts and tells you plainly that your account remains connected, rather than leaving you believing you are signed out when you are not.

Multiple Google accounts

You can connect more than one Google account. When at least one account exists, the card shows **Connected Google Accounts (n)** with one row per account:

- The active account is highlighted with a green check; inactive accounts show a grey dot.
- Tapping an email copies it to the clipboard.
- An account that hit its quota shows a red **Quota Limit Hit** badge.
- Inactive accounts have a **Use** button to switch to them; every row has an X to remove that account.
- When signed in you also get **Add Account** (starts a fresh OAuth flow while stashing the current credential first) and **Sign out all**.

Below the list is the **Auto round-robin accounts** switch (on by default): "Automatically switch to next connected Google account when quota or rate limit is reached." When the active account hits a quota or rate limit, the app switches to the next usable account automatically. The order it tries: first an account that is neither quota-exhausted nor auth-expired with valid credentials; then an account whose quota was hit at least 60 minutes ago; then the account with the oldest exhaustion time. Auth-expired accounts are skipped in every pass.

If no accounts are stored, a fallback row shows a green "G" icon, the account email (or "Not signed in"), "Connected with Google" or "Required for Antigravity", and a **Disconnect** link when signed in.

Active Intelligence Model

Once signed in, the card shows an **Active Intelligence Model** tile with a **Sync** link that refreshes the model list from the CLI. Tapping the tile opens the **Select Model** sheet ("<n> available for Antigravity") with a search box ("Search model or series") and a refresh icon. Each row shows the friendly name (for example "Gemini 3.8 Flash (High)"), the raw model ID in monospace, and a tier badge:

Badge	Shown for
Recommended	Gemini 3.8 models
Stable	Gemini 3.6 models
Pro Reasoning	Gemini 3.1 Pro models
Anthropic	Claude models (for example Claude Sonnet 4.6, Claude Opus 4.6 Thinking)

Open Source	GPT-OSS models (for example GPT-OSS 120B)

When nothing has been synced yet, the sheet offers a built-in list: gemini-3.8-flash-high/medium, gemini-3.6-flash-high/medium/low, gemini-3.1-pro-high/low, claude-sonnet-4-6, claude-opus-4-6-thinking, and gpt-oss-120b-medium. Selecting a model stores it; if the model ID itself encodes an effort level (high/medium/low), the Reasoning Depth setting follows it.

Reasoning Depth

A segmented Low / Medium / High capsule with a caption that changes with the selection:

Setting	Caption
Low	Fast response · light reasoning
Medium	Balanced speed & logic
High	Maximum reasoning depth

Technical note, exactly as the code documents it: agy's complete model configuration IDs (like `gemini-3.8-flash-medium`) already own their effort, so passing a second, different `--effort` would make the CLI reject an otherwise valid model as conflicting. The explicit model ID therefore wins; `--effort` is only used with agy's default model selection. If the current model does not offer the effort level you pick, the app shows a toast: "This model does not offer <Low/Medium/High> reasoning".

Test connection

The outlined **Test connection** button (orange border) runs a lightweight hello probe: it launches agy, sends "Reply with exactly: ok", and returns the reply. No foreground service, no checkpoints, no saved conversation. It has an internal 90-second timeout; if agy does not answer you get "Antigravity did not answer. Try again." The result line shows a green check with the reply or a warning with the failure reason, and the top-bar pill reflects it (Testing… while running, Online or Attention after).

How a session runs behind the scenes

The `AntigravityRuntimeBridge` class drives the CLI:

1. If Gemini API mode is enabled with a saved key, the session takes the Gemini API route instead (chapter B8). Otherwise the bridge requires the Antigravity CLI to be installed.
2. It asks for your explicit launch approval through the full-access launch gate. The operation is named `agy --dangerously-skip-permissions` and the explanation warns that Antigravity "can run any shell command and read, write or delete any file in the project workspace without asking again. Approve only if you trust this task." This is the gate behind the card's "Automatic tool approval" footnote: tool calls themselves have no approval channel in headless agy, so the whole launch is gated once, up front.
3. The CLI is launched with `--input-format stream-json --output-format stream-json --print-timeout 60m --dangerously-skip-permissions`, plus `--model <id>` (or `--effort low|medium|high` when no explicit model is set), plus `--new-project` or `--conversation <saved-id>` to resume a project conversation.
4. Your prompt is sent as a JSON `user` event, wrapped in a `<pocketdev_workspace>` block instructing the agent to work only inside `/workspace/<project>`, never to loop asking clarifying questions, and to execute concrete requests end to end rather than replying with a plan.
5. Events stream back: `init` (the conversation ID is saved for resume), `step_update` text deltas stream into the chat, tool steps surface as ToolStarted/ToolCompleted with friendly names (run_command → Bash, write_to_file/replace_file_content/multi_replace_file_content → Write, view_file → Read, list_dir → List files, grep_search/search_files → Search, schedule → Wait, manage_task → Manage task), and `result` SUCCESS/ERROR ends the turn.
6. API keys, tokens, secrets, passwords, and bearer tokens are redacted to "••••" wherever they appear in tool details.
7. A checkpoint snapshot is taken before the run; changed files are diffed afterwards with the same review/undo/accept flow as the other harness agents.

The 60-minute print timeout is the task ceiling: if agy times out, the failure message reads "Antigravity reached the 60-minute task limit. Your files were kept." Other mapped failures: expired/invalid auth (401) → "reconnect your Google account in Settings"; "out of credits"/quota → "Your Antigravity account is out of credits. Check the account plan or wait for credits to reset."; unknown model → "Refresh models in Settings." Antigravity supports session recovery.

Stopping

Stop destroys the CLI process, cancels the API client and any running tool executor in Gemini API mode, aborts pending launch approvals, and reports "Stopped by user".

B7. Codex CLI Tab

Purpose. Codex is OpenAI's coding agent (150 MB package), installed with npm in the Linux guest. It gives full agentic coding, file edits and terminal, through your own OpenAI API key.

Setting it up

The Codex card is deliberately minimal, with the same save pattern as the Your API card and no provider-validation round trip (an Anthropic-style ping would fail against an OpenAI endpoint):

- **API key**: password-masked field with a show/hide eye. Your key is stored encrypted in Android secure storage and passed to the CLI through its process environment. It is never written to a file or the CLI command line.
- **Model (optional)**: placeholder "gpt-5-codex (blank = Codex default)". Leaving it blank uses the Codex default; the base URL likewise defaults to OpenAI when blank.
- **Save Codex API** button: enabled when the key is non-blank, shows a check and "Saved" on success, and shows the real failure reason if the save fails.

The card's footnote reads: "Your key is stored encrypted in Android secure storage and passed to the CLI through its process environment. Codex runs the official codex exec CLI in the Linux guest." Installation and updates live in the generic not-installed card and the per-tab Runtime updates block.

How a session runs behind the scenes

The `CodexRuntimeBridge` class drives the official `codex exec --json` CLI:

1. It fails fast if Codex CLI is not installed ("Open Settings → Coding agent to install it.") or if no API key is saved ("Add your OpenAI API key in Agent → Codex.").
2. The environment is built with `OPENAI_API_KEY` and `CODEX_API_KEY` set to your key, plus `OPENAI_BASE_URL` if you configured a custom base URL.
3. The CLI is launched as `codex exec --json --skip-git-repo-check -C /workspace/<project> [-m <model>] <prompt>`. The `--skip-git-repo-check` flag is there because project workspaces are not always git repositories; a task should not fail for that. The prompt is wrapped in a `<pocketdev_workspace>` block with the same stay-in-workspace, no-clarification-loop, execute-end-to-end instructions used by the other agents.
4. JSONL events map onto the live feed: `thread.started`, `item.started` / `item.completed` (agent messages stream as text; command executions surface as Bash; file changes as

Write; MCP tool calls as MCP; web searches as Search), `turn.completed`, and `turn.failed` / `error`.

5. Tool activity also drives the foreground notification ("Running Bash · <command>").
6. A checkpoint snapshot is taken before the run; changed files are diffed afterwards with the review/undo/accept flow.

Mapped failures: 401/invalid key → "The OpenAI API key was rejected (401). Check the key in Agent → Codex."; 429 → rate-limit message; quota → out-of-quota message; unknown model → "Pick another model in Agent → Codex."

Codex runs headless with no approval-response channel wired in this driver, so `respondToApproval` intentionally does nothing: no Codex approval can ever be pending.

Stopping

Stop destroys the CLI process and reports "Stopped by user".

B8. Gemini API Mode (Inside the Antigravity Tab)

Purpose. At the very top of the Antigravity card, above the Google account section, sits the **Use Gemini API** switch. It is a conditional toggle: when ON and a key is saved, Antigravity sessions bypass the agy CLI and Google sign-in entirely and run through Google's Gemini API directly with a built-in function-calling agent loop. When OFF, the existing OAuth flow is untouched.

The subtitle says it plainly: "Use your own Gemini API key instead of Google sign-in".



Controls

Control	What it does
Use Gemini API switch	Turns API mode on or off. The mode only takes effect when a key is saved: route resolution requires the switch ON and a non-blank key, otherwise the session falls back to Antigravity OAuth.
API key field	Password-masked input. Labelled "Gemini API key", or "Replace API key" when one is already saved. Saving clears the field.
Get API key	Opens `https://aistudio.google.com/apikey` in the browser so you can create a key in AI Studio.
Save	Stores the key (enabled only when the field is non-blank). Blank keys are ignored.
Key status row	When saved: green check plus "API key saved on this device" and a **Remove** button that deletes the key. When not saved: red text "Save an API key to turn on Gemini API mode".
	Selects the Gemini model used for API sessions.

Model dropdown	

Available models

Label (as shown)	Model ID
Gemini 4 Argon	`gemini-4-argon` (default)
Gemini 3.8 Flash	`gemini-3.8-flash`
Gemini 3.1 Pro	`gemini-3.1-pro`

Only these listed IDs can be selected; anything else is rejected by the store.

Key storage

The `GeminiApiStore` keeps three values in `gemini_api_secure_prefs`, an `EncryptedSharedPreferences` store sealed with an AES256-GCM master key: the mode flag, the API key (trimmed on save), and the selected model. The toggle state and model persist across app restarts.

The function-calling agent loop

When API mode is active, a chat message on the Antigravity tab runs `GeminiAgentLoop` instead of the agy CLI. This is a genuine agentic loop with real tools, not plain chat:

1. Your prompt, the conversation history, and a system prompt are sent to `https://generativelanguage.googleapis.com/v1beta/models/<model>:streamGenerateContent` with the key in the `x-goog-api-key` header, `temperature 0.7`, `maxOutputTokens 8192`, and the five function declarations attached.
2. Text streams back live into the chat. Any function calls the model requests are executed one by one, with ToolStarted/ToolCompleted events, and their outputs are fed back to the model as function responses.
3. The loop repeats until the model answers with no further function calls, or the **25-iteration cap** (`MAX_AGENT_ITERATIONS = 25`) is reached, or you stop it.
4. The agent's system prompt instructs it: work only inside `/workspace/<project>`; when asked for a code change, inspect the project, make the edits, then build or run tests to verify; do not ask for confirmation for normal file operations; explain briefly afterwards.

Two honest caveats, exactly as the feature documents them: this is **direct model chat with function calling**, a streaming request/response loop, not the CLI's own agentic runtime; and it needs a build to verify on-device. It also sits behind the same full-access launch gate as the CLI: before launch the app asks you to approve an operation named `gemini-api-agent` with the warning that it "can run shell commands and read, write or delete files in the project workspace without asking again. Approve only if you trust this task." A workspace checkpoint is taken before the run, and changed files get the same review/undo/accept flow.

The five tools

Tool	Arguments	What it does	Limits and safety
`run_shell`	`command` (required); `workdir` (optional subdirectory)	Runs a shell command inside the project's Linux workspace and returns its output. Used to build, run and test code, install packages, and inspect the project.	Requires the Linux runtime to be installed. The workdir must stay inside the project (any ".." is rejected). Commands time out after 120 seconds. Output is capped at 32 KB. Stopping kills the process.
`read_file`	`path` (required, workspace-relative); `offset` (first line, 0-based, default 0); `limit` (max lines, default 200, max 500)	Reads a file and returns the requested line range.	Unsafe paths (anything escaping the workspace) are rejected. Missing files return an error, not a crash.
`write_file`	`path` (required); `content` (required)	Writes content to a file, creating parent directories as needed. Overwrites if the file exists.	Unsafe paths rejected. Returns the character count written.
`edit_file`	`path`, `old_text`, `new_text` (all required)	Replaces the first occurrence of `old_text` with `new_text`. Both strings must match exactly, including whitespace.	Unsafe paths rejected. Empty `old_text`, missing file, or text-not-found all return clear errors.
`list_dir`			

	`path` (optional, defaults to the project root)	Lists files and directories; directories first.	Unsafe paths rejected. Empty directories report "(empty)".

Every tool runs inside `withContext(Dispatchers.IO)` and catches its own failures, returning them as `Error: ...` text the model can read and recover from. Unknown tool names return `Error: unknown tool '<name>'.`

> **Secret hygiene in tool output:** anything that looks like an API key, token, secret, password, or bearer token is redacted to "...." before tool details are shown or fed back, whitespace is collapsed, and displayed snippets are capped (300 characters in the UI, 32 KB from the tool). If the model produces no text at all, the session emits a single "Done." message rather than going silent.

Stopping and errors

Stop cancels the active HTTP connection and any running tool process, and the session reports "Stopped by user". API errors are mapped to plain messages: 400 → "Gemini API rejected the request."; 401/403 → "Gemini API key is invalid or lacks access. Create a new key in AI Studio."; 404 → "The selected Gemini model is not available through the API."; 429 → "Gemini API quota exceeded. Try again later."; anything else → "Gemini API error <code>." with the provider's detail appended (capped at 200 characters).

B9. Multi-Agent Engineering Teams

Purpose. The groups icon in the Agent screen's top bar opens the **Multi-Agent Engineering Team** studio: a bottom sheet where you describe one engineering goal and a squad of up to 10 specialized agent roles decomposes it, plans a dependency DAG, and executes it, with several agents genuinely running in parallel on their own bridge instances.

If no team orchestrator is wired into the build, the sheet stays closed and the button shows a toast: "Team execution is not configured in this build." There is no silent fallback and no simulation: without a real executor the orchestrator refuses to run instead of fabricating output.

Opening the studio

The bottom sheet header reads **Multi-Agent Engineering Team** with the subtitle **10 Specialized Autonomous Agents**. A sparkle icon opens the full Studio view; an X closes the sheet. The sheet contains:

- **Roles strip**: a horizontally scrollable row of the 10 role chips. The currently executing role is highlighted with a spinner.
- **Team Engineering Goal**: a text field, prefilled with "Create full application architecture and implementation".
- **Plan Team** (sparkle icon): the one-click full run. It cancels any in-flight run, decomposes the goal into the 10-task DAG, and immediately runs the whole team to completion. One click does the full autonomous run; it does not leave a task list sitting idle.
- While running, three more buttons appear: **Step** (executes the next ready task only), **Run All** (dispatches the whole DAG), and **Stop** (cancels; in-flight agent runs finish rather than being killed).
- **Retry Failed**: appears when the run is not running and at least one task failed. It re-queues every FAILED task as READY; BLOCKED dependents stay blocked until their failed dependency completes.
- **Decomposed Subtasks (n/m)**: the task list with a live completed/total count. Each task card shows a role badge (the role's enum name), the task title, its description, and a status icon (green check for completed, spinner for in progress, hourglass otherwise). Tapping the icon manually cycles the task's status (PENDING → READY → IN_PROGRESS → COMPLETED, and so on).
- An error banner appears when a run fails or stalls, naming the cause; a stalled run says so plainly instead of leaving Step as a dead button.

The 10 roles

Role	Title	Job	Default engine	Permissions
MANAGER	Engineering Manager	Decomposes high-level requirements into subtasks, delegates, and oversees team velocity.	Claude Code	network, approve reviews
ARCHITECT	Software Architect	Designs system blueprints, interfaces, data schemas, and technical contracts.	Claude Code	network, approve reviews
DEVELOPER	Fullstack Developer	Implements business logic, APIs, UI components, and integrations.	Antigravity	write code, terminal
UI_ENGINEER	UI/UX Specialist	Refines user interfaces, Compose views, animations, and responsive layouts.	Antigravity	write code, terminal
BACKEND_ENGINEER	Backend Specialist	Constructs endpoints, server logic, database interactions, and middleware.	Antigravity	write code, terminal
DATABASE_ENGINEER	Database Architect	Creates schemas, queries, migrations, and caching layers.	Antigravity	write code, terminal
TESTER	QA & Test Engineer	Designs unit tests, integration suites, edge cases, and verifies execution.	Antigravity	write code, terminal
SECURITY_REVIEWER	Security Auditor	Inspects code for vulnerabilities, secret leaks, permission violations, and risk levels.	Antigravity	approve reviews

CODE_REVIEWER	Code Reviewer	Enforces architecture consistency, SOLID design patterns, and code clarity.	Claude Code	approve reviews
DEVOPS_ENGINEER	DevOps & CI/CD	Manages build scripts, Gradle configurations, GitHub workflows, and container runtimes.	Antigravity	write code, terminal, commit git, network

These engine assignments are defaults. At execution time the **engine resolution** rules apply: the role's wired engine runs if it is installed and authenticated; otherwise the user's currently selected agent is tried; then Antigravity, Claude Code, DeepSeek Harness, and Codex in fixed order. A signed-in Antigravity session counts as authentication (the agy CLI never reads a vault key), so a task wired to Claude Code can run on Antigravity when no Claude key is saved. Your API (Custom) is never a fallback candidate: it is chat-only and would mark engineering tasks complete with zero code written. If no engine is usable, the task fails naming every engine tried and why. The live task and artifact metadata record the engine that actually ran each task.

The plan: a 10-task dependency DAG

Plan Team decomposes your goal into ten linked tasks:

1. **Sprint Planning & Goal Decomposition** (Manager, critical) - scope, milestones, deliverables.
2. **System Architecture & API Contracts** (Architect, critical) - blueprint, module boundaries, interface contracts. Depends on planning.
3. **Database Schemas & Data Model Design** (Database Engineer) - schemas, entities, migrations. Depends on architecture.
4. **Backend Service & Data Pipeline Logic** (Backend Engineer) - endpoints, repositories, network services. Depends on architecture and database.
5. **Compose Screen UI & Responsive Layouts** (UI Engineer) - Material 3 interfaces, design tokens, state bindings. Depends on architecture.
6. **Fullstack Integration & Feature Implementation** (Developer) - integrates UI with backend. Depends on backend and UI.
7. **Automated Unit & Integration Test Suite** (Tester) - happy paths, error states, edge cases. Depends on integration.
8. **Security Audit & Risk Assessment** (Security Reviewer) - scans for leaks, unsafe inputs, permission flaws. Depends on integration.

9. **Code Quality & Architectural Review Gate** (Code Reviewer, critical) - SOLID enforcement, peer sign-off. Depends on tests and security audit.
10. **CI/CD & Release Build Packaging** (DevOps Engineer) - build scripts, Gradle verification, packaging. Depends on the review gate.

Planning also publishes an **Architecture Decision Record** to the shared blackboard ("System Architecture Blueprint: <goal>") and broadcasts a kickoff message on the team communication bus.

Parallel execution

The `ConcurrentTeamDispatcher` runs the DAG stage by stage. Within a stage, every READY task runs concurrently, bounded by a semaphore. The stage limit is the live effective agent count (see adaptive scheduling below), always at least 1.

- **True parallelism:** every task execution builds its **own bridge instance** through the bridge factory. Bridges hold all session state (active process, session ID, streamed text) as instance fields, so separate instances are separate OS processes genuinely running at the same time. There is no shared bridge and no per-kind mutex; the only bound is the dispatcher's semaphore.
- **File safety:** tasks whose `affectedFiles` overlap never run at the same time. Per-file mutexes (normalized lowercase paths) are acquired before execution. A failed task blocks its dependents via the scheduler; it is never silently skipped.
- **Budget guard:** the token quota is checked before every stage and every task launch; when exceeded, dispatch halts and an escalation is sent to the Manager.
- **Cancellation:** Stop prevents new launches; in-flight agent runs are left to finish rather than being unsafely killed.
- **Each task's context:** the role's prompt (system prompt plus task, dependency outputs, and a blackboard summary) is dispatched through the app's real agent runtime; streamed assistant output is captured; SUCCESS is reported only when the bridge genuinely emitted SessionCompleted. Timeouts are 20 minutes per task.
- **Handoffs:** when a task finishes, its dependents receive a handoff message with the real output summary.
- **Live observability:** each live task keeps the last ~2000 characters of streamed output, token usage, duration, and the engine that ran it, so the UI can drill into running agents (the header shows "N agents running").

The 3-agent cap and adaptive throttling

The configured maximum is **3 concurrent agents**; the code is explicit that phones are thermally and memory bound and that raising it risks OOM kills and thermal throttling. On top of the cap sits the `AdaptiveConcurrencyController`, which keeps the app smooth on old and new phones:

Hardware baseline (computed once at startup, never above 3):

Device	Baseline agents
Total RAM 3 GB or less, or 4 or fewer CPU cores	1
Total RAM 6 GB or less	2
Above that	3

Dynamic throttling (re-evaluated before every dispatch stage and every 30 seconds during a run; scales back up when pressure clears, never above baseline):

- Thermal state MODERATE or worse → drop to 1 agent ("thermal throttle").
- Available RAM below 15% of total → drop to 1; below 25% → drop to 2 ("low memory").
- Battery at 14% or below and not charging → drop to 1 ("low battery").

In-flight agent runs are never killed by throttling; the new limit applies to newly dispatched stages. The UI header reflects the live throttle state (for example "2 agents running (thermal throttle)"). Each parallel agent spawns its own CLI process (node/python), so expect roughly N times the single-agent memory and CPU while a stage runs.

> **Workspace note:** team tasks execute in a dedicated team workspace (one directory per team goal). Binding an existing project directory to a team run is documented in the code as future work; the project directory binding on the orchestrator is currently always null.

Blackboard, communication, reviews, budget

- **Shared blackboard:** agents publish typed knowledge artifacts the whole team can see. Artifact types: Architecture Decision Record, API Specification & Contract, Database Schema & Migrations, Test Suite & QA Report, Security Audit & Risk Assessment, Code Review Feedback & Sign-off, Source Code Diff / Implementation, DevOps CI/CD & Deploy Config, and Technical Documentation. Simulated artifacts are flagged as such and never count as real.

- **Communication bus:** typed inter-agent messages: PROPOSAL, REVIEW_REQUEST, FEEDBACK, STATUS_UPDATE, QUERY, RESPONSE, HANDOFF, BROADCAST, ESCALATION. The UI keeps a legacy event view of the same messages plus an execution log.
- **Review gatekeeper:** TESTER and SECURITY_REVIEWER tasks get real validation (actual test/security validation against the project directory), published as artifacts with review records. A CODE_REVIEWER approval is recorded only when the agent emitted an explicit verdict marker; otherwise the run notes that no approval was recorded. Verdicts: APPROVED, CHANGES_REQUESTED, REJECTED, PENDING. Simulated reviews never count as real approval.
- **Budget tracker:** default token quota 100,000 tokens, maximum execution time 600,000 ms (10 minutes), cost accounting at $0.003 per 1,000 tokens with per-role breakdowns. Exceeding the quota halts the run with an escalation to the Manager.
- **Conflict detection:** file-overlap analysis across tasks surfaces "Potential write conflict detected on: <file>" warnings.

Task lifecycle and run reports

Task statuses: PENDING → READY → IN_PROGRESS → COMPLETED, with BLOCKED (waiting on a failed dependency), UNDER_REVIEW, and FAILED as the other states. A run that cannot proceed because a failed task blocked the DAG reports a plain stall message naming the failed task and its error, with "Retry failed" as the recovery path.

Every full run produces a **TeamRunReport**: the goal, one outcome per task (status, output summary, duration, tokens used, review verdict), all review records, total duration, and whether it was cancelled. A run counts as successful only when nothing was cancelled, nothing failed, nothing is blocked, and at least one outcome exists.

B10. Session Lifecycle, Foreground Work, and Undo

All five engines implement the same `RuntimeBridge` contract: `startSession`, `stopSession`/`stopActiveSession`, `respondToApproval`, and the checkpoint family (`undoLastChanges`, `acceptLastChanges`, `loadPendingChanges`, `undoFileChange`, `acceptFileChange`). The UI maps their events (SessionStarted, AssistantDelta, ReasoningProgress, ReasoningSummary, ToolStarted, RuntimeLog, ToolRequested/Approved/Rejected, ToolCompleted, FilesChanged, PreviewStarted, SessionCompleted, SessionFailed) onto the chat.

Foreground service

Harness-agent tasks run under a foreground service so Android does not kill long work. The notification announces itself ("<Agent> is working in <project>"), posts progress as tools run ("Running Bash · <command>", capped at 80 characters), and finishes with "<Agent> finished the task in <project> in <duration>." A user stop cancels the notification instead of completing it. The Gemini API hello-style probes and the Antigravity "Test connection" probe intentionally skip the foreground service.

Checkpoints and undo

Before every harness-agent task (Claude Code, DeepSeek Harness, Antigravity CLI, Gemini API mode), the bridge snapshots the project workspace. After the run, changed files are diffed against the snapshot and reported as a FilesChanged event with per-file diffs (additions, deletions, context lines; binary files flagged). You can then:

- **Undo last changes**: restores every changed file from the snapshot and deletes the checkpoint.
- **Accept last changes**: deletes the checkpoint, keeping the new files.
- **Undo/accept a single file**: restores or keeps just that path and removes it from the pending-changes list.

Internal runtime paths are excluded from all of this; only your project files are ever restored or accepted. The Custom (Your API) agent has no workspace and no checkpoint: its undo operations always report false and its accept operations do nothing.

Stopping, per engine

Engine	What stop does

Claude Code	Destroys the CLI process, waits 500 ms, force-destroys if still alive; pending permission requests are answered "deny"; reports "Stopped by user".
DeepSeek Harness	Same destroy/force-destroy sequence; aborts any pending launch approval; reports "Stopped by user".
Antigravity CLI	Destroys the CLI process; in Gemini API mode also cancels the HTTP client and the tool executor; aborts pending launch approvals; reports "Stopped by user".
Codex CLI	Destroys the CLI process; reports "Stopped by user".
Your API	Disconnects the active streaming HTTP connection immediately; the session reports **Completed**, not Failed.
Team run	**Stop** cancels dispatch; in-flight agent runs finish rather than being killed. There is also **Step** for single-task execution.

Transient network failures are classified centrally: when a failure is transient, the session fails with a friendly network message instead of posting a failed foreground result.

B11. Costs and Billing: What the App Actually Says

Conzyx states billing only where the provider configuration does. Nothing in the agent system adds a Conzyx-side charge; every key-based provider is pay-the-provider-directly under that provider's own terms.

Surface	What the app states
Anthropic API provider	"Usage billed through Console"
Claude subscription provider	"Pro, Max, Team or Enterprise" plans, via a `claude setup-token` token
OpenRouter provider	"Use your OpenRouter API key"
DeepSeek provider	"Use your DeepSeek API key"
Kimi provider	"Anthropic-compatible endpoint" (your key)
NVIDIA NIM provider	"OpenAI-compatible models hosted by NVIDIA · free tier available"
OpenAI API (Codex)	"Use your OpenAI API key"; quota failures surface as "The OpenAI account is out of quota. Check billing or wait for the quota to reset."
Custom API (Your API)	Your endpoint, your key, your bill
Antigravity quota	"Your Antigravity account is out of credits. Check the account plan or wait for credits to reset." Multi-account round-robin exists precisely to ride out per-account quota hits.
Gemini API quota	429 → "Gemini API quota exceeded. Try again later."
Team budget tracker	Default quota 100,000 tokens, 10-minute max execution time, costed at $0.003 per 1,000 tokens with per-role breakdowns. This is an in-app accounting estimate, not a bill.

Model discovery marks provider-advertised free models with a green **FREE** badge where the provider reports them.

Bottom line for your wallet: the app never charges you. Your spend is whatever your chosen provider charges for the models you select, plus whatever plan or credits sit behind your Google account for Antigravity. Zero-cost options the app itself surfaces: NVIDIA NIM's advertised free tier, and the green **FREE** badge on provider-advertised free models during model discovery.

Terminal

The Terminal tab is a full Linux shell running inside Conzyx. Every command you type is executed by a real **bash** process in the app's bundled Linux runtime, not simulated. The tab is built for phone use: multiple concurrent shell sessions, tap-to-run quick commands, a mobile key helper row (history, cursor keys, Ctrl, Alt, Esc), and live streaming output.

What the shell actually is

Behind the tab is a **PRoot sandbox running Ubuntu 24.04** (the subtitle under the title reads "Ubuntu 24.04 · PRoot Sandbox"). Each command you submit is launched as `/usr/bin/bash -c <your command>` inside that guest filesystem, through a native process spawner (`libpocketspawn.so`) with a pseudo-terminal of 40 rows by 120 columns. The prompt you see is the guest shell's: `root@mhe:~#`. You are root inside the sandbox.

> **Where commands run.** If a project is active, commands run in that project's workspace folder, which the guest sees as `/workspace/<project-slug>` (the prompt shows it as `~/<project-slug>`). With no project active, commands run in a shared throwaway directory that appears as `~`. The terminal, the file tree, the editor, and the AI agent all share the same underlying project folder, so a file created in the terminal appears in the Files tab and vice versa.

The shell environment is prepared for unattended phone use:

- `apt` / `apt-get` commands are rewritten before execution: `sudo` is stripped (you are already root), `-y -o Dpkg::Options::=--force-confold` is injected, and `DEBIAN_FRONTEND=noninteractive`, `TZ=Etc/UTC`, `LC_ALL=C.UTF-8` are exported, so package installs never block on Y/N prompts, locale, timezone, or service-restart dialogs.
- If a package manager does print a "Do you want to continue [Y/n]?" prompt, Conzyx answers **y** once automatically by writing to the process's stdin.
- Typing the word `clear` and pressing send clears the console locally; it is not executed as a shell command.

The header bar

At the top of the tab, an app bar shows a terminal icon, the title ("Linux Terminal" by default) and the subtitle ("Ubuntu 24.04 · PRoot Sandbox"). On the right are action icons:

Icon	Control	What it does

Magnifier	Search terminal output	Opens a filter bar under the session row. Typing filters the recorded command/output blocks to those whose command text or output contains the query (case-insensitive). A counter shows "N matches". The X button clears the query and closes the bar. With a query active and no matches, the console shows "No matching terminal output found."
Trash	Clear output	Empties the recorded lines of the active session. Does not affect other sessions and does not kill a running process.

A second trash icon sits at the right end of the session row and does the same thing.

Session tabs (the pill row)

Directly under the header is a horizontally scrollable row of **session pills**. Each pill represents one independent shell session. New sessions are named "Terminal 1", "Terminal 2", and so on.

```
Terminal 1 x
Terminal 2 x
+
```

Element	Behavior
Session pill	Tap to switch to that session. The active pill is highlighted with a filled background and border; inactive pills are plain text.
Green dot	Appears on a pill while that session has a process running, so you can see at a glance which sessions are busy even when you are looking at another one.
X on a pill	Closes the session. The X only appears when there is more than one session; you can never close the last session. Closing the active session switches you to the most recently remaining one.
+ button	Creates a new session ("Terminal N") starting in the project's current terminal directory, with an empty console, and switches to it.

What switching preserves

Each session keeps its own state: the recorded command/output lines, the working directory, and whether a process is running. When you tap another pill, the current session's lines, directory, and running flag are saved into its record and the target session's state is restored. A

process you left running in "Terminal 1" keeps running while you work in "Terminal 2"; its green dot stays lit and its output keeps accumulating.

> **History is persisted per project.** Session lines and the active session are written to a JSON file under the app's private storage (`terminal-history/<projectId>.json`), keeping at most the last 100 lines per session, and restored when you reopen the project. The console is therefore not wiped by rotating the phone or leaving the tab.

Quick command chips

In the standalone terminal (not the project-embedded one), a row of tappable chips sits above the console while the phone keyboard is hidden. Tapping one submits it immediately; if a process is currently running, the text is sent to that process's stdin instead of starting a new command. The built-in chips are:

- `uname -a` (kernel and architecture)
- `ls -la` (detailed listing of the current directory)
- `pwd` (print working directory)
- `node -v` and `python3 --version` (toolchain versions)
- `df -h` (disk space) and `free -m` (memory)
- `claude --version` (check the Claude CLI in the sandbox)

The console

The console is a bordered, scrollable pane. An empty terminal shows the hint "Conzyx Terminal ready. Type a bash command below or tap a quick command chip above."

- **Prompt format.** Every recorded command is shown as `root@mhe:~/path# <command>` , with the prompt in green and the command in bold. The `~` is shorthand: `/workspace` is shown as `~` and `/workspace/sub/dir` as `~/sub/dir` .
- **Output colors.** Output of a command that exited with a non-zero exit code is rendered in the theme's error color; successful output is normal text.
- **Sanitization.** ANSI escape sequences (colors, cursor moves) are stripped from output, and control characters other than newline, carriage return, and tab are removed, so misbehaving programs cannot corrupt the display.
- **Live streaming.** While a command runs, its output is tailed from a temporary file and appended live (capped at 200,000 characters), and the console auto-scrolls to the bottom as new text arrives.

- **Selection.** The whole console is wrapped in a selection container, so you can long-press to select and copy any output text.

Typing and submitting commands

The input line at the bottom of the console shows the same green `root@mhe:~#` prompt prefix drawn in front of your text (the prefix is hidden while a process is running). Pressing the keyboard's **Send / Done / Go** key, or tapping a quick chip, submits:

- If no process is running, the text is executed as a new command. The command is appended to the session's command history (the last 50 commands are kept) and the input clears.
- If a process is running, the text is written to that process's stdin followed by a newline, so you can answer interactive prompts (for example a password prompt or a REPL).
- Blank input is ignored.

Command history

The terminal remembers the commands you submitted this session (up to 50). Use the **up-arrow key** in the helper row to recall the previous command and the **down-arrow key** to move forward again; the recalled text is placed in the input with the cursor at the end, ready to edit or resend.

The mobile key helper row

Phone keyboards usually lack terminal keys, so a helper row appears above the status area whenever the system keyboard is hidden. From left to right:

Key	Label in UI	What it does
Up arrow	"Previous command"	Recalls the previous command from history into the input line.
Down arrow	"Next command"	Moves forward through history toward the newest entry.
Left arrow	"Move cursor left"	Moves the text cursor one character left. If ALT is active, jumps to the start of the previous word instead.
Right arrow	"Move cursor right"	Moves the text cursor one character right. If ALT is active, jumps to the end of the next word instead.
ALT		

	"Alt modifier; arrows move by word"	Toggle. While highlighted, the next arrow press moves by whole words (readline-style Alt+B / Alt+F). Typing any normal character automatically turns it off so it never goes stale.
ESC	"Escape; press again on empty input to restore"	First press clears the input line but stashes the text; pressing ESC again on an empty line restores the stashed text.
CTRL	"Control modifier; press C to interrupt"	Toggle. While highlighted, typing the letter C sends an interrupt to the running process (the same SIGINT signal Ctrl+C produces in a real terminal). If no process is running, it simply clears the current input line. Any other typed character turns CTRL off without acting.

Interrupting and stopping processes

- **Interrupt (Ctrl+C equivalent).** Tapping the CTRL helper key and then typing `c` sends signal 2 (SIGINT) to the native process, the same interrupt Ctrl+C produces in a real terminal. Well-behaved programs (servers, scripts, package managers) stop; the exit code is recorded like any other.
- **Stopping a session's process.** A stop action destroys the process (SIGTERM, signal 15); if it is still alive after a short wait it is killed forcibly (SIGKILL, signal 9).

Destructive-command guard

Before a command is prepared for execution, it is checked against a destructive-command list. If it matches, Conzyx shows a confirmation dialog titled **"Run potentially destructive command?"** with the text "This command can delete files, rewrite Git history, or change the project significantly.", the full command in monospace, and two buttons: **"Run anyway"** and **"Cancel"**. Flagged patterns include:

- `rm -rf` / `rm -fr`
- `git reset --hard`, `git clean -f`, `git push --force`
- `mkfs`, `dd if=`, `chmod -r 777`, `shutdown`, `reboot`
- Fork bombs (`:(){`), `kill $(pgrep ...)`, `pkill -f`
- Piping `curl` or `wget` output into `sh` / `bash`

Server detection and preview

The terminal coordinator watches for local web servers so the Preview tab can pick them up:

- If a command's **output** contains a URL like `http://localhost:3000/...` (or `127.0.0.1` / `0.0.0.0` with a port), the last such URL is recorded as the preview URL.
- If the **command itself** is `python -m http.server [port]`, the corresponding `http://127.0.0.1:<port>/` URL (default port 8000) is recorded directly.

Behind the scenes: how processes run

For readers who want the mechanics:

- **Spawning.** Commands are spawned through `NativeSpawnProcess`, a wrapper around the bundled native library `libpocketspawn.so` (functions `spawn`, `waitFor`, `kill`). If that library is missing or incompatible, the app reports that it must be reinstalled from the official release.
- **Output capture.** The process's output goes to a temporary file; a background "pocket-pty-output" thread pumps the pseudo-terminal's output into that file, and the UI tails the file to stream live output. Standard error is merged into the same stream.
- **Signals.** Interrupt sends signal 2 (SIGINT), normal stop sends 15 (SIGTERM), forced stop sends 9 (SIGKILL).
- **History caps.** Live output is capped at 200,000 characters and each session keeps at most 100 recorded lines in persisted history, so a chatty build log cannot exhaust memory.

Keeping long tasks alive

When an AI agent task (build, install, test run) executes, Conzyx runs a foreground service (`RuntimeExecutionService`) so Android does not kill the work when the screen sleeps or you switch apps. What you see as a user:

- A persistent notification titled **"Conzyx is working"** naming the project and the current step (e.g. "Coding agent is working in MyApp").
- A **Stop** action on the notification that safely stops the task.
- On completion the notification changes to **"Task completed"** (or **"Task needs attention"** on failure).

Technically the service holds a partial wake lock and a high-performance Wi-Fi lock (both capped at 90 minutes) while the task runs, then releases them.

Common terminal workflows

1. **Check the environment:** tap the `uname -a`, `node -v`, or `python3 --version` chips.
2. **Install dependencies:** run `npm install` or `apt-get install <pkg>`; apt prompts are auto-confirmed, and output streams live.
3. **Run a build:** type `./gradlew build`, `npm run build`, `cargo build`, or `go build ./...`. A non-zero exit code colors the output red so failures stand out.
4. **Start a dev server:** e.g. `npm run dev` or `python3 -m http.server 8080`. The terminal detects the localhost URL; open the **Preview** tab to view it in the in-app browser.
5. **Answer a prompt mid-run:** while the process runs, type into the input line and press send; the text goes to the process's stdin.
6. **Stop a runaway process:** use the CTRL helper key + `c` (SIGINT).
7. **Parallel work:** tap **+** to open a second session (e.g. one running the dev server, one free for git commands) and switch between them with the pills.

> **Tip.** Because the terminal shares the project folder with the Files tab and the editor, prefer running project commands from the project terminal (opened while a project is active) so paths like `~/src` match what you see in the file tree.

Files

The **Files** tab is the project file explorer. It shows the project's folder tree, lets you search it, create/rename/move/duplicate/delete entries, bind a folder from the phone's storage, export the project as a ZIP, and run Git operations. Tapping a file opens it in the code editor.

Header toolbar

The top of the tab is a rounded card titled **"Files"** with a horizontally scrollable row of icon buttons (it scrolls so nothing is cut off on narrow phones):

Icon	Label (tooltip)	What it does
+	New file	Opens the New File dialog, creating in the project root.
Folder	New folder	Opens the New Folder dialog, creating in the project root.
Up arrow + text	Collapse all	Appears only when at least one folder is expanded; collapses every expanded folder.
Folder	Bind folder from device	Opens the system folder picker so you can attach an existing folder from the phone's storage as the project.
Download	Export project as ZIP	Appears only when the project contains at least one file; exports the project as a ZIP archive.
Code (</>)	Git operations	Opens the Git Operations dialog (status, diff, branches, commit, push, pull).
Refresh / spinner	Refresh files	Reloads the file tree from the filesystem. While loading, a spinner replaces the button.

Searching the file list

Below the header is a search field labeled **"Search project files"** with the placeholder "Search by path or filename". Typing filters to files (not folders) whose path contains the query, case-insensitively; a line under the field reports e.g. "3 matching files". A **Clear** button appears at the right of the field to reset it. While searching, the folder structure is flattened into a simple result list; clearing the query restores the tree.

Project-root suggestion

If Conzyx detects a likely project folder (for example a bound device folder that looks like a project root), a highlighted card appears: **"Project folder detected"**, with the text "Use <path> as the project root so Chat, Terminal, Changes, and Preview all run from the same folder." and a button **"Use <path> as project root"**. Accepting it aligns the working directory of every tab.

Error and empty states

- If the file list cannot be loaded, an error card shows the message and a **Retry** button.
- If the workspace is empty, a card reads **"No files in workspace yet"** with the hint "Ask the agent to create files, or open and bind an existing project folder from your phone storage." and a button **"Open / Bind folder from device"**.
- A search with no matches shows "No matching files / Try a different filename or path."

The file tree

- **Folders** show a disclosure arrow (right-pointing when collapsed, down-pointing when expanded), a folder icon, the folder name, and the number of direct children in parentheses, e.g. `src (4)`. Tapping a folder toggles it open or closed; collapsing a folder also collapses everything nested inside it.
- **Files** show a document icon, the file name (in the theme's primary color), and the file size at the right (e.g. "2.4 KB"). Tapping a file opens it in the code editor.
- Rows are indented by depth (20 dp per level) so nesting is visible.
- Each folder row also carries a small **tune/sliders icon** ("Set as working directory") that makes that folder the terminal's working directory.

```
v src (3)
> components (5)
= main.kt 2.4 KB
= utils.kt 1.1 KB
```

File operations (the 3-dot menu)

Every file and folder row has a **3-dot (⋮) "Options" button** that opens a context menu. The available actions:

Menu item	Available for	What it does
New file inside	Folders	Opens the New File dialog pre-targeted at that folder ("Creating in: <folder>").
New folder inside	Folders	Opens the New Folder dialog pre-targeted at that folder.
Duplicate	Files	Creates a copy of the file immediately.
Rename	Both	Opens the Rename dialog (see below).
Move to…	Both	Opens the Move dialog (see below).
Copy relative path	Both	Copies the entry's project-relative path (e.g. `src/main.kt`) to the clipboard.
Delete	Both	Opens the Delete confirmation dialog (see below).

The dialogs

- **New File.** Shows "Creating in: <parent>" (or "project root") and a "File name (e.g. Main.kt)" field. Leading/trailing spaces are trimmed. **Create** is disabled while the name is blank or contains `//`. On confirm, the file is created at `<parent>/<name>`.
- **New Folder.** Same pattern with a "Folder name (e.g. models)" field.
- **Rename File / Rename Folder.** Shows the current path in monospace and a "New name" field prefilled with the current name. **Rename** is disabled unless the name is non-blank, different from the current name, and contains no `/` (renames stay in the same folder; use Move to relocate).
- **Move File / Move Folder.** Shows "Current path: <path>" and a "Target relative path" field prefilled with the current path. **Move** is disabled unless the target is non-blank and different from the current path. You can move an entry into another folder by typing the new relative path (e.g. `docs/notes.txt`).
- **Delete File? / Delete Folder?.** A confirmation dialog with a warning icon, the entry name, and its full relative path. For folders it adds, in red: "This will delete all files and subdirectories inside this folder permanently." **Delete** (red, bold) confirms; **Cancel** aborts.

Git operations

The **</> (Git operations)** toolbar button opens the **"Git Operations"** dialog, which inspects status, shows color-coded diffs, and performs commits, branch switches, pushes, and pulls against the project's Git repository.

Status and diff viewer

On open, the dialog loads `git status` output with a spinner ("Working…" plus a progress indicator) and shows it in a monospace pane; an empty result is reported as "Working directory clean". The pane renders up to 200 lines with color coding:

- Added lines (`+`): green-tinted text on a green-tinted background
- Removed lines (`-`): red-tinted text on a red-tinted background
- Hunk headers (`@@`): secondary color
- `diff` headers: primary color; `+++` / `---` markers: neutral variant color

Three outline buttons refresh the pane: **Status**, **Diff** (full working-tree diff), and **Branches** (lists branches under a "Branches:" heading). While any Git operation is in flight the buttons are disabled and the spinner shows.

Push, Pull, Issues

- **Push** (filled button): pushes the current branch; the pane reports "Push completed successfully!" plus output, or "Push failed:" plus the error.
- **Pull** (outline button): pulls; reports "Pull completed successfully!" or "Pull failed:" with details.
- **GitHub Issues & PRs** (outline button, when wired): closes the Git dialog and opens the GitHub Issues bottom sheet, from which an issue can be handed to the AI agent to solve.

Committing

A **"Commit Changes:"** section has a "Commit message" field and a full-width **"Commit All Changes"** button, enabled only when the message is non-blank and no Git work is in progress. On success the pane shows "Committed successfully!" and the message field clears; on failure it shows "Error:" with the details.

Branches

A **"Branch / Checkout:"** section has a "Branch name" field and two buttons:

- **Checkout** (outline): switches to the named existing branch ("Switched to <name>" or the error).
- **New Branch** (filled): creates the branch and switches to it ("Created & switched to <name>" or the error).

The dialog closes with the **Close** button.

> **Behind the scenes.** The dialog itself is pure UI; each button calls back into the app's Git layer, which runs the real `git` command inside the Linux runtime against the project folder and returns the output text and success flag that the dialog displays.

Code Editor

The code editor opens when you tap a file in the Files tab (or pick one from the command palette's quick-open). It is a full editing surface with syntax highlighting, multiple tabs, find and replace, code intelligence (go to definition, references, rename, diagnostics), a mobile symbol toolbar, and a split-view mode for viewing two files side by side.

Opening files and tabs

- Tapping a file in the Files tab opens it in the editor. Opening more files creates a **tab strip** under the top bar (it appears once more than one tab is open).
- The tab strip is horizontally scrollable, with **previous/next arrow buttons** at its ends that jump between tabs. Tapping a tab switches to it; the active tab is highlighted with a background and border.
- Each tab shows the file name and a small **X** to close that tab. If the file has unsaved changes, a small **dot** appears on the active tab as the dirty indicator.
- Your **cursor position and text selection are remembered per file** and restored when you reopen it (persisted editor view state).
- If a file is still loading, a spinner shows in the editing area. Very large files open as a **read-only preview** with the notice "Read-only preview. This file is too large to edit safely."; editing, find-replace, and the symbol toolbar are disabled for them.

Top bar

The top bar shows the **file name** as the title and the **full file path** beneath it (truncated with ellipsis if long).

Control	What it does
X (top-left, "Close editor")	Closes the editor and returns to the previous screen. If the file (or pane 2) has unsaved changes, an **"Unsaved changes"** dialog appears first: "Save your changes to <name> before closing?" with **"Save & close"** and **"Discard"**. Pressing the phone's system back button while dirty opens this same dialog.
Save icon (top-right)	Appears only when the file has unsaved changes. Tapping saves; a spinner replaces the icon while the save is in flight. Disabled while saving.
Magnifier ("Find and replace")	Toggles the find-and-replace bar.

Split icon ("Toggle split editor")	Toggles split view (two panes stacked vertically). The icon is tinted in the primary color while split view is on.
3-dot menu ("Editor actions")	Opens the full editor actions menu (detailed below).

The editing surface

- **Line numbers** run down a 48 dp gutter on the left, right-aligned, scrolling in sync with the text.
- Text is **monospace, 13 sp**, with the theme's primary color as the cursor.
- **Syntax highlighting** colors code as you type: keywords (e.g. `class`, `fun`, `if`, `return`, `import`, `function`, `def`) in the primary color, string literals in the tertiary color, comments (`#` and `//` to end of line) in the variant color, and numbers in the secondary color.
- **Bracket matching:** when the cursor sits on or next to a bracket, its matching partner is highlighted with a background tint, and the status bar reports "Match line N".
- **Word wrap** is on by default; long lines wrap instead of scrolling sideways. Toggle it from the menu ("Word wrap: On/Off").
- Code is indented with real spaces/tabs you type; the Tab key on the symbol toolbar inserts four spaces.

Find and replace in the file

Toggling the magnifier opens a panel with two rows:

- **Find row:** a "Find" field, a **"Sel"** toggle, and up/down arrows for previous/next match. The **Sel** toggle restricts the search to the currently selected text (the field label changes to "Find in selection"); turning it on captures the selection at that moment. Match navigation wraps around the document (or the selection) and is case-insensitive.
- **Replace row:** a "Replace with" field plus **Replace** (replaces the current match; if the selection does not match the query it first jumps to the next match) and **All** (replaces every occurrence). Both are disabled for read-only files.

Status bar

Along the bottom, a status line shows (monospace): the current line ("Ln 12"), character count ("340 chars"), "Unsaved" when dirty, the bracket-match line when applicable, and in split mode a "[Split] … | Pane 2: <name>" prefix. At its right is a **diagnostics chip**:

- Red "N err" with an error icon when errors exist; "N warn" with a warning icon for warnings only; green "0 err" with a check mark when clean.
- Tapping the chip opens the **Code Diagnostics** sheet.

Code completion

While you type, a row of **completion chips** can appear above the symbol toolbar. Each chip shows a suggestion label (monospace, primary color) and its kind (e.g. "function"). Tapping a chip inserts the completion, replacing the partial word before the cursor. Suggestions are computed from the document text, the cursor position, the detected language, and the symbols declared in the file.

The coding symbol toolbar

Below the editing area (hidden for read-only files) sits a horizontally scrollable **symbol toolbar**, so you never have to switch phone keyboards to reach code characters:

Keys	Behavior
Tab	Inserts four spaces at the cursor, or indents every line of the current selection by four spaces.
Untab	Removes one level of indentation (4 spaces, tab, or fewer spaces) from the current line or every line of the selection.
//	Toggles line comments: prefixes each line of the current line/selection with `//` (preserving indentation); pressing again removes it.
Left arrow / Right arrow	Move the text cursor one character left/right.
{ } ( ) [ ] < > " ' `	**Smart pairs:** with no selection, inserts the pair and places the cursor between them; with a selection, wraps the selection in the pair.
; : = -> => + - * / ! ? & | . , $	Inserts the operator at the cursor, replacing any selection.

Hardware-keyboard shortcuts

With a physical keyboard attached, the editor honors:

- **Ctrl+S**: save
- **Ctrl+F**: toggle find and replace
- **Ctrl+P**: open the command palette
- **Ctrl+W**: close (with the unsaved-changes dialog if dirty)
- **Ctrl+**: toggle split view

The editor actions menu (3-dot)

The menu lists every advanced action. Each item and what happens when you tap it:

Menu item	What it does
Command palette	Opens the editor command palette (see the Command Palette chapter).
Diagnostics (N)	Opens the **Code Diagnostics** bottom sheet listing N issues. Errors are tinted red, warnings neutral; each shows its message, line, column, and rule id. Tapping an entry jumps the cursor to that line. An empty list shows a green check with "No syntax errors or warnings detected in file."
Go to definition	Reads the symbol under the cursor and searches workspace files (up to 40 files under 1 MB) for its definition; jumps to the first exact match in the defining file. Needs an open workspace; tells you if the cursor is not on a symbol.
Find references	Searches the workspace for usages of the symbol under the cursor and opens a **References** sheet: "N usages found in workspace", each a card with file path and "Line N (TYPE)". Tapping a card jumps to it.
Rename symbol	Opens the **Rename** dialog (see below).
Undo rename	Reverts the last workspace rename.
Hover doc & info	Opens a **"Symbol Info: <name>"** dialog showing hover documentation for the symbol under the cursor (via the language support layer), or a fallback line "Symbol: <name> (<language> identifier)".
Code actions & fixes	Opens the **"Code Actions & Quick Fixes"** sheet for the cursor line. Tapping a fix applies its edit to the buffer immediately (you still review it and press Save yourself); each action shows its title and kind.

Find implementations	Builds the workspace symbol graph and opens an **Implementations** sheet for the symbol under the cursor; each entry shows name and `file:line (kind)`, and tapping jumps there (across files when a workspace is open).
Call hierarchy	Opens a **"Call Hierarchy"** sheet for the symbol under the cursor with **Incoming Callers (N)** and **Outgoing Calls (N)** sections, each listing up to 6 entries as "name in file:line".
Fold/unfold block	Collapses or expands the foldable block (brace-delimited region, etc.) containing the cursor.
Fold all blocks (N)	Collapses all N detected foldable regions in the file.
Unfold all blocks	Expands everything.
Split editor (Portrait stacked) / Close split editor	Toggles the two-pane split view (same as the toolbar split icon).
Project search & replace	Opens Global Search (only shown when a workspace folder is open).
Word wrap: On/Off	Toggles line wrapping.
Symbol outline	Opens the outline sheet (see below).
Go to line	Opens the Go-to-line dialog (see below).
Close other tabs / Close all tabs	Shown only when more than one tab is open; closes every tab except the current one, or all tabs.

Rename symbol (preview + undo)

The **"Rename symbol: <old>"** dialog has a "New symbol name" field (prefilled with the old name) and a note describing the rename engine as explicitly text-based. The flow is two-step:

1. **Preview:** enabled when the new name is non-blank and different. It walks the workspace and reports "N occurrence(s) in M file(s):", listing up to 8 affected files with per-file match counts (plus "…and K more file(s)").
2. **Rename:** enabled when at least one file is affected; applies the rename across the workspace, then reloads the open buffer from disk. **Undo rename** in the menu reverts it.

Symbol outline

The **"Symbol outline"** bottom sheet lists every detected declaration in the file ("N declarations in <path>"), each row showing the name, its line number, and a small kind tag (e.g. "declaration", "function", "class"). Tapping a row jumps the cursor to that line and closes the sheet. Detection first uses the semantic code engine; if it finds nothing, a fallback scans for common patterns: Kotlin `class` / `interface` / `object` / `fun` (including `data` / `enum` classes and `suspend fun`), Python `def` / `async def`, and JavaScript `function` (including `export` / `async`).

Go to line

A dialog with a digits-only "Line number" field and **Go**/**Cancel**. Go moves the cursor to the start of that line (clamped to the file's range) and scrolls it into view.

Split view (two panes)

Tapping the split icon divides the editing area into two stacked panes: the current file on top, and **Pane 2** below. A divider bar between them reads **"Pane 2: <filename>"** with:

- An **"• Unsaved"** indicator when pane 2 has unsaved edits.
- A **Save (pane 2)** icon, visible only when pane 2 is dirty, that saves just that pane's file (spinner while saving).
- A **"Switch tab"** dropdown listing all open tabs so you can load a different file into pane 2.
- An **X ("Close Split")** that ends split view.

Pane 2 tracks its own saved baseline independently, loads its file off the main thread (a load error shows inline in red), and has its own line numbers, scroll, and word-wrap setting. Guardrails protect your edits:

- Closing split view or switching pane 2's file while it has unsaved edits opens a **"Pane 2 has unsaved changes"** dialog with **"Save & continue"** and **"Discard"** (dismissing the dialog cancels).
- The main unsaved-changes dialog also mentions pane 2 ("Pane 2 (…) has unsaved changes too, and saving will include them.") and **"Save & close"** saves both panes.
- If both panes show the same file, they share one buffer: edits in either pane appear in the other, and pane 2 is never independently dirty for it.

Saving

- **Save** writes the current buffer to disk asynchronously; the dirty dot and "Unsaved" markers clear only after the write succeeds. Pane 2 saves go through a path-aware save for that pane's file.
- Edits are reported to the parent for **dirty-buffer tracking and session persistence**, so an interrupted session can restore what you typed.
- **New file / New folder from the command palette:** name dialogs accept project-root-relative names; names containing path separators or `..` are rejected so a new entry can never escape the project root. A toast confirms "Created <name>".

Run Build / Run Tests (from the command palette)

The palette's **Run Build** and **Run Tests** actions do not guess: they inspect the project folder for manifest files and resolve the real command:

Project marker	Build command	Test command
`build.gradle` / `build.gradle.kts`	`./gradlew build` (or `gradle build` without wrapper)	`./gradlew test` (or `gradle test`)
`package.json` with a `"build"` / `"test"` script	`npm run build`	`npm run test`
`Cargo.toml`	`cargo build`	`cargo test`
`go.mod`	`go build ./...`	`go test ./...`
`requirements.txt` / `pyproject.toml`	(none detected)	`python -m pytest`, only if a `tests/` / `test/` folder or pytest config exists

If no command can be detected, you get an honest toast: "No build command detected for this project." / "No test command detected for this project." When a command is found, it is **prefilled into the Terminal tab as a draft** (not executed): you land on the Terminal tab, review the command in the input line, and press send yourself. The destructive-command confirmation still applies.

Check Runtime Health

The palette's **Check Runtime Health** opens a dialog with a live snapshot computed when it opens:

- **Storage:** status plus "X free of Y", with an explanatory message when relevant.
- **Rootfs:** integrity status, listing any missing or corrupted runtime paths.
- **Memory:** "X used of Y max" for the app process.

Language support

The editor detects the file's language from its path (via the semantic code engine) and uses it for symbol extraction, folding ranges, diagnostics, completions, and the hover/code-action lookups. Hover, code actions, implementations, and call hierarchy are served by the in-app language-support layer over the workspace's parsed symbols; when a lookup has nothing to say, the UI says so plainly ("No code actions available at cursor line.", "No implementations found in workspace.") instead of inventing results.

Preview Browser

The **Preview** tab is a full Chromium-based browser built into Conzyx. It has two jobs: **previewing your project** (when a dev server is running, the first tab opens the project URL automatically) and **general web browsing** with multiple tabs, desktop mode, find-in-page, bookmarks, history, zoom, a DOM inspector, developer tools, and an AI visual debugger that can send a screenshot plus notes to the coding agent.

Start page

When no page is loaded, the tab shows a start page headed **"Chromium Browser & Preview"** with the subtitle "Multiple tabs, Desktop view, DevTools inspection, and online/local web navigation." Below it is a **"Self-hosted tools"** card ("Explore self-hosted systems, multi-agent harnesses, and big data datasets:") with tappable shortcut cards; tapping one navigates the browser to its URL:

Shortcut	Tag	Opens
Hermes 3D	Virtual Office	https://github.com/iamlukethedev/Hermes3D
OpenGym	Fitness Tracker	https://github.com/DuarteSantos8/openGym
TikTok 5.6B	Video Dataset	https://huggingface.co/datasets/datasocial/tiktok-5.6B-videos
GitFrames	WebGPU VFX	https://github.com/gatewai-dev/gitframes
AutoSocial	Automation	https://github.com/Katzca/AutoSocial
Munder Difflin	Office Harness	https://github.com/chaitanyagiri/munder-difflin

The address bar

The top row is a single slim pill (rounded, 40 dp tall) containing, left to right:

Control	What it does
Reload (circular arrow, left inside the pill)	Reloads the current page. It is tinted green when a page is loaded; with no page loaded it performs the same navigation as Go. Pressing the phone keyboard's Go key in the URL field also navigates.
URL field (center)	Type a URL, a bare domain, or just a port. Placeholder text: "Search or enter address". Uses a URL-optimized keyboard with a Go action.

X "Clear" (appears when text is present)	Clears the field (and any address error).
Go (blue circle with arrow, right)	Navigates to the normalized address.

How typed addresses are interpreted

Conzyx normalizes what you type before loading:

- **A bare port** (`3000` or `:3000`) becomes `http://localhost:3000` . This is the fastest way to open your dev server.
- `localhost` , `127.0.0.1` , **or** `0.0.0.0` (with optional port/path) get an `http://` prefix; `0.0.0.0` is rewritten to `127.0.0.1` .
- **Anything else without a scheme** (e.g. `example.com`) gets `https://` .
- `http://` , `https://` , `about:` , and `file://` are accepted as typed (the address must still have a host unless it is `about:`).
- If the input cannot be interpreted, a red error appears under the bar: **"Enter a valid URL or port (e.g. 3000, localhost:8080, google.com)"** and nothing loads.

The toolbar (second row)

Under the address bar is a horizontally scrollable row of action buttons. Tapping the **^ ("Hide toolbar")** button at its end collapses the row to a single **v ("Show toolbar")** chevron, giving the page maximum space; tapping the chevron restores it.

Button	What it does
Tab counter (number in a rounded square)	Shows how many tabs are open. Tapping toggles the **tabs strip** (described below). It is highlighted while the strip is visible.
Viewport switcher (phone/tablet/ laptop icon)	Opens a dropdown of viewport presets. The icon reflects the active preset and is tinted in the primary color for fixed-width presets. See "Viewport presets".
Debug with AI (sparkle icon, primary color)	Captures a screenshot of the current page, saves it as a visual snapshot, and opens the **AI Visual Debugger** dialog (see below).
Inspect Element (crosshair icon)	Toggles the **DOM touch inspector**: while active (green) you tap any element on the page and Conzyx reports the selected element's tag/

	classes; a toast confirms "DOM Inspector enabled: Tap element" / "disabled".
DevTools (developer-mode icon)	Opens the **DevTools sheet** (console logs, network logs, inspected element, ports, snapshots). A red **badge** on the icon shows the live count of console errors plus failed network requests (capped at 99); the icon itself turns red when any exist.
3-dot menu ("Browser options")	Opens the full browser options menu (see below).

> **No on-screen back/forward buttons.** Page-history navigation is handled by the phone's **system back** button/gesture: while the WebView can go back, system back goes back in page history instead of leaving the tab. There are no dedicated forward/back toolbar buttons.

The 3-dot browser options menu

Item	What it does
Desktop site	Toggles desktop mode for the current tab (check mark when active). Switches the user agent to desktop Chrome 130 on 64-bit Linux (mobile default is Chrome 130 on Android 14), enables wide viewport, and reloads the page.
New tab	Opens a blank "New Tab" and switches to it.
Close tab	Closes the current tab.
Find in page	Opens the find-in-page bar (see below).
Bookmark page / Remove bookmark	Adds the current page to bookmarks (star icon, orange when bookmarked) or removes it.
History (N)	Opens the History & Bookmarks sheet; N is the number of stored entries.
Extensions & Scripts	Opens the Extensions & Scripts dialog (see below).
Zoom in / Zoom out	Zooms the WebView content in/out (built-in zoom; no on-screen zoom controls are shown).
Clear browsing data	Clears the WebView cache and history, removes all cookies, and empties the in-app history list. Toast: "Browser cache and data cleared".

Chromium CDP (Port 9222)	Navigates to `http://127.0.0.1:9222`, the Chromium remote-debugging endpoint.
Inspect UI with AI	Attaches the current page as visual context to the AI agent and shows the toast "Attached visual preview context to Agent".

Tabs

The **tabs strip** appears automatically when more than one tab is open, or when toggled via the tab counter. Each tab chip shows a globe icon, the page title (up to 28 characters), and an X to close it; tapping a chip switches to that tab. A **+** button at the end opens a new tab. Behavior notes:

- New tabs start as "New Tab" with an empty address; tabs opened with a URL show "Loading…" until the title arrives.
- Closing the active tab activates a neighboring tab (preferring the previous one).
- Closing the last tab resets the browser to a single blank "New Tab" rather than leaving zero tabs.
- Each tab remembers its own URL, title, desktop-mode flag, and back/forward availability.
- Tabs, history, and bookmarks are **persisted as JSON** and restored after rotation or process recreation.

Find in page

Find in page (3-dot menu) opens a slim search bar under the tabs strip: a magnifier, the query field, a live **"current/total"** match counter, up/down arrows for previous/next match, and an X that dismisses the bar and clears the highlights. Matches are highlighted natively by the WebView as you type.

Loading and error states

- While a page loads, a thin **linear progress indicator** appears under the toolbar.
- If the main page fails to load (network error or HTTP error), a red **error banner** appears above the page: "Could not load this page: <reason>" or "Page failed to load (HTTP <code>)", with a **Retry** button. Sub-resource failures (a broken image or tracker) do not trigger the banner; they are recorded in the network log instead.

History and bookmarks

The **History** menu item (or its sheet) opens a bottom sheet with two sections:

- **Bookmarks** (green heading): each shows an orange star, title, and URL; tapping opens the page.
- **Browsing History**: the last **50** visited pages (title + URL), newest first; tapping opens the page. A **"Clear all"** button (red) empties history.

History entries are recorded automatically when a page finishes loading (duplicates of the most recent entry are skipped).

Extensions & Scripts

The **"Extensions & Scripts"** dialog injects developer tools, styles, or your own JavaScript into the active page:

Tool	What it does
Eruda Mobile DevTools: **Inject**	Loads the Eruda console (from `cdn.jsdelivr.net/npm/eruda`) into the page and shows it: a full mobile devtools panel with element inspector, DOM, console, and network tabs. If Eruda is already present, it is simply shown again.
DOM Element Inspector: **Inspect / Stop**	Same toggle as the toolbar crosshair: tap page elements to inspect their tags and classes.
Dark Reader CSS: **Toggle**	Injects (or removes) a stylesheet that inverts the page into a dark theme (`html { filter: invert(90%) hue-rotate(180deg) }` with images/video/canvas counter-inverted).
Run Custom Script	A monospace text area for arbitrary JavaScript plus an **"Execute JS"** button. The script runs in the page; its return value is shown as **"Output: <result>"** in green.

DevTools sheet

The **DevTools** button opens a sheet that centralizes everything the browser observes about the page. A diagnostics script is injected into every loaded page, and its `console.log` messages

are parsed into three streams: DOM-inspector selections, console log entries, and network log entries.

- **Console logs**: captured `console.*` output with severity levels; errors feed the toolbar badge.
- **Network logs**: requests with method, URL, status code/text; failures (network errors and HTTP errors) are flagged and feed the toolbar badge.
- **Inspected element**: the element picked with the DOM inspector, with its display name and CSS selector; the inspector can be toggled from the sheet too.
- **Detected ports**: localhost ports discovered in the project, with rescan, open-port (navigates to `http://localhost:<port>`), and add-custom-port actions.
- **Visual snapshots**: capture a screenshot of the current page, set baseline/comparison snapshots, and launch the **visual diff viewer** to compare before/after.
- **Clear logs** wipes the recorded console and network entries.

Viewport presets

The viewport switcher constrains the page to a fixed device width so you can check responsive layouts. Presets:

Preset	Width	Height
Responsive	Fills the available width (default)	
Pixel 8	412 dp	915 dp
iPhone 15	393 dp	852 dp
iPad Air	820 dp	1180 dp
Desktop 1080p	1920 dp	1080 dp

A fixed-width viewport renders the page inside a framed, elevated surface centered on screen; if it is wider than the display you can scroll horizontally to pan across it. The active preset's name is the switcher button's tooltip.

AI Visual Debugger

Tapping **Debug with AI** captures a screenshot of the current page and opens the **"AI Visual Debugger"** dialog:

- **Target:** the current page URL.
- **Selected element** (if you used the inspector): its display name and CSS selector.
- **Error summary:** counts of console errors and failed requests ("No runtime errors detected" in green when clean).
- **Screenshot:** the saved snapshot's file path.
- **"Describe problem or instruction for AI:"** a text field (placeholder: "e.g. Fix button height and resolve 404 endpoint").

"Send to AI Agent" (green button) dispatches the URL, your notes, and the screenshot as a visual bug report to the coding agent, closes the dialog, shows "Visual bug report dispatched to AI Agent", and switches you to the **Chat** tab. **Cancel** closes without sending.

Security model

- JavaScript and DOM storage are enabled (pages need them); **file:// URL access is disabled**, **mixed content is never allowed** (no http subresources on https pages), and content URIs remain available so file uploads work.
- Only `http`, `https`, and `about` URLs load inside the WebView. `tel:`, `mailto:`, and `sms:` links open the appropriate external app; all other schemes (including `intent://`) are blocked.
- Cookies are accepted, including third-party cookies, so logins and sessions work.

Typical preview workflow

1. In the **Terminal** tab, start your server, e.g. `npm run dev` or `python3 -m http.server 8080`.
2. Open the **Preview** tab; the project URL is loaded automatically when the runtime reports the project ready, or type just the port (`8080`) into the address bar.
3. Use the **viewport switcher** to check mobile and desktop layouts.
4. Open **DevTools** to watch console errors and failed requests while you interact with the page; the badge tells you at a glance if anything broke.
5. Spot a visual bug? Tap **Debug with AI**, optionally tap the offending element with the inspector first, describe the problem, and **Send to AI Agent**: the agent receives the screenshot, the element selector, the error counts, and your note in the Chat tab.

Command Palette

Conzyx has **two** command palettes, and they serve different scopes:

- **The workspace command palette** (top bar of the IDE, 3-dot/"Command palette" icon): jumps between the app's engineering tools without leaving the workspace.
- **The editor command palette** (inside the code editor, 3-dot menu → "Command palette", or **Ctrl+P** on a hardware keyboard): runs editor actions, file operations, navigation, and build commands, and quick-opens files by name.

Workspace command palette

Opened from the IDE top bar's **3-dot "Command palette"** icon (visible on every tab except Preview and Terminal, which use compact headers). It is a bottom sheet titled **"Command palette"** with the subtitle "Jump between engineering tools without leaving the workspace." A **"Search commands"** field filters the list by title or keywords; with no match it shows "No commands match your search." The commands:

Command	What it does
Go to Chat	Switches to the Chat tab (AI conversation assistant).
Go to Files	Switches to the Files tab and refreshes the file tree.
Open Terminal	Switches to the Terminal tab and prepares the terminal session.
Review Changes	Switches to the Changes tab (pending edits and diffs).
Open Preview	Switches to the Preview browser tab.
Refresh Project Files	Reloads the file tree from the filesystem (stays on the current tab).
Bind Folder from Device	Opens the system folder picker to attach a folder from phone storage as the project.
Android App Factory	Opens the App Factory (blueprint-driven Android app scaffolding).
Multi-Agent Engineering Team	Opens the Team Studio (multi-agent team orchestration).
Cloud + Phone Hybrid	Opens the Cloud Hybrid screen (offload work to cloud runners).
AI Developer Marketplace	Opens the Marketplace (agents, MCP servers, skills, templates).

Editor command palette

Opened from the editor's 3-dot menu → **"Command palette"** or **Ctrl+P**. It is a bottom sheet titled **"Command Palette"** with an X close button, a search field ("Type command or file name..."), and category filter chips: **All, Editor, Nav, Files, Build**. Typing filters actions by title, description, and keywords (e.g. typing "wrap" finds Toggle Word Wrap, "grep" finds Global Search).

Quick-open files

When the search box is non-empty and no category chip is selected, the sheet first shows a **"Matching Files (N)"** section with up to 8 files whose project path contains the query. Each row shows the file name, its full relative path, and an **"Open"** label; tapping opens the file in the editor and closes the palette. Below it, the matching **"Actions"** continue.

All commands by category

Each action row shows an icon, the title, a one-line description, and a small category tag. The full command list (the Editor-category actions only appear while a file is open in the editor):

Category	Command	Description (as shown)	What it does
Editor	Save File	Save current changes to disk	Saves the open file (same as the toolbar Save icon).
	Find & Replace	Search and replace within current file	Opens the editor's find-and-replace bar.
	Global Search & Replace	Search across the entire workspace	Opens Global Search (see next chapter).
	Go to Line	Jump to a specific line number	Opens the Go-to-line dialog.
	Symbol Outline	Inspect classes, functions, and symbols	Opens the symbol outline sheet.
	Toggle Word Wrap	Wrap long lines in editor	Toggles word wrap immediately.
	Close Tab		Closes the current file tab.

		Close current editor tab	
	Close Other Tabs	Keep only active file open	Closes every tab except the current file.
	Close All Tabs	Close all open editor tabs	Closes every editor tab.
Files	New File	Create a new file in workspace	Opens the New File dialog (project-root-relative name; separators and `..` rejected).
	New Folder	Create a new directory	Opens the New Folder dialog (same naming rules).
	Refresh Workspace Files	Reload file tree from filesystem	Reloads the Files tab tree.
	Global Search	Search all workspace files	Opens Global Search (same as the Editor-category entry).
Nav	Open AI Chat	Switch to AI conversation assistant	Goes to the Chat tab.
	Open File Explorer	Browse project directory tree	Goes to the Files tab.
	Open Terminal	Access interactive shell environment	Goes to the Terminal tab.
	View Git Changes	Review pending edits and diffs	Goes to the Changes tab.
	Open Web Preview	Launch local web preview	Goes to the Preview browser tab.
Build	Run Build	Execute project build command	Detects the project's real build command from manifest files and prefills it as a draft in the Terminal tab for review; honest "No build command detected" toast if none is found.
	Run Tests	Execute test suite	Same flow for the test command.

	Check Runtime Health	Inspect storage, rootfs, and memory	Opens the Runtime Health dialog (storage free/total, rootfs integrity, memory usage).

Every palette action does something real. Selecting an action always dismisses the sheet and then performs the action (or tells you plainly when it cannot, e.g. no build command detected). There are no dead entries.

How the two palettes differ

	Workspace palette	Editor palette
Opened from	IDE top bar, 3-dot "Command palette" icon	Editor 3-dot menu → "Command palette", or Ctrl+P
Scope	App-level navigation and tools	Editor actions + file ops + navigation + build
Categories	None (flat searchable list)	Editor / Nav / Files / Build filter chips
File quick-open	No	Yes (up to 8 path matches)

Global Search

Global Search ("Project Search & Replace") searches file contents across the entire workspace folder, with match-case, whole-word, and regex options, and an optional replace mode with per-file preview and undo. It is a bottom sheet dialog, opened from:

- The editor's 3-dot menu → **"Project search & replace"** (shown when a workspace folder is open)
- The editor command palette → **"Global Search & Replace"** or **"Global Search"**

When opened from the editor, the current find text is carried over as the initial query.

Searching

- Type in the **"Search everywhere..."** field. Searching starts automatically **300 ms after you stop typing** (debounced), so results update as you type without hammering the filesystem.
- A spinner appears next to the option chips while a search runs. All searching happens on a background thread.
- The result header reports e.g. **"24 matches across 5 files"**, with "(truncated)" appended when the engine caps a very large result set.
- Each match is a card showing a document icon, the **file path** (primary color), the **:line number** (monospace), and up to two lines of the matching **line content** (monospace). Tapping a card closes the sheet and jumps to that file and line in the editor.
- No matches shows "No matches found."

Search options (filter chips)

Chip	Option	Effect
Aa	Match case	Case-sensitive matching when on; case-insensitive by default.
Word	Whole word	Matches only whole words (no partial-word hits).
.*	Regex	Interprets the query as a regular expression.
Replace	Replace mode	Reveals the replacement field, Preview button, and undo controls (see below).

Changing any option re-runs the search immediately.

Replace mode (preview + undo)

Toggling the **Replace** chip reveals a **"Replace with..."** field and a **Preview** button. Replace is deliberately a two-step, reviewed operation:

1. **Preview.** Builds a replace plan across the workspace and reports **"N occurrence(s) in M file(s). Uncheck files to skip them."** Each affected file is listed with a checkbox and its match count (the list scrolls within a capped height). Uncheck any file to exclude it from the replacement. The **"Replace in N file(s)"** button is enabled only when at least one file is checked.
2. **Apply.** Writes the replacements and reports **"Replaced X occurrence(s) in Y file(s)."**, then re-runs the search so you see the new state.
3. **Undo.** After applying, an **"Undo replace"** button appears and restores the changed files ("Undo complete: N file(s) restored."). For very large changes undo may be unavailable, in which case the button reads **"Undo unavailable (large change)"** and is disabled.

A new search invalidates any pending preview, and applying a replace clears the previous undo transaction, so each replace stands on its own.

> **Scope.** Global Search covers the open workspace folder (the project root). It searches file contents; to find files by name, use the Files tab's "Search project files" field or the command palette's quick-open instead.



Pen Testing

What Pen Testing Is

Pen Testing is a full multi-agent AI penetration testing system that runs on your phone. You describe a target and a goal in plain language; a team of specialized AI agents plans the work, runs professional security tools inside an isolated Linux environment, and hands you a report. It lives in the Studios section of the home screen as the Pen Testing card.

One-Tap Setup

Tapping the Pen Testing card opens the setup screen. The screen first checks the installation state, then offers a single **Setup Pen Testing** button that runs the whole installation with no further input:

1. **Download.** The app fetches `pentagi-ondevice-test.tar.gz` (about 57 MB) from the `pentagi-ondevice-1` GitHub release and verifies its SHA-256 checksum. Progress is shown live in megabytes, and the download resumes if it is interrupted.
2. **Extract.** The package is unpacked into the proot Linux environment at `/root/pentagi-setup`, and the backend binary is marked executable.
3. **Install.** The setup script installs PostgreSQL with the pgvector extension, initializes the database, and boots the backend with the proot-native executor. No Docker daemon is needed.

Each step is shown with a live log tail. The first setup takes 10 to 15 minutes. Keep the setup screen open until it finishes; leaving the screen cancels the run.

> **Note:** The core Linux runtime must be installed before Pen Testing setup can run. If it is missing, the setup screen tells you to complete the main runtime setup first.

The Web UI

When the backend is up, the setup screen shows the API and web UI addresses. Tapping **Open Web UI** loads the backend interface at `http://127.0.0.1:18080` inside the app's own in-app browser, so you never leave the app. From the web UI you start assessments, watch the agents work, and read their reports. The phone's Back button returns you to the setup screen.

Managing the Backend

The setup screen doubles as the backend's control panel:

- **Start Pen Testing** boots the backend when it is installed but not running.
- **Stop** shuts the backend down.
- **Reinstall** wipes the installation and runs setup again from scratch.
- **View Logs** tails the backend log at `<rootfs>/root/pentagi-data/backend.log`, the first place to look when something fails.

If the phone's battery optimization stops the backend in the background, start it again with one tap.

Requirements and Troubleshooting

- Internet is required once, for the 57 MB download. After that everything runs on-device.
- The first setup takes 10 to 15 minutes and times out after 20 minutes. On a timeout, check your connection and retry.
- If setup fails, the error message shows the last lines of the install log; View Logs has the full story.
- The backend listens on port 18080 on the device loopback interface only. It is not reachable from other devices on your network.

Chapter D1: Skills

1.1 What a skill is

A skill is a reusable bundle of agent knowledge. Think of it as a small instruction pack that teaches Conzyx's AI agents how to do a particular job well: browser automation, logo design, Android development, contract review, and dozens of other specialties. When a skill is installed, the app copies it into the agent's working knowledge so agents can consult it while they work on your projects.

Technically, a skill is a folder with a `SKILL.md` manifest at its root. The manifest is plain Markdown that starts with YAML front matter containing two required fields:

- **name**: a short lowercase identifier such as `browser-automation`.
- **description**: a one line summary of what the skill does.

The body of the manifest then holds the actual instructions: capabilities, commands, and usage examples the agent should follow. Skills are installed at user level, inside the app's own private storage on your device (not system-wide), and Conzyx treats imported skills as untrusted project content: they stay inside normal safety, approval, and workspace boundaries.

A skill folder may also carry optional subfolders that the app creates automatically when they are missing:

- `instructions`: extra guidance files.
- `tools`: helper scripts or tools for the skill.
- `examples`: example inputs and outputs.
- `resources`: reference material.

Skills are kept in the app's private `skills` folder. When you open a project, the app syncs your installed skills into that workspace under `.agents/skills`, and stale skill folders that you removed are cleaned out of the workspace automatically. The active agent also receives a prompt block listing every installed skill by name and description, with the instruction to inspect the matching `SKILL.md` whenever a skill's description applies to the task.

> **Note:** Skills give agents knowledge, not new powers. A skill cannot bypass the permission toggles in Settings or the safety boundaries of the app. It only tells the agent how to work better inside the boundaries you already set.

1.2 The built-in skill library

Conzyx ships with 28 built-in skills that are created automatically the first time the skill system runs. You never have to install these; they simply appear in your installed list. Each one has the source label `built-in`, or a GitHub repository as its source where one exists.

Skill name	What it does
`browser-automation`	Controls Chromium via Puppeteer: browse websites, fill forms, click buttons, take screenshots, inspect elements, and automate web tasks.
`geo-sleuth`	Geolocates photographs by analyzing OpenStreetMap geometry, elevation skylines, satellite imagery, and street evidence.
`stickman-video-director`	Directs 60-second stickman explainer animations: generates 6-scene storyboards and consistent video prompts.
`api-anything`	Turns any website into a self-healing API for AI agents with reverse-engineered endpoints, action flows, and an MCP server.
`logo-design-skill`	Disciplined brand identity workflow: crafts, refines, and stress-tests scalable SVG logos, favicons, and asset kits.
`anyps5-director`	Translates, relinks, and runs PS5 executables natively; manages shader recompilation and binary verification.
`youtube-video-intel`	Deep multimodal YouTube video understanding: timestamped transcripts, visual concepts, code snippets, executive digests.
`fridge-vision-chef`	Computer vision meal planner: analyzes fridge and pantry photos, estimates freshness and calories, generates recipes.
`receipt-expense-sleuth`	Personal finance auditor: scans receipts and invoices, flags hidden subscriptions and duplicate charges, exports ledgers.
`contract-legal-buster`	Rental lease and legal contract auditor: exposes predatory clauses with plain-language risk scoring.
`smarthome-iot-commander`	Home automation orchestrator for Home Assistant, ESPHome, and Zigbee/Matter devices with daily routines.
`academic-paper-architect`	PhD research paper assistant: LaTeX/BibTeX templates, literature reviews, methodology sections, peer-review rebuttals.

`document-transmuter`	Universal document transformation: converts Markdown, PDF, DOCX, and LaTeX with table extraction and OCR cleanup.
`marp-presentation-deck`	Slide deck generator using Marp/RevealJS: layouts, diagrams, speaker notes, PDF/HTML export.
`creative-image-director`	Visual prompt engineering for AI image generators: camera angles, color palettes, and SVG asset output.
`viral-growth-creator`	Social media growth engine: YouTube hooks, X threads, LinkedIn carousels, SEO packaging.
`fullstack-dev-accelerator`	Fullstack engineering co-pilot: architectures, refactors, test suites, CI/CD pipelines, API contract validation.
`android-development`	Modern Android development with Jetpack Compose, Kotlin Coroutines, MVVM, Gradle builds, and APK generation.
`flutter-development`	Cross-platform mobile apps with Flutter and Dart, reactive state management, and custom widgets.
`laravel-development`	PHP 8 web engineering with Laravel, Artisan CLI, Eloquent ORM, RESTful APIs, and migrations.
`react-development`	Modern React development: custom hooks, functional patterns, TypeScript JSX, Tailwind CSS.
`nextjs-development`	Fullstack Next.js apps: App Router, React Server Components, SSR/SSG, server actions, API routes.
`nodejs-development`	Backend Node.js service engineering with Express, Fastify, TypeScript, async I/O, and npm tooling.
`python-development`	Python 3 development with FastAPI, Flask, Django, virtualenvs, pip isolation, and typing.
`devops-engineering`	DevOps automation: multi-stage Docker containers, CI/CD authoring, shell scripting, Linux administration.
`database-engineering`	Database design and SQL: PostgreSQL, SQLite, MySQL schema migrations, indexing, query optimization.
`testing-qa`	Test-driven development: unit testing (JUnit, Jest, PyTest), integration mocking, automated verification.

`ui-ux-design`	Design systems, accessibility standards, responsive touch UI layouts, Material 3 design tokens.

1.3 The Skills screen: Agent Skills tab

Open the Skills screen from the app's navigation. The top bar shows the title "Skills Platform" with the subtitle "Reusable multi-folder agent skills". Below it are two tabs:

- **Agent Skills (N)**: the skill manager described in this chapter. The number shows how many skills are installed.
- **MCP Servers (N)**: the Model Context Protocol server platform (section 1.6).

The Agent Skills tab scrolls through three cards, then your installed skills, then any GitHub search results.

```
Skills Platform - Reusable multi-folder agent skills
[ Tab: Agent Skills (29) ] [ Tab: MCP Servers (0) ]
Card: Search GitHub Skills - search box + "Search skills" button
Card: Direct Import - GitHub URL field, "Import URL" and "Upload ZIP" buttons
Card: Auto-update skills - switch + "Check updates" button
Heading: Installed Agent Skills (29)
Skill row: name, description, source, [Update if available] [delete]
```

1.3.1 Search GitHub Skills

The first card is titled **Search GitHub Skills** with the description "Find and explore reusable capabilities from GitHub". It has a search box with the placeholder "e.g. android, react, devops, database..." and a full-width **Search skills** button.

- The button stays disabled until you type at least 2 characters.
- The search queries GitHub's repository search for "agent skills" plus your keywords and returns up to 20 repositories.
- While the search runs, the button shows a spinning indicator; only one search runs at a time.
- Each result appears under a **GitHub results** heading as a row with the repository name, its description, its URL, and an **Import** button that installs that repository as a skill immediately.
- After a search finishes, a status line under the Auto-update card reports how many repositories were found, e.g. "Found 14 repositories."

1.3.2 Direct Import

The second card is titled **Direct Import**, described as "Import from GitHub repo URL or local ZIP package". It has two controls side by side:

- **GitHub URL field + Import URL button:** paste a public HTTPS GitHub repository URL such as `https://github.com/owner/skill-repository`. The button enables only when the field starts with `https://github.com/`. Tapping it downloads the repository (main branch first, master as fallback) and installs every skill folder it finds. The field clears after the import starts.
- **Upload ZIP button:** opens Android's document picker filtered to ZIP files. Pick a skill package ZIP from your device to import it. The source is recorded as the file's URI.

1.3.3 Auto-update skills

The third card is titled **Auto-update skills**, described as "Automatically keep installed skills up to date". It has:

- **A switch (default: off).** Turning it on saves the preference and immediately runs a full update pass: it checks every installed skill, every installed agent/CLI tool, and every installed development toolchain, and updates anything outdated. When it is off, nothing updates by itself.
- **A "Check updates" button.** Tapping it checks your installed skills against their GitHub sources and reports which ones have changed. It is disabled while another skills operation is running.
- **A status line** under the button showing the latest message, such as "2 skill updates available." or "All remotely sourced skills are up to date." If a check could not reach a skill, the message names it instead of pretending everything is fine (e.g. "Could not check for skill updates · Could not check my-skill").

Only skills imported from a GitHub URL are checked. Built-in skills and ZIP imports have no remote source, so they are skipped.

1.3.4 Installed Agent Skills

Below the cards, the heading **Installed Agent Skills (N)** lists every installed skill as a row with:

- The skill name in semibold text.
- The skill description (up to 3 lines).
- The source (one line): `built-in`, a GitHub URL, or a file URI for ZIP imports.
- An **Update** button, shown only when the skill has a pending update and its source is a GitHub URL. Tapping it re-imports the skill from its repository.
- A trash icon that removes the skill. Removal deletes the skill's folder and its registry entry.

While no skills are installed and no search results exist, the screen shows a centered empty state: "No skills installed yet" with the hint "Search GitHub above or import one directly." If you searched and nothing matched, it shows "No skills found for 'your query'" with "Try different keywords, or import one directly below."

Chapter D1 continued: Skills import pipeline, updates, and the MCP Platform tab

1.4 What happens during an import

Whether you import from a GitHub URL, a search result, or a local ZIP, the same pipeline runs:

1. **Fetch.** For GitHub URLs, the app downloads the repository as a ZIP from the branch named `main`, falling back to `master`. Only public HTTPS `github.com` URLs are accepted.
2. **Safety gate.** Archives larger than 100 MB are rejected. Entries are capped at 10,000 files. The extractor refuses absolute paths and `..` segments, and verifies that no extracted file escapes the extraction directory (zip-slip protection).
3. **Manifest scan.** The app walks the extracted tree looking for `SKILL.md` (or `skill.md`, up to 100 manifests). If none is found, the import fails with the message that no `SKILL.md` was found. A valid manifest must start with YAML front matter containing a `name` and a `description`; otherwise the import fails.
4. **Install.** Each skill folder is copied into the app's private `skills` directory under a sanitized name (lowercased, unsafe characters replaced with hyphens, max 80 characters). If a skill with the same name already exists, it is replaced. The portable layout folders (`instructions`, `tools`, `examples`, `resources`) are created when missing, and the registry is updated with the name, description, source, install time, and path.

One repository can install several skills at once: every folder containing a `SKILL.md` becomes its own installed skill, and the completion message reports how many skill bundles were imported.

> **Honest failure:** if a download fails, a manifest is missing or malformed, or an archive is unsafe, the import reports the real reason. It never claims success when nothing was installed.

1.5 Updating skills in detail

Manual update check ("Check updates" button): for each installed skill whose source is a GitHub URL, the app re-downloads that repository's ZIP and compares the remote `SKILL.md` content against the installed one. Skills whose manifests differ are marked as having updates, which makes the per-skill **Update** button appear. Skills that cannot be checked (network

failure, repository gone) are reported by name in the status line; they are never silently reported as up to date.

Per-skill update: re-runs the full import pipeline for that skill's source URL, replacing the installed copy. The status line confirms "skill-name updated successfully."

Automatic update engine (the "Auto-update tools & skills" switch in Settings, mirrored on the Skills screen): when enabled, the app runs a combined update pass that covers three things in order: (1) universal skills via the check described above, (2) installed agent/CLI tools, and (3) installed development toolchains. It runs at most every 4 hours unless forced, and never while a task, terminal session, or install is in progress. The final message lists what was updated and what failed; a failed update is never reported as "up to date".

1.6 The MCP Platform tab

The second tab of the Skills screen is titled **MCP Platform** with the subtitle "Model Context Protocol tools & servers". MCP (Model Context Protocol) servers are small programs that expose tools, for example database queries, web fetching, or file operations, which agents can call. The top bar gains an **Add** (+) button here that opens the custom server dialog.

The tab has three parts:

1.6.1 Installed Servers

The heading **Installed Servers (N)** sits next to an **Add Custom** text button. Each installed server is a card showing:

- The server name and an **ACTIVE** or **DISABLED** badge.
- Its description.
- An on/off **switch** that enables or disables the server without removing it.
- The launch command in monospace (the executable plus its arguments).
- A **Discover Tools** button (enabled only when the server is active) that queries the server for the tools it offers. Discovered tools appear inside the card with their names, descriptions, and a **Run** button per tool.
- A trash icon that deletes the server.

With no servers installed, the section shows: "No MCP servers installed yet. Choose a server from the catalog below or add a custom one."

1.6.2 MCP Catalog & Templates

Below that, the heading **MCP Catalog & Templates** ("One-click install pre-configured Model Context Protocol servers") lists 9 built-in templates. Each template card shows the name, a category chip, the description, the default command in monospace, and either an **Install**

button or a green **Installed** badge. The same catalog also feeds the Marketplace's MCP Server listings, so the two surfaces can never disagree on names, descriptions, or install commands.

Template	Category	What it does	Default command
SQLite Server	Database	Interact with and query SQLite databases directly via MCP tools.	`npx -y @modelcontextprotocol/server-sqlite`
Filesystem Explorer	Storage	Read, write, and inspect local project directories with secure boundaries.	`npx -y @modelcontextprotocol/server-filesystem`
GitHub Platform	Development	Search repos, create issues, view pull requests, and commit files via GitHub API.	`npx -y @modelcontextprotocol/server-github`
PostgreSQL Server	Database	Inspect database schemas, query tables, and execute SQL statements.	`npx -y @modelcontextprotocol/server-postgres`
Fetch & Markdown	Web	Fetch web pages, extract content, and convert HTML to readable Markdown for agents.	`npx -y @modelcontextprotocol/server-fetch`
Puppeteer Automation	Web Automation	Automate browser interactions, capture screenshots, and scrape dynamic pages.	`npx -y @modelcontextprotocol/server-puppeteer`
Knowledge Graph Memory	Agent Memory	Persistent graph-based memory for entities, relationships, and context recall.	`npx -y @modelcontextprotocol/server-memory`
Brave Search	Search	Perform private web searches and retrieve up-to-date web answers.	`npx -y @modelcontextprotocol/server-brave-search`
Sequential Thinking	Utility	Structured step-by-step reasoning and hypothesis refinement server.	`npx -y @modelcontextprotocol/server-sequential-thinking`

Three of these templates need credentials to work: GitHub Platform needs `GITHUB_PERSONAL_ACCESS_TOKEN`, PostgreSQL Server needs `POSTGRES_CONNECTION_STRING`, and Brave Search needs `BRAVE_API_KEY`.

1.6.3 Add Custom MCP Server

The **Add Custom** button (or the + icon in the top bar) opens the **Add Custom MCP Server** dialog. It starts with a red warning: "this command will run on your device with the app's full privileges. Only add servers from sources you trust."

Field	What to enter
Server ID	A unique id such as `my-server`. If left blank, the app generates one from the current time.
Display Name	The human-readable name shown on the card. Defaults to the Server ID if blank.
Executable Command	The program to run, e.g. `node`, `python`, or `npx`. Required.
Arguments	Space-separated arguments passed to the command.
Environment	Comma-separated `KEY=VAL` pairs for non-secret settings. Entries without `=` are ignored.
Secrets	Comma-separated `KEY=VAL` pairs for sensitive values. Stored encrypted in the credential store, never in the server config. Do not put secrets in the Environment field.
Acknowledgement checkbox	"I understand this command runs with the app's full privileges." Required before the Add Server button enables.

The **Add Server** button enables only when the Executable Command is filled in and the checkbox is ticked.

1.6.4 Running a tool manually

Tapping **Run** next to a discovered tool opens the **Run tool-name** dialog with an **Arguments (JSON)** text field (prefilled with an empty JSON object). If the arguments are not valid JSON, the dialog shows an error instead of running. Depending on the tool's permission policy, you may be asked to confirm: **Back** returns to the argument editor, **Allow once** runs it this time, **Always allow** approves future runs. If the tool is not allowed at all, the dialog shows the reason instead of running. While a tool runs, a dialog shows "Running…"; when it finishes, the dialog shows the result (or the error) and how many milliseconds it took. The **Close** button is disabled until the run completes.

1.6.5 MCP Execution Audit Trail

When tool runs have been logged, a section titled **MCP Execution Audit Trail** appears with a red **Clear** button that wipes the log. It shows the most recent 20 entries. Each entry shows the server and tool name (`serverId → toolName`), the outcome (**SUCCESS (Nms)** in green or **ERROR** in red), the JSON arguments, a result preview, and any error text.

> **Free and open:** every skill and MCP server on these screens is free to install and use. There are no paid listings, no purchase gates, and no payment flows anywhere in the app.

Chapter D2: Marketplace

2.1 What the Marketplace is

The **Marketplace** (titled "Agents, MCP, Skills & Templates" under its header) is Conzyx's in-app catalog of reusable AI developer components. It lists four kinds of items:

- **Agents:** pre-configured AI agent profiles backed by well-known models.
- **Skills:** reusable agent skill bundles, the same kind of content managed on the Skills screen.
- **MCP Servers:** Model Context Protocol servers exposing tools to agents.
- **Templates:** project starter templates for common stacks.

Everything in the Marketplace is **fully free and open source**. There are no prices, no paid listings, no purchase gates, and no payment flows anywhere in the app. Installing an item costs nothing; the only thing some items need is your own credentials (for example an API key) to function, and you keep full control of those.

The catalog mixes curated listings (built into the app) with items you publish yourself from the device. Items you publish are saved on the device only; no registry is configured, so nothing you publish is shared with other devices.

2.2 Browsing the catalog

The top bar shows a storefront icon, the title **Marketplace**, and the subtitle "Agents, MCP, Skills & Templates". On the right are two icons:

- **Search** (magnifier): opens a search field that drops down from the top of the screen over a dimmed backdrop. Type to filter the catalog by name, author, description, or tags ("Search catalog by name, author, or tags..."). The X button clears the query and closes the search; tapping the dimmed backdrop also closes it.
- **Publish** (+): opens the "Publish to Marketplace" dialog (section 2.9).

Below the top bar, a horizontally scrollable row of filter chips controls what you see:

Chip	What it does
All (N)	Shows the whole catalog. The number is the total count of items.
Installed (N)	Toggles a view that shows only items you have installed. The number is your installed count.
Agents (N)	Shows only agent profiles.

Skills (N)	Shows only skills.
MCP Servers (N)	Shows only MCP servers.
Templates (N)	Shows only project templates.

Next to the chips, a sort control shows the current order ("Relevance", "Top rated", or "Most installed") with a sort icon that opens a dropdown menu:

- **Relevance:** the catalog's default order.
- **Top rated:** highest rated first.
- **Most installed:** highest download count first.

The current choice is marked with a check in the menu. Empty states are explained: a search with no matches says "No items match 'your query'" with "Try a different search."; the Installed view with nothing installed says "Nothing installed yet" with "Browse the catalog and tap Install on anything you want."; an empty category says "No items in this category yet" with "Try another category."

2.3 Reading a listing card

Every listing is a card with the same anatomy:

- **Name** in bold title text, with a **Custom** badge beside it for items you published yourself.
- **by author** underneath, e.g. "by morluto".
- A **category pill** on the right (AGENTS, MCP SERVERS, SKILLS, or TEMPLATES) with a color per category.
- The **description** in small text.
- **Tags** as chips like `#rust` or `#android`, scrollable horizontally.
- A rating row: either a gold star with the rating and "N installs", or the honest **"No ratings yet"** when no rating exists. The app deliberately never fabricates ratings or download counts.
- Action buttons at the bottom right, which change with state (section 2.5).
- A trash icon on cards for items you published, letting you delete your own custom listing.

```
Marketplace card mock:
rea [Custom?] ..... [SKILLS pill] [trash if custom]
by morluto
Reverse engineer anything with agents, from app behavior down to native binaries.
#reverse-engineering #agents #security #binaries
```

star 0.0 or "No ratings yet" ..... [Install] or [Update available][Update][Installed - Remove]

2.4 What is in the catalog

The curated catalog ships with the app and is the same on every device. It has 8 skills, 9 MCP servers, 4 templates, and 4 agents.

2.4.1 Skills

Skill	Author	Description	Source
rea	morluto	Reverse engineer anything with agents, from app behavior down to native binaries.	github.com/morluto/rea
desigma	deepfounder-ai	Free Figma alternative made on Rust.	github.com/deepfounder-ai/desigma
flick	creatorberry	Give it a video or a script. Turn it into a scene-by-scene animations.	github.com/creatorberry/flick
app-store-screenshots	ParthJadhav	End to end app store screenshot creation using AI.	github.com/ParthJadhav/app-store-screenshots
Jetpack Compose Architect	Android Dev Community	Best practices, performance guidelines, and state hoisting for modern Android UI.	github.com/pocketdev/skills-compose-architect
DevOps & Docker Packager	DevOps Labs	Multi-stage Dockerfiles, GitHub Actions workflows, and Linux container automation.	github.com/pocketdev/skills-devops-packager
Next.js 15 Full-Stack Wizard	React Core Community	Server components, server actions, route handlers, and SEO optimization for Next.js.	github.com/pocketdev/skills-nextjs-wizard
Rust Memory & Systems Optimization	Rust Foundation Contribs	Borrow checker mastery, zero-cost abstractions, and unsafe code auditing.	github.com/pocketdev/skills-rust-systems

The first four rows are the **pre-seeded skills**: on first launch, the app automatically seeds these four into your installed set (recorded as installed with the upstream commit it saw at seed time), so they are ready without you doing anything. Because their install state is stored durably, the seeding happens exactly once per device.

2.4.2 MCP Servers

The nine MCP server listings are derived from the same template list used by the Skills screen's MCP catalog (section 1.6.2), so names, descriptions, and install commands always match: SQLite Server, Filesystem Explorer, GitHub Platform, PostgreSQL Server, Fetch & Markdown, Puppeteer Automation, Knowledge Graph Memory, Brave Search, and Sequential Thinking. Each install value is a launch command such as `npx -y @modelcontextprotocol/server-sqlite --db workspace.db`.

2.4.3 Templates

Template	Author	Description
Android Kotlin Clean Starter	Conzyx	Jetpack Compose + Material 3 + Coroutines production starter with Gradle kts.
FastAPI Backend Microservice	Python Squad	FastAPI with Pydantic v2, async endpoints, SQLAlchemy 2.0, and pytest suite.
React Native Expo Mobile Starter	Conzyx	TypeScript Expo router starter with NativeWind styling and offline SQLite caching.
Rust Axum High-Performance API	Systems Lab	Blazing fast Tokio + Axum async HTTP server with Tower middleware and tracing.

2.4.4 Agents

Agent	Author	Description
Claude 3.7 Sonnet Engineer	Anthropic	Hybrid reasoning and coding model with extended thinking for complex engineering.
DeepSeek Coder R1	DeepSeek AI	Open reasoning model optimized for mathematical logic, algorithms, and debugging.
Gemini 2.5 Pro Agent	Google DeepMind	Long context window (1M+ tokens) with multimodal reasoning and deep refactoring.
	Meta AI	

Llama 3.3 70B Code Specialist	State-of-the-art open-weights model fine-tuned for syntax precision and speed.

Chapter D2 continued: Marketplace install, updates, upstream tracking, publishing

2.5 Installing a listing, step by step

Tapping **Install** on a card starts a real installation that runs in the background. The screen does not pretend: the button state comes from a durable install manager, and every outcome is reported in a toast message. The steps:

1. **Tap Install.** If the item is already installed, the button area instead shows a green check with **Installed · Remove** and, when an update exists, an orange **Update available** label with an **Update** button.
2. **The manager works.** The app creates a package folder for the item under its private `marketplace` directory, writes a `manifest.json` (id, name, author, category, version, source, checksums, install time), and saves a durable record so the install survives app restarts.
3. **The result is announced honestly.** Possible outcomes:
   - "Installed item-name" when a package was actually downloaded and written. For MCP server items, the message adds "(MCP server registered)" because the item is also registered as a usable MCP server (with its required credentials, if any).
   - "Saved item-name as a reference, no package was downloaded" when the listing has no downloadable artifact (for example an `npx …` command or a catalog-only entry). This is reported as a reference, never as an install.
   - "Install failed: reason" when the download failed, the hash did not match, or the source was insecure.
   - "item-name is already installed" when you tap Install twice.

2.6 What happens during an install (under the hood)

The install manager handles each listing's `installUrl` according to what it is:

Install source type	How it is handled
HTTPS URL (`https://…`)	The file is really downloaded (up to 50 MB, with retries and resume support on transient failures). If the listing carries a SHA-256 checksum, a mismatch fails the install and the bytes are discarded,

	never written. The downloaded bytes are saved as `package.bin` in the item's folder.
Plain HTTP URL (`http://…`)	Refused outright. An unencrypted download could be swapped by a network attacker, so the install fails instead of fetching.
Command or other text (e.g. `npx -y …`, `template://…`, `agent://…`)	Recorded as a local catalog reference with a note like "local reference: npx -y …". Nothing is downloaded.
Blank	Recorded as a catalog entry with no downloadable artifact.

Install records are stored in the app's private preferences as JSON, with the item id, name, version, install time, source URL, package folder path, the listing's SHA-256, and the upstream commit SHA (section 2.8). The version shown on records is "unknown" because listings carry no version field; the app will not invent one.

Uninstalling ("Installed · Remove") deletes the item's package folder and its durable record. For MCP server items, the registered MCP server is also removed so nothing is left behind.

> **Safety:** downloads are size-capped at 50 MB, hash-gated when the listing provides a checksum, retried with backoff on transient server errors, and resumable after interruption. Interrupted downloads resume from a versioned partial file; partials from other versions are dropped. Insecure HTTP sources are never downloaded.

2.7 Updates: the "Update available" badge and one-click update

When the app learns that an installed item's upstream source has moved on (section 2.8), the card shows an orange **Update available** label next to an orange **Update** button. Tapping **Update**:

1. The app re-checks the item's GitHub repository for its current latest commit.
2. If reachable, it re-downloads the artifact (with the same SHA-256 verification as a fresh install) into the existing package folder, refreshes the install record, and clears the badge.
3. A toast reports the outcome: "Updated item-name to the latest version", or "Update failed: reason", or "Could not reach GitHub to update item-name; try again later".

Updates are also checked automatically: every time you open the Marketplace screen, the app checks installed items against their GitHub sources, at most once every 6 hours. Items that have no GitHub repository linked cannot be updated this way; the app tells you "Cannot update item-name: no GitHub repo linked" rather than failing silently.

2.8 Upstream tracking in plain language

For listings whose install URL points to a GitHub repository, Conzyx keeps track of the repository's latest commit, like a bookmark of "the newest change the author published". Here is how it works:

- **The bookmark is a commit SHA.** Every change committed to a GitHub repository has a unique commit hash (a long hexadecimal string). When the app installs or seeds a skill, it asks GitHub for the repository's default branch (usually `main` or `master`) and records that branch's current head commit SHA as the item's `upstreamSha`.
- **Checking is cheap and throttled.** At most once every **6 hours**, the app asks GitHub for each installed item's current head SHA. If the stored SHA and the live SHA differ, the item gets the "Update available" badge. If they match, or the check fails, nothing changes; the last-check timestamp is saved only when at least one repository answered, so a failed check retries instead of being forgotten.
- **Updating moves the bookmark.** A successful one-click update records the new head SHA, which clears the badge.
- **Unauthenticated.** The check uses GitHub's public API with the user agent "Conzyx-Marketplace"; no login or token is involved.

The same SHA tracking powers the Skills screen's per-skill updates: there the comparison is done on the actual `SKILL.md` content rather than the commit hash.

2.9 Publishing your own item

The + button opens the **Publish to Marketplace** dialog. Fill in the fields and tap **Save Item**:

Field	Rules
Item Name *	Required. At least 3 characters.
Category	Pick one of Agents, Skills, MCP Servers, Templates via the chip selector (defaults to Skills).
Author	Free text. Defaults to "Local Developer" when blank.
Description *	Required. At least 20 characters: "Describe what the item does and how to use it".
Install Command or URL *	Required and must not be blank. For MCP servers this is usually a command such as `npx -y package-name`; it may also be a URL like `https://github.com/…`.

Tags (optional)	Comma-separated. At most 10 tags; each 1 to 32 characters of letters, digits, dot, underscore, or hyphen.
SHA-256 Checksum (optional)	If the install URL points to a downloadable file, paste its 64-character hex SHA-256. Installs then refuse bytes that do not hash to it. Must be exactly 64 hex characters when provided.

The dialog shows validation errors in red as you type (the *-marked fields and the format rules all gate the button), and **Save Item** stays disabled until everything passes. A published item gets a random id, a **Custom** badge, is persisted durably on the device so it survives restarts, and can be deleted later with the trash icon on its card. The dialog states plainly: "Saved on this device. No registry is configured, so this item is not shared with other devices."

Chapter D3: Settings

3.0 Layout overview

The Settings screen is headed by a gear icon, the title **Settings**, and the subtitle "Preferences & Configuration". Everything below is a list of **accordions** (expandable sections). Tap a section to expand it; tapping it again collapses it. Each accordion's subtitle previews its current state, for example "Dark theme" or "Guest mode · Local only", so you can often see what you need without opening it. The sections, in order, are:

1. Account
2. Appearance
3. Developer tools
4. Linux runtime
5. Environment updates
6. App updates
7. Permissions & Safety
8. Contributors

Below the sections, a footer row shows the app name "Conzyx", the line "Local AI coding workspace", and the installed version, followed by a **Privacy policy** row that opens the full privacy policy (Chapter D5).

3.1 Account

The subtitle shows your display name when signed in, or **"Guest mode · Local only"** when you are not. Conzyx is local-first: without an account, all your projects, tools, and code run privately on the device.

Guest mode

When signed out, the section explains that you are using Conzyx in local guest mode and offers a single orange button: **Sign In / Create Account**, which navigates to the authentication screen.

Signed in

When signed in, the section shows a profile card with:

- Your avatar circle (first letter of your display name or email).
- Display name and email address.

- **Provider**: the sign-in method used (shown as a badge).
- **Email Verification**: a "Verified" (green) or "Unverified" (red) badge, and a **Send Verification Email** button when your email provider account is unverified.
- **Member Since**: your account creation date, when available.

Below the card are four actions:

Button	What it does
Edit Display Name	Opens a dialog to change the name shown on your account.
Change Password	Shown only for email/password accounts. Opens a dialog asking for the current password, the new password, and a confirmation.
Sign Out	Signs you out (red outlined button).
Delete Account	Opens a confirmation dialog for permanent account deletion. For email accounts it asks for the password.

Error and success messages from account actions appear as banners at the top of the section (red for errors, green for confirmations), each with a dismiss X.

3.2 Appearance

Subtitle shows the current choice: "Dark theme", "Light theme", or "Follow system". Expanding shows three large selectable cards:

Choice	Effect
Dark	Forces the dark theme.
Light	Forces the light theme.
System	Follows the Android system theme.

The selected card is highlighted with a selection dot. The choice applies immediately and is remembered.

3.3 Developer tools

Subtitle: "Core tools + N optional toolchains". This section manages the Linux toolchains that run inside the app's isolated PRoot userspace.

Runtime resources card

The top card, "Runtime resources", explains that core Linux tooling is included and optional development stacks are isolated in PRoot userspace. It lists what is always present:

Resource	Status
Ubuntu 20.04 ARM64	Included
Node.js + npm	Included
Git + OpenSSL + curl	Included
Claude Code Engine	Included

Auto-update tools & skills

A card with a **switch (default: off)**. When on, the app automatically keeps installed toolchains, tools, and skills up to date: it checks for updates at most every 4 hours (immediately when you first enable it), skips the check while a task, terminal session, install, or another update is running, and reports a combined message of what was updated and what failed. While an auto-update pass is running, an orange progress bar shows under the switch and the switch is disabled. The same switch appears on the Skills screen; both flip the same preference.

Toolchain list

Below, one row per toolchain (Web, Python, Android, C++, PHP, Browser). Each row shows the toolchain name, its status (one of **INCLUDED**, **INSTALLED**, **AVAILABLE**, **INSTALLING**, **REMOVING**), and a version badge when installed (or "Unknown version" rather than a guessed one). Tap a row to expand it:

- A short description of what the toolchain installs and the line "Target environment: PRoot Debian/Ubuntu userspace".
- **Available:** a full-width **Install Stack** button.
- **Installed: Update** and **Remove** buttons side by side.
- **Web:** shown as "Bundled in Core Rootfs"; it cannot be installed or removed separately.

- **Installing:** a percentage, an orange progress bar, and a transfer panel showing "X.X MB of Y.Y MB", the current speed, and estimated time left, plus a status message ("Downloading…" or the current step).
- **Removing:** a "Removing toolchain…" indicator.

Tapping **Remove** first opens a confirmation dialog: "Remove toolchain-name? This removes the toolchain and its runtime caches to free storage. Your projects and source files will not be deleted." Confirm with the red **Remove** button or **Cancel**.

Status messages appear in a banner under the list: green with a check for success, red with an info icon for failures ("could not…", "failed…").

3.4 Linux runtime

Subtitle: "Ubuntu 20.04 PRoot · ARM64". This section is read-only diagnostics plus maintenance buttons:

- **Architecture:** ARM64 (aarch64).
- **Environment:** Ubuntu 20.04 PRoot.
- **Active agent:** the currently selected agent's title, with "· Not installed" if it is not installed.
- **Installed agents:** one row per agent with its installed version (e.g. "v2.1.4"), or "No verified agent installation" when none is installed.

Button	What it does
Clear terminal history	Wipes the terminal's history. The button label changes to "Terminal history cleared" after you tap it.
Backup all projects to storage	Backs up all workspaces to device storage (Workspace Data Protection & Rollback).
Restore projects from backup	Restores workspaces from a previous backup.
Advanced runtime reliability	Expands a help panel (see below).

The **Advanced runtime reliability** panel explains how to keep long tasks alive when you switch apps or lock the phone:

- **Battery optimization (Unrestricted):** opens Android's battery settings for Conzyx (falling back to the app-details settings page) so the system does not kill background work. If it cannot open the page, a toast says "Could not open battery settings".

- **Configure <manufacturer> Autostart:** appears only on devices from manufacturers known for aggressive background killers, opening that manufacturer's autostart/background settings.
- **Open Developer options:** a note explains that if large builds stop unexpectedly, Android's Developer options may have a child-process restriction toggle; the button opens Developer options (or the main Settings app as a fallback).

Chapter D3 continued: Settings, Environment updates, App updates, Permissions & Safety, Contributors

3.5 Environment updates

Subtitle: "Core, Python, Android & AI agent runtimes". This section is the **Environment Update Center**: it manages the downloadable components of the Linux runtime (the core rootfs, agents, and toolchains). A note at the top clarifies: "Agents are managed only from the Agents screen. This page manages the Linux runtime, toolchains and development resources."

- **Check for updates** (outlined button, full width): refreshes the component catalog. It first tries the remote manifest from the configured update URL; if that is unreachable it falls back to the verified catalog bundled with the app. Status text reports which source was used ("Catalog up to date.", "Loaded verified environment catalog.", or "Showing verified local catalog.").
- Each component is a card showing its name, a status line ("Not installed · latest X", "Installed X · latest Y", or "Up to date · vX"), an uppercase status chip, any notes, and the line "Package verification: SHA-256" when the component can be installed or updated.
- **Install / Update** button: downloads and applies the component with live progress messages. While busy, a **Cancel** button appears beside it; cancelling stops the download.
- **Rollback** button: appears when a previous version is available to roll back to.

Components without a standalone update package show the note: "Managed by the bundled runtime. No standalone update package is published for this component."

3.6 App updates

Subtitle: the installed version and build number, e.g. "v3.1.0 (Build 37) · Online". Expanding shows a single row:

- A pill badge with the version and the build flavor: blue **v3.1.0 ONLINE** or green **v3.1.0 OFFLINE**.
- A **Check now** button that immediately checks for a newer Conzyx release.

On Google Play builds there is no Check-now button; the section instead says **"Managed by Google Play"**, because Play builds never self-update from a side-channel APK (see Chapter D4). Tapping Check now when the app is already current shows a toast like "Conzyx is up to date (online · v3.1.0)"; when the check itself fails, the toast says "Could not check for updates: reason" instead of pretending everything is current.

3.7 Permissions & Safety

Subtitle: "Granular execution boundaries & audit trails". This section is the app's safety control panel. Two scope notes are printed at the top and bottom: the toggles are enforced for **Claude Code only**; Antigravity and DeepSeek run headless with their CLI defaults (full access, no per-action prompts); and the emergency kill switch blocks new sessions for all agents.

Seven toggles, each with a title, an explanation, and a switch:

Toggle	What it does	Default
Shell execution	Allow the agent to execute terminal commands. When off, shell commands are blocked or require approval.	On
Network access	Allow the agent outbound HTTP and curl requests.	On
Auto-delete files	When off, the agent must show an explicit prompt before deleting files. Turn on to permit deletion without a prompt.	Off
Git push	When off, pushing commits to a remote requires a prompt. Turn on to allow pushes without asking.	Off
Install device APKs	Allow background APK installations without a prompt. Off by default; Android still requires its own system confirmation.	Off
External services	Allow calls to external APIs without a prompt. When off, connecting to third-party services requires confirmation.	Off
Access environment variables	Allow reading environment variables without a prompt. When off, reading them requires authorization.	Off

When to change them: keep the destructive ones (file deletion, git push, APK install, external services, environment variables) off unless you are doing repetitive trusted work and the prompts slow you down. Shell execution and network access are on by default because normal development needs them.

3.7.1 View Security Audit Log

The full-width **View Security Audit Log** button opens a bottom sheet titled **Security Audit Trail** ("Recent access evaluations, policy checks, and blocked dangerous actions") covering up to 85% of the screen. It has:

- A red **Clear** button (shown only when entries exist) that wipes the log.
- A search field: "Search action, target, reason..." with a clear X.

- Filter chips: **All**, one chip per category (SHELL_EXEC, NETWORK, FILE_DELETE, GIT_PUSH, INSTALL_APK, EXTERNAL_SERVICES, ACCESS_ENV, SECRETS), and **Blocked Only** to show only blocked or denied events.
- Each entry shows the category chip, the action, the decision in color (green ALLOWED, red BLOCKED_DANGEROUS or DENIED, orange otherwise), plus the target and the reason when present.
- Empty states: "No security events recorded yet." or "No matching security events found."

3.8 Contributors

Subtitle: "People building Conzyx". Three contributor cards, each with a circular avatar (loaded from their GitHub profile, falling back to their initials), name, role line, handle, and link buttons that open the profile in your browser (with a "Could not open link" toast if opening fails):

Name	Role	Handle	Links
Sunmughan Swamy	Founder · Technical Architect · Software Engineer	@sunmughan	GitHub, Instagram, Facebook
Valerio	Hi, I'm Valerio, but everyone calls me Massi!	@0xMassi	GitHub, LinkedIn
Tech Jarves	I break, tweak, and rebuild Android & Linux systems.	@techjarves	GitHub, Telegram, YouTube

3.9 Version footer and privacy policy row

At the bottom of the screen, a footer row shows the Conzyx gear icon, the name **Conzyx**, the line "Local AI coding workspace", and the installed version on the right. Below it, the **Privacy policy** row ("How local data and AI provider requests are handled", with a privacy icon and a forward arrow) opens the full privacy policy screen covered in Chapter D5.

Chapter D4: In-app updates

4.1 Automatic update checks

Conzyx checks for new releases of itself automatically. After the app finishes its startup sequence, it runs an update check in the background, then repeats at most **once every 24 hours** while the app is running. A failed check is not recorded as a completed check, so the next opportunity retries instead of waiting another day.

When an update is found, an **update dialog** appears automatically (the same dialog as the manual flow, section 4.5). Google Play builds are exempt: they never check any side channel and never offer a GitHub update (section 4.6).

4.2 Manual check: "Check now"

Open Settings, expand **App updates**, and tap **Check now**. The check runs immediately, regardless of the 24-hour interval:

- If an update is available, the update dialog opens.
- If the app is current, a toast confirms it, e.g. "Conzyx is up to date (online · v3.1.0)".
- If the check fails (no network, server error), the toast says "Could not check for updates: reason". A failed check is never misreported as "up to date".

On Play builds, tapping the settings entry shows the toast "App updates are managed by Google Play on this build." and no check is made.

4.3 What the updater downloads and where it looks

The updater resolves new releases from two sources, in order:

1. **The configured manifest URL** (a JSON update manifest), or your override of it (section 4.7).
2. **The project's own GitHub releases API**: `https://api.github.com/repos/sunmughan/Conzyx/releases/latest`. CI publishes a `mobile-harness-update.json` manifest asset on every release with the true version code and SHA-256; a release without that asset offers no update rather than an unverifiable one. The app deliberately does not consult any other author's manifest.

An update is offered only when the manifest's `versionCode` is higher than the installed build's, and only for the matching build variant (online or offline). The manifest names the APK artifact for that variant, its SHA-256, its size, and the release notes. Only `https://` URLs are accepted.

When a personal access token is configured (for private repository releases), it is sent only to GitHub domains, never to redirect targets such as download mirrors.

The APK downloads into the app's private `updates` folder. Downloads are **resumable**: an interrupted download continues from a versioned partial file, and partials from other versions are dropped so they cannot corrupt the new download. You can cancel a download; the partial is kept for resume.

4.4 Verification before install

A downloaded APK is verified before Android is ever asked to install it:

1. **SHA-256 check:** the file's hash must match the manifest's `sha256` (when provided); a mismatch deletes the file and fails the update.
2. **Package name:** the APK's package must be Conzyx's own package; anything else is rejected.
3. **Version code:** the APK's internal version code must equal the manifest's `versionCode`.
4. **Signing key:** the APK must be signed with the same key as the installed app. An APK signed by anyone else cannot be installed over your copy, which blocks tampered updates.

A file that fails any of these checks is deleted and the failure is reported.

4.5 The update dialog and install flow

When an update is available, a dialog titled **"Update to X.Y.Z"** appears with a download icon. It shows:

- The release notes from the manifest (or "Get the latest improvements and fixes for Conzyx.").
- The download size when known, e.g. "Download size: 84.2 MB".
- On Android 8 and above, a warning if Conzyx lacks the **"Install unknown apps"** permission: "Allow 'Install unknown apps' for Conzyx. Without this permission, Android will not install the update." The confirm button then reads **Grant permission** and opens the system page where you enable it.
- During download: a progress bar with "Downloading 12.4 MB / 84.2 MB · 14%", and a **Cancel** option.
- After verification: "Download verified. Opening Android installer…" in green.
- Any error in red text.

The confirm button reads **Download and install** (changing to "Downloading…" and "Installing…" while busy). The dialog can always be dismissed; the download and the Android installer run

detached, so dismissing only hides the dialog. You can reopen it from the update banner. The final step hands the verified APK to **Android's own system installer**, which shows its standard install confirmation; Conzyx cannot silently install an app.

4.6 Google Play builds

Builds distributed through Google Play never self-update. The updater's single decision point disables both checking and downloading on Play builds, because Play policy reserves updates to Google Play itself. In Settings the App updates section shows "Managed by Google Play" instead of a Check-now button.

4.7 Update channel settings (manifest URL and auth token)

Advanced users can point the updater at a custom update channel. The app persists two values:

Setting	What it is	Default
Update manifest URL override	A custom manifest URL or proxy endpoint used in place of the built-in one. Setting it resets the check timer and triggers an immediate forced check. Clearing it returns to the default release channel.	Empty (built-in channel)
Update auth token	An optional Bearer / personal access token for reading releases of a private GitHub repository. Sent only to GitHub domains.	Empty
Debug manifest URL	Debug builds only: an override used to test updates against a local server without publishing a release.	Empty

> **Summary:** Conzyx updates itself from its own GitHub releases, checks automatically at startup and every 24 hours, verifies every APK by hash, package name, version, and signing key, and installs only through Android's system installer. Play Store builds update through Play instead.

Chapter D5: Privacy policy summary

The full privacy policy is available in the app under Settings, at the bottom of the list: the **Privacy policy** row opens a dedicated screen that renders the complete policy text. The same text lives in the project as `PRIVACY.md`. It is effective **October 3, 2026**, and privacy questions can be raised as an issue in the Conzyx repository at `https://github.com/sunmughan/Conzyx/issues`. This chapter is a plain-language summary of what it says.

5.1 What information the app handles

Conzyx may handle information you provide or generate while using it:

- AI-provider API keys, authentication details, model selections, and connection settings.
- Prompts, AI responses, conversation history, and agent task context.
- Project files, source code, imported ZIP files, Git/GitHub repository data, documents, images, audio, and other attachments you explicitly provide.
- Terminal commands, command output, working-directory information, file changes, diffs, checkpoints, and diagnostics.
- Build and runtime information needed to run the local development environment: Android version, processor architecture, memory, storage, installed runtime/agent versions.
- Information needed to download, install, update, and verify runtime components and app updates.

5.2 Where your data lives: local-first storage

Projects, conversations, attachments, imported repositories, editor content, terminal history, checkpoints, runtime files, and diagnostics are stored in the app's **private storage on your device**, unless you explicitly use a feature that sends information elsewhere.

- **Credentials are encrypted.** AI-provider API keys and MCP server secrets are protected with Android Keystore-backed encryption before they are saved. OAuth tokens used by integrated command-line tools (GitHub, Antigravity) are kept in the app's private on-device Linux environment.
- **No advertising SDKs.** The app includes no advertising SDKs and operates no first-party user account system.
- **Crash reporting:** the app includes Firebase Crashlytics for crash reporting. Crash reports may include stack traces, device information (such as Android version and device model), and file paths from the app's private storage. Crash reporting is always on; there is no in-app opt-out.

5.3 When information leaves the device

Conzyx does **not** send your project contents to its developer as part of ordinary local use. Information can leave the device only when you explicitly use an external service, such as:

- An AI provider you configured: depending on the task, this can include prompts, conversation context, relevant source code, file paths, file contents, tool results, and attachments.
- Git/GitHub operations, repository imports, and repository metadata requests.
- Runtime, toolchain, package, and agent downloads; app update checks and release metadata.
- Web previews or other network resources you explicitly open.
- Crash and diagnostic reports sent to Firebase Crashlytics.

Those services may receive standard network information such as your IP address, request time, requested resource, and user-agent. Each third-party service's own privacy policy governs what it receives. Conzyx does not sell personal information.

5.4 Permissions the app may request

Permission / capability	Why
Internet and network state	To reach AI providers, Git/GitHub, runtime and package sources, update services, and web resources.
Notifications	To report user-started setup, builds, agent tasks, downloads, and other foreground/background work. Can be declined; some background features may be less reliable without it.
Foreground service and wake lock	To keep a user-started task active long enough to finish.
Document picker access	To import only the files and folders you explicitly select through Android's system picker.
Camera / microphone / media access	Only when a feature explicitly requires user-selected media or device input and Android grants it.
Install packages	To submit an APK built from your project to Android's system installer. Android requires your confirmation; Conzyx cannot silently install an app.

5.5 Retention and deletion

Local projects, conversations, terminal history, editor data, attachments, checkpoints, runtime files, and diagnostics remain on the device until you delete the relevant data, clear it from the app, or uninstall Conzyx. Uninstalling removes the app's private local data per Android's normal behavior. Information already transmitted to an AI provider, Git/GitHub, a package or runtime host, or another third-party service is governed by that service's retention and deletion policies; contact the provider directly for requests about data they hold.

5.6 Security, children, and policy changes

- **Security:** the app uses app-private Android storage, Android Keystore-backed credential encryption, HTTPS where external services support it, and checksum verification for supported runtime downloads. Local Linux commands and project code run with the capabilities granted to the app and runtime. No system guarantees complete security, so only run code and install dependencies you trust.
- **Children:** Conzyx is a developer tool and is not directed to children under 13; it does not knowingly collect personal information from children.
- **Changes:** the policy may be updated as Conzyx changes; material changes are reflected by updating the effective date and publishing the revised policy at the repository location above.

> **The short version:** Conzyx by Codeair Software Solutions is local-first. Your code, chats, and files stay in the app's private storage on your phone, credentials are encrypted with the Android Keystore, and nothing is sent to the developer. Data leaves the device only when you ask the app to talk to an outside service: your AI provider, GitHub, a download host, an update server, or crash reporting.

Chapter 1: How Conzyx Works Under the Hood

1.1 The big picture

Conzyx is an Android app written in Kotlin with a Jetpack Compose interface, but the coding work itself does not run on Android. It runs inside a real Linux environment that lives inside the app's private storage. When you open the terminal, run an agent task, or build a project, your commands execute inside Ubuntu 20.04 on ARM64, translated to the Android kernel by a compatibility layer. No root is required, and nothing is installed system-wide: everything the app needs lives under the app's own data directory and can be removed by uninstalling the app.

1.2 The three layers

Think of Conzyx as three stacked layers:

- **The Android app layer.** The Compose UI you tap: editor, terminal screen, chat, team orchestration, file manager, settings. It lives in the `com.codeair.mhe` package with application ID `com.codeair.mhe`.
- **The bridge layer.** Native code and services that translate between Android and the Linux guest: the PRoot launcher, the process manager, and the foreground services that keep long tasks alive.
- **The Linux guest layer.** A complete Ubuntu 20.04 ARM64 userspace with Bash, Node.js, Git, Python, build tools, and the AI coding agents (Claude Code, DeepSeek Harness, Antigravity CLI, Codex CLI). This is where your projects are compiled, tested, and edited by agents.

1.3 PRoot: Linux without root

Android and desktop Linux share the Linux kernel family, but Android blocks normal container tricks and has no root access for apps. Conzyx solves this with **PRoot**, a userspace program that runs a full Linux filesystem on top of the host without any privileges. The app ships PRoot as a native library, `libproot.so`, embedded in the APK. When the app needs to run something inside Linux, it launches PRoot, points it at the Ubuntu filesystem image stored in the app's files directory, and PRoot makes that image look like the real root filesystem (`/`) to every program inside it.

From the guest's point of view, `/usr/bin/bash` , `/usr/local/bin/node` , `/usr/local/bin/claude` and the rest are ordinary Linux paths. From Android's point of

view, they are just files under `/data/data/com.codeair.mhe/files/runtime/ubuntu/...`. Programs inside the guest can read and write files, run compilers, and open network connections, while the Android host partitions stay untouched. This rootless isolation is also a security feature: agent-run commands cannot escape the userspace container to damage the phone.

1.4 Where everything lives on the device

Location (relative to the app's private storage)	What it holds
`files/runtime/ubuntu`	The Ubuntu root filesystem: the guest's `/usr`, `/bin`, `/root`, toolchains, and agent binaries.
`files/workspaces`	Your projects. Each project is a directory here; legacy scaffold files from old alpha builds are cleaned out automatically.
`files/secure-credentials`	Login credentials, split by service: `github/` and `antigravity/`. These are bind-mounted into the guest at `/root/.config/gh` and `/root/.gemini/antigravity-cli`, so the agents can use them without copying them into project folders.
`cache/runtime-downloads`	Temporary download staging for runtime bundles and tool updates. Cleaned after successful installs.
`files/updates`	Downloaded app update APKs (plus resumable `.part` files) from the in-app updater.

> All of these directories are inside the app's private sandbox. Other apps cannot read them, and uninstalling Conzyx deletes them all. Android's own auto-backup is disabled for this app (`android:allowBackup="false"` in the manifest), so the app's own manual backup feature (see the Backup chapter) is the supported way to protect your projects.

1.5 How processes run

Two foreground services do the heavy lifting:

- **RuntimeExecutionService** runs while an agent task or terminal job is active. It starts as a foreground service with a persistent notification (for example, "Conzyx is working", "Coding agent is working in your project") so Android's process killer leaves it alone. It holds a **partial**

wake lock and a **high-performance Wi-Fi lock** while work runs, and releases both the moment the task finishes, fails, or is cancelled. Progress updates refresh the notification live, and completion or failure posts a result notification.

- **RuntimeSetupService** handles the one-time installation and later updates of the Linux runtime, reporting staged progress (downloading, unpacking, verifying) in a notification.
- **KeepAliveService** keeps the IDE session alive while the app is in the background.

Long tasks are capped by the system in one specific way: the wake lock and Wi-Fi lock are acquired with a **90-minute timeout**. If a single task runs longer than 90 minutes without finishing, the locks release and Android is free to throttle the CPU or Wi-Fi again. For most coding tasks this is plenty; for multi-hour builds, expect the app to prefer foreground use or split work into stages.

1.6 Networking inside the guest

Android apps do not share the host's DNS configuration with a PRoot guest automatically, so the installer writes `/etc/resolv.conf` inside the Ubuntu filesystem. It copies the DNS servers from the phone's currently active network, and falls back to `8.8.8.8` and `1.1.1.1` if none are advertised. That is why `npm install`, `apt`, and agent network calls work inside the guest: DNS is provisioned at install time and refreshed whenever the runtime is installed or updated.

1.7 Version markers: how the app knows what is installed

The app never guesses what is in the guest. It records proof. Small marker files inside the Ubuntu tree record exactly what was installed and verified:

Marker file (inside the Ubuntu tree)	Meaning
`.pocket-rootfs-version`	Core Ubuntu image version (currently `ubuntu-20.04.5-arm64`).
`.pocket-runtime-ready`	The core runtime finished its full install and verification.
`.pocket-core-tools-version`	Core toolchain bundle version (currently `core-bundle-2026.09.5`; older `core-bundle-2026.09.4` is accepted as a legacy version).
`.pocket-claude-version`	Installed Claude Code version.
`.pocket-dsh-version`	Installed DeepSeek Harness version.

`.pocket-agy-version`	Installed Antigravity CLI version.
`.pocket-codex-version`	Installed Codex CLI version.
`.pocket-github-cli-version`	Installed GitHub CLI version (currently `2.100.0`).
`.pocket-python-tools-version`	Installed Python stack version, including the pip version in the form `3.12 (pip 25.1)`.
`.pocket-php-tools-version`, `.pocket-cpp-tools-version`, `.pocket-browser-tools-version`, `root/.pocket-android-tools-version`	Installed versions of the PHP, C/C++, browser-automation, and Android stacks.
`.pocket-dev-stacks.json`	Which optional stacks the user selected during onboarding.

The "installed" check reads these markers and verifies the actual binaries exist and are executable (`usr/bin/bash`, `usr/bin/git`, `usr/local/bin/node`, the PRoot launcher). The Environment screen in Settings compares these recorded versions against the latest published versions and offers updates. This is why the app can tell the difference between "not installed", "installed but outdated", and "the update check itself failed": each state is computed from markers and live version probes, not from assumptions.

Chapter 2: The Linux Runtime: What Gets Installed

2.1 First launch: the setup sequence

The first time you open Conzyx after installing it, the app runs a guided setup through `RuntimeSetupService` with a progress screen. At a user level, the sequence is:

1. **Architecture check.** The app verifies the device reports an ARM64 kernel and the `arm64-v8a` ABI. If not, setup stops with the message "Unsupported architecture: Conzyx requires an ARM64 device or ARM64 emulator". There is no x86 or 32-bit ARM build of the runtime.
2. **PRoot launcher check.** The embedded `libproot.so` must exist and be executable. If it is missing, setup stops with "The embedded PRoot launcher is unavailable".
3. **Core runtime.** The Ubuntu 20.04 base image (the "Core" bundle) is obtained, SHA-256 verified, and unpacked into `files/runtime/ubuntu`. It is unpacked into a staging directory first and then swapped into place, so an interrupted install cannot leave you with a half-written runtime: if activation fails, the previous copy is restored.
4. **Compatibility links and DNS.** Filesystem compatibility links are created and the DNS resolver (`etc/resolv.conf`) is written from the phone's current network.
5. **Base tools.** Node.js is installed if needed, then Git and CA certificates are installed (via `apt` in the online edition), verified with `git --version`, and recorded with the core-tools marker.
6. **Optional stacks.** Any extra toolchains you picked during onboarding (Python, Android, C/C++, PHP, Browser) are installed now. In the offline edition, only the stacks that are bundled can be installed here (see 2.4).
7. **Coding agent.** Your selected agent (Claude Code, DeepSeek Harness, Antigravity CLI, or Codex CLI) is installed and verified. Additional agents can be installed later on demand from Settings; each agent is a separate overlay and nothing downloads until you select it. The "Custom" agent is bring-your-own-API and installs nothing.

Keep the app in the foreground and stay on Wi-Fi during this step in the online edition. The offline edition does this from its bundled archives and needs no network for the core setup.

2.2 What is downloaded (online edition)

In the online edition, every bundle is downloaded from the project's own GitHub releases at `https://github.com/sunmughan/Conzyx/releases/download/v<version>/` (never from the upstream author's repository) and **SHA-256 verified before extraction**. The pinned bundles built into the current release are:

Bundle	File name	Compressed size	Contents
Core	`conzyx-core-arm64-2026.09.5.tar.zst`	about 68.9 MB	Ubuntu 20.04 ARM64 base, Bash, Node.js tooling
Claude Code	`conzyx-claude-arm64-2026.09.1.tar.zst`	about 71.8 MB	Claude Code CLI 2.1.263 (installed to `/usr/local/bin/claude`)
Python	`conzyx-python-arm64-2026.09.2.tar.zst`	about 52.9 MB	Python 3.12, pip, venv, build tools
Android	`conzyx-android-arm64-2026.09.1.tar.zst`	about 543.3 MB	JDK 17, Android SDK 36, Build Tools 35, Gradle 8.14.3, offline Maven cache
DeepSeek Harness	`conzyx-dsh-arm64-2026.09.1.tar.zst`	about 26.5 MB	DeepSeek Harness (pinned release 0.1.2-rc.1, installed via npm inside the guest)
Antigravity CLI	`conzyx-agy-arm64-2026.09.1.tar.zst`	about 39.9 MB	Antigravity CLI (installed to `/root/.local/bin/agy`)

The compressed sizes above are the pinned fallback values compiled into the app. At runtime the app can resolve newer verified bundles from the latest published runtime release and prefers those, so the actual download on your device may be a newer version than the table shows. Either way, every byte is hash-verified before it is unpacked.

2.3 Where agents and tools come from

Agents and auxiliary tools are fetched from their official sources, with verification at each step:

- **Claude Code.** Downloaded from the official Claude release CDN. The expected SHA-256 checksum is read from the official release manifest (`manifest.json`) for the `linux-arm64` platform, the binary is hash-verified after download, swapped into `/usr/local/bin/claude` with a backup of the previous binary kept until verification passes, and confirmed with `claude --version`. If verification fails, the previous binary is restored automatically.
- **DeepSeek Harness.** Installed with `npm install` from the official `@deepseek-ai/dsh` package inside the guest, moved into place only after the install succeeds (the old copy is preserved until the swap completes), then verified with `dsh --profile headless --help`.

- **Antigravity CLI.** Downloaded from the URL in the official Antigravity updater manifest, verified with the manifest's SHA-512 checksum, installed to `/root/.local/bin/agy` with the same backup-and-restore safety as Claude Code.
- **Codex CLI.** Installed with `npm install -g @openai/codex`. Since npm replaces the binary in place, the updater records the previously installed version first; if the new install or its verification fails, it best-effort reinstalls the previous version.
- **GitHub CLI.** The official `gh` 2.100.0 ARM64 binary is downloaded on demand from the official GitHub CLI releases (even in the offline edition, because GitHub is inherently an online service), SHA-256 verified, installed to `/root/.local/bin/gh`, and confirmed with `gh --version`.

2.4 Optional development stacks

Node.js, npm, Git, and your selected coding agent are always installed because the agent runtime depends on them. Heavier toolchains are opt-in during onboarding:

Stack	What it installs
Web (JavaScript / TypeScript)	Node.js and npm (already included in Core)
Python	python3, pip, venv, build tools; pip, setuptools, and wheel are upgraded from the official PyPI after the bundle install
Android (Java / Kotlin)	JDK 17, ARM64 Android SDK 36, Build Tools 35, Gradle 8.14.3, and an offline Maven cache
C / C++	gcc, g++, make, cmake, gdb
PHP	PHP 8.4 (via the online Ubuntu package path), common extensions, and the latest stable Composer from getcomposer.org
Browser Automation	chromium-browser, puppeteer-core, and the pocket-browser CLI

> Offline edition limit: the offline APK only embeds the Core, Python, and Android bundles (plus the agent bundles). If you select C/C++, PHP, or Browser Automation during onboarding in the offline edition, setup tells you those stacks were skipped because they are not included in the offline build, and directs you to the online edition to install them.

2.5 Agent updates and rollbacks

Settings can check each installed agent against its own authoritative release source: the npm registry for Claude Code, DeepSeek Harness, and Codex CLI, and the official updater manifest for Antigravity CLI. A check that fails because of network problems is reported as "could not check", never as "up to date". When an update is available, the update runs with the same verify-then-commit pattern as the first install, and a rollback backup is taken first: the previous agent binary (or directory) is zipped with a `.version` record inside, so a failed update restores the working version instead of leaving a broken agent behind.

The Environment screen also tracks the Core, Python, and Android stacks against their latest verified bundles, plus live upstream versions of pip, Composer, and Puppeteer, so you can update toolchains from one place.

Chapter 3: Build Variants and Editions

3.1 One app, two runtime-delivery flavors

Conzyx is built from a single codebase with two product flavors on the `runtimeDelivery` dimension. They share the same application ID (`com.codeair.mhe`), the same version number, the same feature set, and the same signing key. The only difference is how the Linux runtime reaches your device.

	Online	Offline
APK file	`app-online-release.apk`	`app-offline-release.apk`
APK size (approx)	about 150 MB	about 1.2 GB
What is inside	The app plus the shared Antigravity agent bundle. Runtime archives are downloaded on demand from verified GitHub releases.	The app plus embedded runtime archives: Core, Claude Code, Python, Android SDK, and DeepSeek Harness bundles, plus the shared Antigravity bundle.
First-run network	Required: downloads the Core bundle (about 69 MB compressed) and whatever else you select, all SHA-256 verified.	Not required for core setup: the embedded archives are unpacked locally. (GitHub CLI and agent updates still need network, since GitHub is an online service.)
Best for	Devices with limited storage, or anyone who wants a small download and installs only the stacks they need.	Metered or unreliable connections, offline-first use, or installing once and working without further downloads.
Stack limits	All six optional stacks available (Web, Python, Android, C/C++, PHP, Browser).	C/C++, PHP, and Browser stacks are not embedded; they are skipped at setup with a notice.

> Because both editions share the application ID and signing key, you can switch editions later by installing the other APK over the existing one. Your projects, settings, and the installed runtime are preserved.

3.2 The AAB (Play Store bundle)

CI also builds `app-online-release.aab`, the Android App Bundle for Google Play submission. It is built from the `online` flavor with `-PplayBuild=true`, which switches the target SDK from 28 to 36 and runs the Play readiness check (valid HTTPS privacy policy URL plus the upload signing credentials). The AAB is signed and upload-ready. Important behavioral difference: **Play builds never self-update from GitHub**. The in-app updater is completely disabled on Play builds because Play policy reserves updates to Google Play itself; updates arrive only through the Play Store.

3.3 Version scheme

- **versionName** is the human version, for example `3.1.0` (defined as `appVersionName` in `app/build.gradle.kts`; release tags are `v3.1.0` style).
- **versionCode** is the integer Android uses to decide which APK is newer, for example `37`. CI verifies on every release that the new APK's versionCode is strictly greater than the previous release's.
- **targetSdk:** 28 for the direct-download APKs (this keeps the proven PRoot execution path; the comment in the build file notes the Play build targets current Android while its runtime path is validated), 36 for Play builds. **minSdk** is 28 (Android 9). Only **arm64-v8a** is packaged.

3.4 How CI builds and publishes a release

Releases are built by the `Android QA` GitHub Actions workflow (`.github/workflows/android.yml`) on Codeair Software Solutions infrastructure. The pipeline, in order:

1. **Runtime bundle resolution.** Before compiling, CI fetches the newest verified runtime bundles (Core, Claude, Python, Android, DeepSeek, Antigravity) from the published `runtime-*` releases, verifies each file's SHA-256 against its published digest, and records them in `dist/runtime-bundles/manifest.json`. These become the embedded assets for the offline APK and the fallback bundle metadata for the online APK.
2. **Tests and debug builds.** Unit tests run for both flavors, then both debug APKs are assembled.
3. **Release builds.** `assembleOnlineRelease`, `assembleOfflineRelease`, and `bundleOnlineRelease -PplayBuild=true` produce the two release APKs and the Play AAB.
4. **Signing.** Release builds are signed with the permanent Codeair Software Solutions release key from CI secrets. The workflow verifies the keystore's SHA-256 certificate fingerprint before building, so a wrong key fails the build instead of shipping a mismatched signature.

5. **Signature and version verification.** `apksigner` verifies every APK's signature and alignment. A compatibility step downloads the previous release's APK and checks that the new APK carries the same signing certificate, a strictly increasing versionCode, the expected versionName, and the package name `com.codeair.mhe`. This is what makes in-place updates over the old app always work.
6. **Release publication.** On the `main`, `v2`, or `v3` branch, the workflow creates or updates the GitHub release for the version tag and uploads: `app-online-release.apk`, `app-offline-release.apk`, `app-online-release.aab`, the `mobile-harness-update.json` manifest, and any new `conzyx-*.tar.zst` runtime bundles (bundles already on the release are skipped, so routine builds do not re-upload hundreds of megabytes).

Debug APKs are CI-only artifacts and are not published as release downloads.

Chapter 4: In-App Updates

4.1 How the update check works

The app checks for updates against a manifest published on the project's own GitHub releases. The flow:

1. The app fetches `mobile-harness-update.json` from `https://github.com/sunmughan/Conzyx/releases/latest/download/mobile-harness-update.json` (8-second connect timeout, 10-second read timeout). If that fails, it falls back to the GitHub Releases API for `sunmughan/Conzyx` and looks for the `mobile-harness-update.json` asset there.
2. It reads the manifest's `versionCode`. If it is not higher than the installed app's versionCode, there is no update.
3. If it is higher, the app picks the artifact matching its own variant (`online` or `offline`), each with a download URL, SHA-256 hash, and size in bytes. Only an `https://` URL is accepted.
4. A release that has no readable `mobile-harness-update.json` offers **no update at all**, rather than an unverifiable one. A failed check (network error, bad HTTP status, invalid JSON) is reported as a check failure, never silently as "up to date".

> Play Store builds skip this entire flow: self-update is disabled and no GitHub update is ever offered on a Play build. Updates come from Google Play.

4.2 The update manifest

The `mobile-harness-update.json` published by CI on every release looks like this (abridged):

```
schemaVersion: 2
versionCode: 37
versionName: "3.1.0"
channel: "stable"
notes: "Conzyx v3.1.0 … verified latest runtime assets …"
artifacts:
  online: { url, sha256, sizeBytes } → app-online-release.apk
  offline: { url, sha256, sizeBytes } → app-offline-release.apk
components:
  core, python, android, claude, deepseek, antigravity … (versions + hashes of the verified runtime bundles)
  php: "online-only" (PHP 8.4 via the Ubuntu package path)
```

Besides driving app updates, the manifest's `components` section is the source the Environment screen uses to know the latest verified versions of every runtime bundle.

4.3 Downloading and installing an update

1. **Download.** The APK downloads to the app's private `files/updates` directory with progress reporting. Downloads follow up to 5 redirects, and **interrupted downloads resume**: the app keeps a versioned `.part` file and sends an HTTP `Range` request to continue where it stopped. Partial files from other versions are discarded so they can never corrupt the new download.
2. **Verification (three gates).** After downloading, the app verifies: (1) the file's SHA-256 matches the manifest hash; (2) the APK's package name is `com.codeair.mhe` and its versionCode equals the manifest's versionCode; (3) the APK's signing certificate matches the installed app's signing key. A failed verification deletes the partial file so the next attempt restarts clean instead of resuming corrupt bytes forever.
3. **Install.** The verified APK is opened through Android's package installer via a FileProvider URI (this is what the `REQUEST_INSTALL_PACKAGES` permission is for). You confirm the install in the system dialog, just like the first install. Your projects and settings carry over because the signature matches.

4.4 Manual update check

If you do not want to wait for the automatic check, open Settings and run the update check manually. It reports one of three honest outcomes: up to date, update available (with release notes from the manifest), or check failed (with the reason, for example a network error), so you can retry. You can also always download the latest APKs directly from the GitHub releases page and install over the existing app.

Chapter 5: Android Permissions, Explained

Conzyx declares only the permissions its code actually uses. Each one below is listed exactly as it appears in `AndroidManifest.xml`, with a plain-language reason grounded in what the app does with it.

Permission	Why the app needs it
`INTERNET`	Downloading runtime bundles and agent updates, checking for app updates, GitHub workflows, AI provider calls, push notification registration, and everything the Linux guest fetches (npm, apt, pip).
`ACCESS_NETWORK_STATE`	Checking whether the device is online before starting downloads or update checks, so the app can report "check failed" (network) instead of hanging.
`POST_NOTIFICATIONS`	Showing the persistent "working" notification while agent tasks and runtime installs run, plus task-completed / task-failed result notifications and push notifications via Firebase Cloud Messaging.
`FOREGROUND_SERVICE`	Running the execution, setup, and keep-alive services as foreground services so Android does not kill long coding tasks, runtime installs, or IDE sessions in the background.
`FOREGROUND_SERVICE_SPECIAL_USE`	The special-use subtype required on newer Android for these foreground services. The manifest declares the exact subtypes: "User-started local development task" (execution), "User-started installation of the local coding environment" (setup), and "Keeps the IDE session alive when the app is in the background" (keep-alive).
`WAKE_LOCK`	Holding a partial wake lock while a coding task runs so the CPU does not sleep mid-task. The lock is capped at 90 minutes per task and released immediately when the task ends.
`REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`	Asking the system to exempt the app from aggressive battery optimization (Doze), so

	background agent runs and long builds are not throttled. You can grant or deny this in system settings.
`REQUEST_INSTALL_PACKAGES`	Installing APKs: the in-app updater's verified update APK, and apps you build with the Android App Factory that you choose to install on the phone.
`RECORD_AUDIO`	Voice input: the microphone button in chat dictation. The microphone hardware feature is marked not required, so the app also installs on devices without a mic.

Related manifest declarations:

- **Speech recognition query.** The app queries for `android.speech.RecognitionService` so voice dictation can find the system's speech recognizer. No audio is recorded until you tap the mic.
- **Touchscreen not required.** `android.hardware.touchscreen` is declared with `required="false"`, so the app can install on devices without a touchscreen (for example, Android TV-style or DeX-style setups).
- **Cleartext traffic disabled.** `android:usesCleartextTraffic="false"` plus a network security config: all network traffic must be HTTPS.
- **No auto-backup.** `android:allowBackup="false"`: the app opts out of Android's automatic cloud backup. Project protection is handled by the manual backup feature described in the next chapter.
- **Firebase messaging service.** `MheMessagingService` receives Firebase Cloud Messaging push notifications and registers the device token at startup (available in both online and offline flavors).

Chapter 6: System Requirements

These are the requirements stated by the app's own configuration and documentation, not guesses:

Requirement	Minimum	Recommended
Android version	9.0 (API 28)	12 or newer
CPU architecture	ARM64 (aarch64) with the `arm64-v8a` ABI	ARM64
RAM	3 GB	4 GB or more (large builds, multi-agent runs)
Free storage	2 GB	4 GB (projects, build caches)
Root	Not required. The app runs 100 percent in rootless userspace.	

Notes:

- The runtime is compiled only for **ARM64**. On a 32-bit ARM or x86 device (or a non-ARM64 emulator), setup refuses to start with the message "Unsupported architecture: Conzyx requires an ARM64 device or ARM64 emulator". CI's emulator verification uses an ARM64-capable configuration for this reason.
- Only the `arm64-v8a` native library is packaged (`ndk.abiFilters += "arm64-v8a"`), which is also why the APKs stay as small as they are.
- The online edition needs a working internet connection on first launch to download the runtime. The offline edition does not need network for core setup, but agent updates, toolchain updates, GitHub features, and AI provider calls always need connectivity.
- Installing an update APK requires the system "Install unknown apps" permission for your browser or file manager on first sideloaded install, and the update must be signed with the same release key, which CI guarantees.

Chapter 7: Backup and Restore

7.1 What gets backed up

The backup feature mirrors **all of your project workspaces** (the contents of the app's internal `workspaces` directory) into a backup folder. For each project, every file is copied with its directory structure preserved. A fixed exclusion list skips regenerated artifacts so backups stay small and restores stay clean:

- `.git/`, `.idea/`, `.gradle/` (IDE and build metadata)
- `build/`, `target/` (compiled output)
- `node_modules/` (reinstallable dependencies)
- `.agents/` (agent working state)

Everything else in your projects, your source files, configs, assets, and documents, is copied.

7.2 Where backups go

Backups are written to the app's external files directory under a `backups` folder, inside a `workspaces_mirror` subdirectory (falling back to internal storage if external storage is unavailable). This is app-private external storage: it survives app updates, and you can reach it from a file manager or a computer over USB, but it is deleted if you uninstall the app. A legacy path from the earlier package name (`/storage/emulated/0/Android/data/com.codeair.mhe/files/backups`) is checked automatically during restore as a migration fallback.

> Because backups live in the app's own external directory, they are removed on uninstall. If you are about to uninstall or factory-reset, copy the `backups/workspaces_mirror` folder somewhere safe first.

7.3 Restore behavior

Restore copies projects from `workspaces_mirror` back into the internal `workspaces` directory and reloads the project list. It is deliberately non-destructive: a file is only copied back if it does not already exist at the destination, so restoring never overwrites newer work you did after the backup. The app reports how many projects were backed up or restored, or tells you plainly when there was nothing to back up or no backup was found.

Chapter 8: Troubleshooting

Every issue below is grounded in the app's actual behavior and the exact messages its code produces. Fixes describe only what the code supports.

8.1 Setup and runtime install problems

"Unsupported architecture: Conzyx requires an ARM64 device or ARM64 emulator"

The device does not report an ARM64 kernel with the `arm64-v8a` ABI. This happens on 32-bit ARM phones and on x86/x86_64 emulators. There is no 32-bit or x86 runtime build. Fix: install on an ARM64 phone or an ARM64 emulator image.

"The embedded PRoot launcher is unavailable"

The native `libproot.so` inside the APK is missing or not executable on this device. Fix: reinstall the APK from the official release (do not use repackaged copies). If it persists on a stock device, report it with the device model and Android version.

"Core runtime setup is incomplete. Reopen Conzyx to repair it."

The installed-runtime check failed: one of the required markers or binaries (`usr/bin/bash` , `usr/bin/git` , `usr/local/bin/node` , the rootfs version marker, or the ready marker) is missing or mismatched. Fix: reopen the app and let setup run its repair pass. The installer stages every change and swaps it into place with a backup of the previous copy, so a killed or failed install restores the old runtime instead of leaving a broken one.

Setup stalls or fails during download (online edition)

Possible causes, in order: no network, a captive portal, or the download being interrupted. The installer verifies every bundle's SHA-256 before unpacking, so a corrupt partial download cannot poison the runtime; failed downloads are retried and the progress screen shows bytes downloaded versus total. Fix: stay on stable Wi-Fi with the app in the foreground, then reopen the app to resume. Downloads that support it resume from the byte offset where they stopped.

"Skipped ...: not included in the offline build. Use the online build to install these stacks."

You selected C/C++, PHP, or Browser Automation during onboarding in the offline edition. Those bundles are not embedded in the offline APK, so setup skips them and tells you so. This is expected, not an error. Fix: install the online edition APK over the current one (same app ID and signing key, data preserved) and install the stack from there.

8.2 Agent install and update problems

An agent shows "could not check" for updates

The version probe for that agent's official source failed (network down, registry unreachable). The app deliberately reports this as a failed check instead of pretending everything is up to date. Fix: check your connection and retry the check. Installed agents keep working on their current version meanwhile.

An agent update fails partway

Every agent updater follows the same safety pattern: download, hash-verify, stage the new binary, back up the live binary, swap, verify with `--version`, and only then delete the backup. If the download hash does not match, the file is discarded. If the post-swap verification fails, the previous binary is restored automatically. For Codex CLI (which npm replaces in place), a failed update triggers a reinstall of the previously recorded version. Fix: retry the update; if it keeps failing, the previous working version should still be in place, and the rollback backup can be re-applied from the Environment screen.

"DeepSeek Harness update failed; the installed version was preserved"

The npm-based update inside the guest did not complete, so the swap never happened and your existing install is untouched. Fix: retry when the network is stable.

8.3 App update problems

Update check says it failed

The manifest fetch failed (HTTP error, timeout, or invalid JSON) on both the primary manifest URL and the GitHub Releases API fallback. The app reports the failure reason instead of saying "up to date". Fix: verify connectivity and retry, or download the APK manually from the GitHub releases page.

No update is offered even though a newer release exists

Two possible reasons, both by design. First, a release without a readable `mobile-harness-update.json` asset offers no update rather than an unverifiable one, because only the manifest carries the true versionCode and SHA-256. Second, Play Store builds never offer GitHub updates at all; updates come from Google Play.

"Downloaded APK failed its SHA-256 verification"

The downloaded bytes do not match the manifest hash. The partial file is deleted automatically so the next attempt restarts clean instead of resuming corrupt bytes. Fix: retry the download on a stable connection.

"Update package name does not match Conzyx" / "Update version does not match its manifest" / "Update is not signed with the installed app's signing key"

These are the install-time verification gates. They trigger if the APK is for a different app, if its versionCode differs from what the manifest advertised, or if it was signed with a different key (for example, a debug build or a repackaged APK). Fix: only install update APKs from the official `sunmughan/Conzyx` releases. Official releases are signed with the permanent Codeair Software Solutions release key, and CI checks every build's certificate against the previous release, so legitimate in-place updates always pass.

Download keeps restarting from zero

Resumable downloads depend on the server honoring the `Range` request. If a server ignores it, the download restarts from the beginning; partial files from a different version are always discarded. Fix: use stable Wi-Fi for large (offline-edition) updates.

8.4 Background execution problems

Long tasks stop when the screen is off

Agent runs are kept alive by a foreground service with a wake lock and a high-performance Wi-Fi lock, but both locks are capped at 90 minutes per task, and the system can still throttle the app if battery optimization is applied aggressively. Fix: keep the app in the foreground for very long runs, split work into stages, and consider exempting Conzyx from battery optimization in system settings (the app requests this via `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`). The completion/failure notification tells you the final state even if the screen was off.

Notifications do not appear

Progress and result notifications require the `POST_NOTIFICATIONS` permission. On Android 13 and newer this is a runtime permission: if you denied it, grant it in system Settings under Apps, Conzyx, Notifications.

Install prompt never appears for a downloaded update

The installer intent needs the `REQUEST_INSTALL_PACKAGES` permission and the system's "Install unknown apps" grant for the installing app. Fix: allow install-unknown-apps for your browser or file manager in system settings, then tap the downloaded APK (or re-run the in-app update).

8.5 Storage problems

"No space" during runtime install or stack install

The Android SDK bundle alone is about 543 MB compressed and much larger unpacked; the full offline payload plus build caches needs gigabytes. The documented minimum is 2 GB free, 4 GB recommended. Fix: free storage, install only the stacks you need (online edition), and remember backups exclude `build/`, `node_modules/`, and other regenerable directories, but your live workspaces still hold them.

8.6 When all else fails

- **Repair by reopening.** The first-run installer doubles as a repair tool: reopening the app re-runs the install checks and fixes missing pieces without wiping your projects.
- **Back up before drastic steps.** Use the backup feature to mirror your workspaces to external storage before reinstalling.
- **Clean reinstall (last resort).** Uninstalling removes the app, the entire Ubuntu runtime, credentials, and projects. Back up first. Reinstall the same edition (or switch editions) from the official releases, and restore your workspaces afterward. Restore never overwrites files that already exist, so it is safe to run on a fresh install.

Chapter 9: Frequently Asked Questions

Does Conzyx need root?

No. Everything runs in a rootless PRoot userspace inside the app's private storage. No root, no custom ROM, no system modifications.

Which edition should I install, online or offline?

Install **online** (`app-online-release.apk` , about 150 MB) if you have limited storage or a good connection; it downloads only the runtime pieces you actually use, all SHA-256 verified. Install **offline** (`app-offline-release.apk` , about 1.2 GB) if your connection is metered or unreliable; the Core, Python, Android, Claude Code, and DeepSeek Harness bundles ship inside the APK. You can switch later by installing the other APK over the current one; your data is preserved.

What Android versions and devices are supported?

Android 9 (API 28) and newer, ARM64 only. 32-bit ARM and x86 devices or emulators are rejected at setup with an explicit message.

How do in-app updates work?

The app fetches `mobile-harness-update.json` from the project's GitHub releases, compares its versionCode with the installed one, and, if newer, downloads the APK for your variant (online or offline), verifies its SHA-256, package name, versionCode, and signing certificate, then hands it to the system installer. Downloads resume after interruptions. Play Store builds never use this path; they update through Google Play.

Can I build the app myself?

Yes. The repository builds with `./gradlew assembleOnlineRelease` / `assembleOfflineRelease` (JDK 17, Android SDK platform 36, build-tools 35.0.0, NDK r26b). Release signing needs the private Codeair Software Solutions key, which is not in the repo; without it you get a debug-signed build. For local offline builds, place the `conzyx-*-arm64-*.tar.zst` bundles plus a `manifest.json` in `dist/runtime-bundles/` and the build picks them up automatically.

Where do my API keys and logins live?

In the app's private `secure-credentials` directory (split into `github/` and `antigravity/`), bind-mounted into the Linux guest at `/root/.config/gh` and `/root/.gemini/antigravity-cli` . They are never copied into project folders, and Android auto-backup is disabled for the app, so they do not leave the device through the OS backup channel.

What exactly does the backup feature save, and where?

It mirrors every project workspace (minus regenerable folders like `node_modules/`, `build/`, `.git/`, `.gradle/`, `.idea/`, `target/`, and `.agents/`) into `backups/workspaces_mirror` in the app's external files directory. Restore copies projects back without overwriting files that already exist.

Why does the app ask to ignore battery optimizations?

So Android's Doze mode does not throttle the CPU or Wi-Fi in the middle of a long agent run or build. The app also holds a wake lock and a Wi-Fi lock during tasks (capped at 90 minutes each), released the moment the task ends.

Why does the app need microphone access?

Only for the voice-dictation button in chat. Nothing is recorded until you tap it, and devices without a microphone can still install and use the app.

Is my code sent anywhere?

Code parsing, builds, git operations, and test runs happen on-device inside the Linux runtime. Network use is limited to what you ask for: AI provider calls with your own keys, GitHub operations you trigger, runtime and app updates from the project's releases, and push notification registration. All traffic is HTTPS only; cleartext traffic is disabled in the manifest.

Who makes Conzyx?

Conzyx is developed by **Codeair Software Solutions** (codeair.tech). Release builds are signed with the company's permanent release key, whose certificate fingerprint is verified by CI on every build, and the project's privacy policy ships in the repository as `PRIVACY.md`.

ABOUT

Built by Codeair Software Solutions

Conzyx is developed and maintained by **Codeair Software Solutions**, an AI-first software company building websites, mobile apps, AI systems, and developer tools.

Contact	Details
Website	www.codeair.tech
Phone / WhatsApp	+91 9584215603
Email	biz@codeair.tech

This manual was generated from the Conzyx source code (branch v3, version 3.1.0) in October 2026. Every feature described here reflects the app as implemented. For the latest releases, visit the GitHub repository.

NEW IN CONZYX 3.2.0

Synced Agent Browser

The Preview Browser chapter described a browser with two jobs: previewing your project and giving **you** a full Chromium tab with DevTools, a DOM inspector, and an AI visual debugger. The Synced Agent Browser adds the third job, and it is the reason this chapter exists: the **agent** can now drive the visible browser tab **live**, while you watch.

Say `fill this form` in chat. The agent attaches to the browser tab open on your screen and every navigate, click, and keystroke happens there, in real time, in front of your eyes. There is no hidden headless session doing the work somewhere else. What the agent does is what you see.

This is the Conzyx answer to the hosted-assistant problem described in the opening chapters: cloud agents work inside company sandboxes under company limits, and you cannot see what they are doing. Here the runtime is your phone, the browser tab is your tab, and the driving is fully visible. It is also the practical answer for everyone who does not have a personal AI assistant on their phone: open Conzyx, command an agent, and watch it execute in the browser tab.

> **How this chapter relates to the rest of the manual.** The mechanics of the browser tab itself (address bar, toolbar, tabs, viewport presets, DevTools) are documented in **Preview Browser**. The agent side (providers, sessions, skills) is documented in **Section B**. This chapter documents only the sync layer between them: discovery, attach, the driving banner, the Settings toggle, the updated skill, and the safety model.

How it works: the sync pipeline

Six stages connect a chat command to pixels moving in your browser tab. Every stage is implemented in the app; nothing here is a mock or a stub.

1 WebView remote debugging is switched on

An Android WebView does not expose its DevTools protocol unless the app asks for it. Conzyx calls `WebView.setWebContentsDebuggingEnabled(true)` in two places: once at app start (in `MobileHarnessApplication`) and once when the browser tab creates its WebView.

Both are gated by the Settings toggle **Let agents drive the browser tab** (default ON, documented below). Turn the toggle off and debugging is never enabled.

2 The app discovers the tab over the DevTools socket

A debuggable WebView publishes a DevTools server on the abstract Unix socket `webview_devtools_remote_<pid>`. The new `com.codeair.mhe.browser.AgentBrowserDrive` module queries its `/json` endpoint and parses the target list. Target selection is deliberate:

- only targets of type `page` are considered;
- `devtools://` pages are skipped, so the agent never grabs a DevTools window;
- if you have a page open, the selector prefers the tab whose URL or host matches, then the most recently loaded non-blank page.

The result is written to `<rootfs>/tmp/pocket-browser-webview.json` with the tab URL, tab title, socket name, and WebSocket debugger URL. The file is considered fresh for 60 seconds; the browser tab refreshes it every 2.5 seconds while it is open.

3 A bridge makes the tab reachable from the Linux container

The agent's tooling (Puppeteer) runs inside the Linux runtime and cannot dial an Android Unix socket directly. The WebSocket debugger URL is therefore rewritten to TCP port **9223**, and a small bridge forwards that port to the DevTools socket. From the agent's point of view, the visible tab is just another Puppeteer endpoint.

4 `pocket-browser attach` connects the agent

The agent runs `pocket-browser attach` (optionally `attach --cdp <wsUrl>` to pin a specific target). The CLI reads the discovery file, validates its freshness, connects Puppeteer with no viewport override (so the page keeps your exact layout), and writes the attach state to `<rootfs>/tmp/pocket-browser-attach.json`, valid for 15 minutes. If no tab is found, or the discovery is stale, attach fails with a plain message: open a page in the app browser tab first, then try again.

5 Commands drive the visible tab

Once attached, the familiar commands all drive the tab you are looking at:

```
$ pocket-browser open "https://example.com"
$ pocket-browser click "#submit"
$ pocket-browser type "#email" "you@example.com"
```

```
$ pocket-browser press Enter
$ pocket-browser screenshot /tmp/form-done.png
$ pocket-browser get-text "main"
$ pocket-browser eval "document.title"
$ pocket-browser status
```

While attached, tab management is disabled (`tabs` , `new-tab` , `switch-tab` , `close-tab` refuse with `Tab management is disabled while attached to the user browser tab.`). The agent cannot open, close, or switch **your** tabs. Every executed action is appended to the agent session transcript automatically, so the chat history shows exactly what the agent did and when.

6 Detach returns the tab to you

The attach ends through any of five paths: `pocket-browser detach` , `pocket-browser stop` (which also detaches), turning the Settings toggle off, your Take over (below), or the 15-minute attach expiry, after which the state file is deleted and the next command must attach again.

The driving banner and take-over

There is no silent driving in Conzyx. While an agent is attached, the browser tab shows a persistent orange banner at the top of the page:

> **Agent is controlling this tab**
> The banner shows the driven tab's title or URL underneath, plus a **Take over** button.
> Subtitle: `Watching live. Tap anywhere or Take over to reclaim control.`

- **Take over button.** Tapping it clears the attach state immediately, hides the banner, and shows the toast `You took over the browser tab` . The agent's next browser command will fail cleanly and it must ask before attaching again.
- **Tap anywhere.** A transparent gesture layer covers the page while the agent drives. Your first tap anywhere on the page takes over instantly, no need to find the button.
- **Live state.** The tab polls the attach state every 2.5 seconds, so the banner appears within moments of the agent attaching and disappears the moment control returns to you.

Settings toggle

Settings → Permissions & Safety → Let agents drive the browser tab (default **ON**).

The row reads: `Agents can control the visible browser tab with pocket-browser attach. You watch live and can take over anytime.` Turning it off disables WebView remote debugging and clears any active attach state, so no agent can drive the tab until you turn it back on. It is stored in the app preferences as `agent_browser_drive_enabled`.

The browser-automation skill, version 2

The built-in `browser-automation` skill that teaches agents the `pocket-browser` CLI was rewritten for this feature (skill version 2; the app auto-upgrades the installed copy via a `.skill-version` marker, so existing installs pick it up without reinstalling). The new skill text leads with synced mode:

- **Prefer synced mode** whenever you ask the agent to act on a page you can see. Headless mode (`pocket-browser start` on port 9222) remains for background work you do not need to watch.
- **Attach only on explicit ask.** The agent may run `pocket-browser attach` only when you explicitly asked in chat for browser work. Never silently, never speculatively, never on its own initiative.
- **Announce it.** When the agent attaches, it tells you in chat that it is now driving your visible tab and that you can take over at any moment.
- **Respect the tab.** The agent cannot open, close, or switch your tabs, and every action is logged to the session transcript.

Command reference

Command	What it does
`attach [--cdp <wsUrl>]`	Attach to the visible in-app browser tab. You watch live; take over anytime.
`detach`	Release the browser tab back to you immediately.
`open <url>`	Navigate the attached tab (or headless browser) to a URL.

Command	What it does
`click <selector>`	Click an element by CSS selector.
`type <selector> <text>`	Type into an input field or textarea.
`press <key>`	Press a key: Enter, Tab, Escape, and similar.
`screenshot [path]`	Capture the active page to a PNG file.
`get-text [selector]`	Extract inner text for the agent to analyze.
`eval "<code>"`	Evaluate a JavaScript expression in the page.
`status`	Show active URL, title, tab count, mode (attached or headless), and connection state.
`start` / `stop`	Start or terminate the headless Chromium daemon. `stop` also detaches from the browser tab.
`tabs` , `new-tab` , `switch-tab` , `close-tab`	Headless-only tab management. Disabled while attached to your visible tab.

A typical session

1. You open the browser tab and navigate to a form, for example a signup page.
2. In chat you write: `fill this form with the details from my notes` .
3. The agent runs `pocket-browser attach` and tells you: `Attached. I am now driving your visible tab. You can take over at any moment.`
4. The orange banner appears. You watch each field fill and each click happen live.
5. The agent runs `pocket-browser detach` and summarizes what it did. The transcript holds the full action log.
6. If anything looks wrong mid-way, you tap the page. Control is yours instantly.

Safety model

> **Five guarantees, enforced in code, not in prose.**
> 1. **Explicit ask only.** The agent attaches only when you asked for browser work in chat.
> 2. **Always visible.** The driving banner is on screen for the entire attach; there is no silent mode.
> 3. **Instant take-over.** The Take over button or a single tap reclaims the tab immediately.
> 4. **Bounded sessions.** Attach state expires after 15 minutes; discovery data after 60 seconds.
> 5. **Full audit.** Every driven action is written to the session transcript with a timestamp.

Honest limitations

- The browser tab must be open with a page loaded. If there is nothing to attach to, `attach` fails and tells you to open a page first.
- Only the visible page target is driven; the agent cannot reach your other tabs, history, or bookmarks.
- CDP attach against a physical device is implemented but its on-device behavior is pending 3.2.0 QA; treat the first runs as supervised.
- Discovery requires the Linux runtime to be installed, because the discovery and attach state files live under the runtime's `tmp` directory.
- WebView remote debugging is a developer setting on your own device. If you are uncomfortable with any app-level debugging surface, turn the Settings toggle off; the feature degrades to the headless mode documented in the skill.

This chapter documents the Synced Agent Browser as shipped in Conzyx 3.2.0 (`com.codeair.mhe.browser.AgentBrowserDrive`, the `pocket-browser attach` / `detach` commands, the driving banner in `WebBrowserScreen`, and the **Let agents drive the browser tab** toggle). It extends the **Preview Browser** chapter; the rest of this manual reflects version 3.1.0.